



The Show Verse: plataforma web de gestión y seguimiento de películas y series

Trabajo de Fin de Grado

Convocatoria: Abril 2026

Alumno/a: Pablo Santos Hernández

Tutor/a: Gonzalo Bailador del Pozo

Grado: Ingeniería del Software

Índice general

Índice de figuras **6**

Índice de tablas **7**

Resumen **8**

Abstract **9**

1 Introducción **10**

1.1 Justificación y contexto 10

1.1.1 Motivación 10

1.2 Planteamiento del problema 11

1.3 Objetivos del trabajo 12

1.4 Modelo de negocio 13

1.4.1 Viabilidad real y dependencia de APIs 13

1.4.2 Limitaciones por proveedor (coste y condiciones) 13

1.4.3 Modelos de ingreso viables 14

1.4.4 Conclusión de viabilidad 14

2 Estado de la cuestión **15**

2.1 Marco teórico del trabajo 15

2.1.1 Las plataformas de streaming y sus consecuencias para el usuario 15

2.1.2 Autenticación y APIs REST 15

2.1.3 Rendimiento web 16

2.1.4 Diseño de interfaz 17

2.2 Trabajos relacionados 18

3 Aspectos metodológicos **21**

3.1 Metodología 21

3.1.1 Criterios de integración de APIs 21

3.1.2 Criterios de decisión para la autenticación 22

3.1.3 Estrategia de caché . . . 22
3.2 Tecnologías empleadas . . . 24
3.3 Gestión del desarrollo y control de versiones . . . 24
3.3.1 Flujo de trabajo con GitHub . . . 24
3.4 Proceso de despliegue en producción . . . 24
3.4.1 Fase inicial: plataforma Vercel . . . 25
3.4.2 Proceso de deployment en Vercel . . . 25
3.4.3 Despliegue actual: servidor NAS propio . . . 25
3.4.4 GitHub Actions . . . 26

4 Desarrollo del trabajo . . . 27
4.1 Arquitectura del sistema . . . 27
4.1.1 Estructura de capas . . . 27
4.1.2 Estructura de carpetas . . . 27
4.2 Implementación del sistema de autenticación OAuth 2.0 . . . 28
4.3 Integración con las APIs externas . . . 29
4.3.1 The Movie Database (TMDb) API v3 . . . 29
4.3.2 OMDb API (Open Movie Database) . . . 31
4.3.3 Proxy de Trakt y transformación de datos . . . 31
4.3.4 JustWatch (API GraphQL no oficial) . . . 32
4.3.5 Plex Media Server . . . 32
4.4 Arquitectura de API Routes propias . . . 33
4.4.1 Funciones principales de las API Routes . . . 33
4.4.2 Categorías funcionales de endpoints . . . 34
4.4.2.1 Autenticación y gestión de sesiones . . . 34
4.4.2.2 Datos de usuario y actividad . . . 34
4.4.2.3 Valoraciones y ratings . . . 35
4.4.2.4 Proxy y caché de datos externos . . . 35
4.4.2.5 Integración con Plex Media Server . . . 35
4.4.2.6 Enlaces externos y descubrimiento . . . 36
4.4.2.7 Listas y colecciones . . . 36
4.4.3 Patrones técnicos transversales . . . 37
4.5 Implementación de los módulos funcionales . . . 37
4.5.1 Dashboard principal (`/`) . . . 37
4.5.2 Página de detalles (`/details/[type]/[id]`) . . . 38
4.5.2.1 Arquitectura de datos . . . 38

4.5.2.2 Menú de navegación por secciones . . . 39
4.5.2.3 Sistema de puntuaciones . . . 40
4.5.2.4 Botones de acción . . . 40
4.5.2.5 Gestión avanzada de episodios para series . . . 41
4.5.2.6 Enlaces externos y plataformas de streaming . . . 42
4.5.2.7 Imágenes y posters . . . 43
4.5.2.8 Sección de reparto y ficha de personas . . . 44
4.5.2.9 Comentarios, análisis de sentimientos y listas de Trakt . . . 45
4.5.2.10 Recomendaciones y contenido similar . . . 45
4.5.3 Favoritos (`/favorites`) . . . 46
4.5.4 Pendientes (`/watchlist`) . . . 49
4.5.5 Historial de visionado (`/history`) . . . 49
4.5.6 En Progreso (`/in-progress`) . . . 50
4.5.7 Estadísticas del usuario (`/stats`) . . . 51
4.5.8 Calendario de estrenos (`/calendar`) . . . 53
4.5.9 Biblioteca Personal (`/biblioteca`) . . . 53
4.5.10 Descubrir contenido (`/discover`) . . . 55
4.5.11 Gestión de listas (`/lists`) . . . 55
4.5.12 Películas y Series (`/movies`, `/series`) . . . 57
4.5.13 Ficha de personas (`/details/person/[id]`) . . . 59
4.5.14 Barra de navegación (Navbar) . . . 60
4.5.15 Patrones de optimización transversales . . . 62
4.6 Diseño de la interfaz . . . 62
4.6.1 Sistema de diseño global . . . 62
4.6.2 Componentes de Next.js utilizados . . . 63
4.6.3 Componentes propios: Navbar . . . 64
4.6.4 Componente propio: LiquidButton . . . 65
4.6.5 Componente propio: StarRating . . . 66
4.6.6 Componente propio: DetailsSectionMenu . . . 67
4.6.7 Componente propio: EpisodeRatingsGrid . . . 67
4.6.8 Componente propio: ScoreboardBar . . . 68
4.6.9 Componentes propios: control de visionado en Trakt . . . 69
4.6.10 Componentes propios: modales de apoyo en DetailsClient . . . 70
4.6.11 Sistema de animaciones con Framer Motion . . . 70
4.6.12 Sistema de estilos global (`globals.css`) . . . 72
4.6.13 Sistema de tres vistas (Grid, List, Compact) . . . 72

4.7 Rendimiento en producción . . . 73
4.8 Sistema de caché multinivel . . . 73
4.8.1 Nivel 1: Incremental Static Regeneration (ISR) . . . 73
4.8.2 Nivel 2: Caché en memoria del proceso Node.js . . . 74
4.8.3 Nivel 3: Deduplicación de peticiones concurrentes . . . 74
4.8.4 Nivel 4: Caché en el navegador (sessionStorage) . . . 74
4.9 Middleware y optimización SEO . . . 75
4.9.1 Detección de bots y reescritura de rutas . . . 75
4.9.2 Generación de HTML optimizado para compartir . . . 75
4.9.3 Metadatos globales y por página . . . 76
4.9.4 Resumen de la arquitectura dual de rutas . . . 76
4.10 Sistema de artwork personalizado . . . 77
4.10.1 Motivación . . . 77
4.10.2 Funcionamiento general . . . 77
4.11 Integración con comunidad Trakt . . . 78
4.11.1 Sistema de comentarios . . . 78
4.11.2 Listas de la comunidad . . . 79
4.11.3 Estadísticas de temporada . . . 79
4.12 Integración con Letterboxd . . . 79
4.12.1 Búsqueda de películas en Letterboxd . . . 79
4.12.2 Botón de acceso rápido . . . 80
4.13 Manejo de errores y estados de carga . . . 80
4.13.1 Estrategia de *graceful degradation* . . . 80
4.13.2 Skeleton loaders . . . 81
4.13.3 Mensajes de error contextuales . . . 81
4.13.4 Estados vacíos . . . 81
4.14 Diseño responsive y adaptación móvil . . . 82
4.14.1 Breakpoints y estrategia mobile-first . . . 82
4.14.2 Barra de navegación adaptativa . . . 82
4.14.3 Imágenes responsive . . . 83
4.15 Plan de calidad: estrategia de testing . . . 83

5 Conclusiones . . . 85
5.1 Logros alcanzados . . . 86
5.2 Discusión . . . 87
5.3 Análisis de riesgos y estrategias de mitigación . . . 87

5.3.1 Riesgos técnicos 88
5.3.2 Riesgos legales y de privacidad 88
5.3.3 Riesgos de producto y adopción 88
5.3.4 Síntesis operativa 89

Referencias **91**

Glosario **92**

Anexo **93**

Índice de figuras

2.1 Interfaz principal de Letterboxd. 18
2.2 Interfaz principal de Trakt.tv. 19
2.3 Interfaz principal de Simkl. 19
2.4 Interfaz principal de JustWatch. 20

4.1 Interfaz de gestión de portadas y fondos. 44
4.2 Interfaz de la sección de favoritos. 48
4.3 Vista tipo lista de la sección de favoritos. 48
4.4 Interfaz de la sección de pendientes. 49
4.5 Interfaz de la sección de historial de visionado. 50
4.6 Interfaz de la sección de series en progreso. 51
4.7 Interfaz de la sección de estadísticas del usuario (vista general y vista de patrones). 52
4.8 Interfaz del calendario de estrenos de películas. 53
4.9 Interfaz de la biblioteca personal integrada con Plex Media Server. 54
4.10 Interfaz de la sección Descubrir contenido. 55
4.11 Interfaz de las listas públicas de Trakt. 56
4.12 Interfaz de la sección de colecciones. 57
4.13 Interfaz de la sección de películas. 58
4.14 Interfaz de la sección de series. 58
4.15 Interfaz de la ficha de persona. 60
4.16 Barra de navegación en escritorio. 61
4.17 Barra de navegación en móvil. 61
4.18 Botones de acción implementados con el componente LiquidButton. 66
4.19 Interfaz del componente de valoración personal StarRating. 67
4.20 Interfaz del componente DetailsSectionMenu. 67
4.21 Interfaz del componente EpisodeRatingsGrid. 68
4.22 Interfaz del componente ScoreboardBar. 69
4.23 Interfaz del modal TraktEpisodesWatchedModal para gestionar episodios vistos. . . 69
4.24 Interfaz del modal TraktWatchedModal con el historial de visionado. 70

Índice de tablas

3.1 Estrategia de caché por tipo de dato 23

4.1 Rutas principales de la aplicación 28
4.2 Principales funciones del cliente TMDb y sus *endpoints* 30
4.3 Principales *endpoints* de Trakt utilizados 32
4.4 Secciones del dashboard y sus fuentes de datos 38
4.5 Métricas Lighthouse en producción 73
4.6 Breakpoints responsive 82

5.2 Variables de entorno del proyecto 93
5.3 Sistema de caché multicapa 94

Resumen

El presente Trabajo de Fin de Grado aborda el diseño y la implementación de *The Show Verse*, una aplicación web avanzada de gestión y descubrimiento de películas y series que permite a los usuarios unificar todo el contenido en un solo lugar. La plataforma cuenta con secciones personalizadas para cada usuario gestionadas a través de APIs externas.

La dispersión del contenido entre las distintas plataformas de *streaming* genera una experiencia de usuario fragmentada y dificulta el seguimiento del contenido consumido. *The Show Verse* soluciona este problema ofreciendo una plataforma que unifica el contenido de las principales páginas de puntuaciones y plataformas de *streaming*.

El trabajo consiste en el desarrollo completo de una aplicación web en Next.js que integra las APIs de TMDb, Trakt.tv, OMDb, JustWatch y Plex. Se ha implementado un flujo de autenticación completo para permitir la sincronización del historial, los favoritos, la lista de pendientes, las estadísticas y las valoraciones del usuario.

Desde el punto de vista de la arquitectura, la solución combina Server Components para la carga inicial eficiente de datos y Client Components para la interacción en tiempo real, junto con API Routes propias que actúan como capa intermedia para normalizar respuestas, gestionar errores y aplicar estrategias de optimización. A nivel de diseño, se ha construido una interfaz *responsive* orientada a distintos dispositivos y con navegación adaptativa.

Entre las funcionalidades más relevantes destacan el control detallado del visionado de series, permitiendo marcar episodios como vistos y mostrar de forma automática cuánto se ha avanzado en cada temporada, la consulta de valoraciones por episodio y la sincronización con Trakt y TMDb para que acciones como puntuar, guardar en favoritos o añadir a pendientes se reflejen tanto en la aplicación como en la cuenta externa del usuario.

La plataforma incorpora además enlaces externos para acceder a los recursos en otras plataformas, un sistema con varios modos de visualización en todas las listas y también se integran contenidos de comunidad, como comentarios y listas públicas, que facilitan descubrir recomendaciones de otros usuarios. En conjunto, estas capacidades convierten a *The Show Verse* en una plataforma unificada, escalable y centrada en la usabilidad para la gestión del contenido personal.

Abstract

This Final Degree Project addresses the design and implementation of *The Show Verse*, an advanced web application for managing and discovering films and TV series that allows users to bring all content together in a single place. The platform includes personalized sections for each user, managed through external APIs.

The dispersion of content across different *streaming* platforms creates a fragmented user experience and makes it difficult to track consumed content. *The Show Verse* solves this problem by offering a platform that unifies content from the main rating websites and *streaming* platforms.

The work consists of the full development of a Next.js web application that integrates TMDb, Trakt.tv, OMDb, JustWatch, and Plex APIs. A complete authentication flow has been implemented to enable synchronization of the user's history, favorites, watchlist, statistics, and ratings.

From an architectural perspective, the solution combines Server Components for efficient initial data loading and Client Components for real-time interaction, together with custom API Routes that act as an intermediate layer to normalize responses, handle errors, and apply optimization strategies. At the design level, a responsive interface has been built for different devices, with adaptive navigation.

Among the most relevant features are detailed TV-series watch tracking, allowing episodes to be marked as watched and automatically showing progress in each season, consultation of per-episode ratings, and synchronization with Trakt and TMDb so that actions such as rating, saving to favorites, or adding to watchlist are reflected both in the application and in the user's external account.

The platform also incorporates external links to access resources on other platforms, a system with several display modes across all lists and community content integration, such as comments and public lists, which helps users discover recommendations from others. Overall, these capabilities make *The Show Verse* a unified, scalable, and usability-focused platform for personal content management.

Capítulo 1
Introducción

1.1 Justificación y contexto

En los últimos años, el consumo de películas y series ha cambiado radicalmente debido a que la mayoría del contenido se distribuye a través de plataformas de *streaming* como Netflix, HBO Max, Disney+, Amazon Prime Video o Movistar+, cada una con su propio catálogo independiente, lo que genera un problema habitual para cualquier usuario porque el contenido está disperso en múltiples servicios y no existe ninguna herramienta que permita gestionarlo todo desde un único lugar.

Esto provoca situaciones como perder el seguimiento de una serie que se dejó a medias, no recordar qué episodios se han visto, tener que buscar en varios sitios a la vez si una película está disponible en *streaming*, consultar las puntuaciones y críticas de otros usuarios, o simplemente no tener un registro de lo que ya se ha visto. Las plataformas de *tracking* que existen actualmente, como Trakt.tv o Letterboxd, resuelven parte del problema pero presentan sus propias limitaciones con falta de funcionalidades.

The Show Verse nace como respuesta a esa necesidad ofreciendo una plataforma web que centraliza gran parte de la gestión del contenido audiovisual, integrando los datos de múltiples fuentes mediante APIs externas y ofreciendo al usuario una experiencia completa y funcional.

Desde el punto de vista técnico, el proyecto aborda varios retos propios del desarrollo web como la integración de APIs REST de terceros con distintos esquemas de autenticación y formatos de datos, la implementación de la autenticación para el acceso con TMDb y Trakt.tv, el diseño de una estrategia de caché en varios niveles para optimizar el rendimiento, y la construcción de una interfaz *responsive* con un diseño propio y cómodo para el usuario.

1.1.1 Motivación

El interés por el desarrollo de este proyecto surge de la experiencia personal como consumidor habitual de películas y series a través de diversas plataformas de *streaming* y la dificultad que hay para gestionar el consumo personal del contenido a través de los diferentes servicios disponibles.

Es por ello que resulta complicado recordar qué títulos se han visualizado, cuáles han sido las valoraciones asignadas o qué contenido queda pendiente por ver y cuáles están en progreso. A esta problemática se suma la necesidad de consultar información adicional (como críticas, puntuaciones o detalles técnicos) en diferentes páginas o *webs* independientes, lo que genera una experiencia fragmentada y poco eficiente si habitualmente tienes que estar consultando diferentes sitios para un mismo propósito.

Las aplicaciones existentes en el mercado (Letterboxd, Trakt.tv, Simkl) ofrecen soluciones parciales, pero ninguna combina todos los datos y funcionalidades realmente necesarios a la hora de gestionar este tipo de contenido. Algunas destacan por su capacidad de seguimiento, mientras que otras priorizan el diseño visual o la experiencia de usuario, pero no logran combinar de manera equilibrada todas las funcionalidades necesarias para una gestión completa del consumo audiovisual.

1.2 Planteamiento del problema

El problema abordado en este trabajo puede descomponerse en varias categorías técnicas.

1. Integración de fuentes de datos. Los datos necesarios para una plataforma de gestión de contenido audiovisual completa no están disponibles en una única API. TMDb proporciona los metadatos de películas y series (sinopsis, reparto, imágenes, trailers), además de las puntuaciones y listas de usuario. Trakt.tv gestiona el *tracking* personal del usuario (historial de visionado, sección en progreso y estadísticas). OMDb complementa con las puntuaciones de IMDb y Rotten Tomatoes además de los premios y nominaciones de cada título. Cada API tiene su propio esquema de autenticación, formato de respuesta, límites de uso e identificadores.

2. Autenticación y gestión de sesiones. La funcionalidad de sincronización con Trakt requiere que el usuario delegue permisos en la aplicación mediante OAuth 2.0, lo que implica implementar el flujo completo de autorización (redirección, intercambio de código por token, almacenamiento seguro, actualización automática), sin exponer en ningún momento las credenciales del usuario ni las claves privadas de la aplicación.

3. Rendimiento y experiencia de usuario. Este tipo de aplicación maneja grandes volúmenes de imágenes de alta resolución y realiza múltiples llamadas a APIs externas con límites de peticiones en algunos casos reducidos. Es importante que haya un equilibrio entre datos almacenados en caché y aquellos que se muestran en tiempo real.

4. Sincronización de estado entre servidor y cliente. Las acciones del usuario (marcar como visto, añadir a favoritos, puntuar) deben reflejarse de forma inmediata en la interfaz sin esperar la confirmación del servidor, pero manteniendo la coherencia en caso de error.

1.3 Objetivos del trabajo

El objetivo principal es diseñar e implementar una aplicación web completa y funcional para la gestión personalizada y el descubrimiento de nuevas películas y series, que integre múltiples APIs externas y ofrezca una experiencia de usuario centralizada, moderna y eficiente.

Objetivos específicos:

1) Implementar la integración con la API de TMDb para obtener los metadatos completos de películas y series (sinopsis, características, imágenes, vídeos, temporadas y episodios).

2) Desarrollar un sistema de autenticación OAuth 2.0 completo con Trakt.tv, incluyendo almacenamiento seguro de tokens en cookies `httpOnly`, actualización automática mediante *refresh tokens* y revocación.

3) Construir un módulo de gestión personal (favoritos, pendientes, historial de visionado, series en progreso y puntuaciones de usuario) con sincronización bidireccional con Trakt y TMDb.

4) Implementar un sistema de seguimiento de episodios completo para series, incluyendo marcado individual por episodio, marcado completo por temporada, visualización de progreso y tabla de puntuaciones por episodio y temporada.

5) Diseñar una interfaz de usuario responsiva y accesible que ofrezca múltiples modos de visualización (Grid, List, Compact) con transiciones fluidas mediante Framer Motion y compatibilidad con dispositivos móviles.

6) Optimizar el rendimiento de la aplicación mediante un sistema de caché multinivel (ISR en servidor, memoria de proceso Node.js, sessionStorage, localStorage) con deduplicación en vuelo para reducir el tráfico API.

7) Implementar un *middleware* Edge para optimización SEO, generación de metadatos Open Graph y reescritura de URLs para garantizar una indexación correcta.

8) Integrar Plex Media Server para el acceso a la biblioteca local del usuario, incluyendo autenticación JWT, enlaces directos multiplataforma y búsqueda unificada.

9) Desarrollar componentes interactivos avanzados con efectos visuales y sistema de actualización optimista para todas las acciones de usuario.

10) Implementar una gestión de errores con *graceful degradation* y *skeleton loaders* para garantizar la funcionalidad parcial aunque fallen APIs externas.

11) Desplegar la aplicación en un entorno de producción (Vercel).

1.4 Modelo de negocio

The Show Verse se ha desarrollado como proyecto académico, pero su arquitectura permite analizar su viabilidad en un escenario real. No obstante, al tratarse de una aplicación fuertemente dependiente de APIs de terceros, la sostenibilidad económica no puede evaluarse solo por costes de infraestructura, sino que también está condicionada por licencias de uso, límites de petición y cambios de política de proveedores externos.

1.4.1 Viabilidad real y dependencia de APIs

Desde una perspectiva técnica y de producto, el proyecto es viable como servicio de nicho orientado a usuarios avanzados que desean centralizar seguimiento, valoraciones y descubrimiento en una única interfaz. Sin embargo, su potencial de monetización está limitado por la naturaleza de los datos consumidos: una parte relevante del valor funcional procede de metadatos y catálogos de plataformas externas cuya explotación comercial no depende exclusivamente del desarrollador.

En consecuencia, un despliegue público a pequeña escala es factible, pero una monetización agresiva (suscripción, publicidad intensiva o reventa de datos agregados) requeriría revisar de forma previa la conformidad legal y contractual con cada proveedor.

1.4.2 Limitaciones por proveedor (coste y condiciones)

Para analizar la viabilidad con mayor precisión, conviene considerar cada proveedor de forma individual.

TMDb. Aunque su uso básico no se articula en una tarifa pública estándar para proyectos pequeños, cualquier explotación comercial exige revisar de manera específica las condiciones del proveedor. En la práctica, esto implica que no puede asumirse una monetización directa y estable sin un marco contractual explícito.

Trakt.tv. La API pública permite construir funcionalidades avanzadas, pero su disponibilidad está sujeta a límites de uso y a la propia evolución del ecosistema de Trakt. Además, la existencia de planes VIP para usuarios finales (en torno a 4–6 USD/mes) condiciona indirectamente el alcance de ciertas experiencias premium dentro de la plataforma.

OMDb. Su modelo de acceso mediante planes públicos en Patreon (aprox. 1 EUR/mes, 4,5 EUR/mes y 9 EUR/mes) introduce un coste recurrente asumible para fases iniciales. No obstante, también añade una dependencia económica y operativa que debe contemplarse en la planificación a medio plazo.

JustWatch. La integración se apoya en un ecosistema orientado a acuerdos B2B, con condiciones económicas no publicadas de forma abierta. Por ello, cualquier uso comercial robusto de esta capa requiere negociación previa con el proveedor.

En conjunto, la principal restricción no es tanto el coste unitario de hosting, sino la incertidumbre contractual de dos piezas críticas del producto: si existe explotación comercial y si se desea cobertura robusta de disponibilidad en plataformas.

1.4.3 Modelos de ingreso viables

Con el marco técnico actual, los modelos más razonables serían los siguientes:

- **Freemium conservador**: capa gratuita funcional y un plan de pago centrado en productividad personal (exportaciones, automatizaciones, informes), evitando vender el acceso a datos de terceros como propuesta principal.
- **Servicios para power users**: configuración avanzada de sincronizaciones (Plex/Trakt), backups, reglas personalizadas y soporte prioritario.
- **Afiliación limitada**: enlaces salientes a plataformas bajo acuerdos explícitos, siempre como canal complementario y no como base financiera única.

1.4.4 Conclusión de viabilidad

La viabilidad del proyecto es media para un producto independiente y alta como plataforma técnica demostrable, siempre que se mantenga un crecimiento controlado y una estrategia de costes prudente. La principal barrera para monetizar no es la complejidad de implementación, sino la gobernanza de dependencias externas: condiciones de uso, posibles acuerdos comerciales y cambios de política de APIs.

Por tanto, antes de cualquier lanzamiento comercial, el paso crítico no sería añadir más funcionalidades, sino formalizar un marco de cumplimiento con proveedores y revisar periódicamente costes y límites vigentes, ya que estos parámetros pueden variar en el tiempo.

Capítulo 2
Estado de la cuestión

El presente estado de la cuestión aborda el contexto tecnológico y de mercado en el que se enmarca el proyecto. Se comienza con una visión del sector de las plataformas de *streaming* y sus implicaciones para el usuario; a continuación se analizan las tecnologías de desarrollo web actuales que hacen posible la solución propuesta; y se finaliza examinando las plataformas existentes más relevantes.

2.1 Marco teórico del trabajo

2.1.1 Las plataformas de streaming y sus consecuencias para el usuario

La transición del modelo tradicional al *streaming* bajo demanda se inició con el lanzamiento del servicio de Netflix en 2007, aunque su consolidación definitiva se produjo a partir de 2019, con la irrupción simultánea de Disney+, Apple TV+ y HBO Max. En 2024, el usuario medio en mercados como España o Estados Unidos tiene acceso simultáneo a entre 3 y 5 servicios de *streaming* distintos, según datos de CNMC y Statista (CNMC, 2024; Statista, 2024).

Esta fragmentación genera una sobrecarga de elección en cada usuario, lo que provoca que inviertan una gran cantidad de tiempo en decidir qué ver, olviden mantener un registro de lo visto y pierdan el seguimiento de las series que consumen de forma no continua (Schwartz, 2004).

La necesidad de una capa de abstracción sobre las plataformas de *streaming* ha generado el desarrollo de aplicaciones conocidas como **media trackers**.

2.1.2 Autenticación y APIs REST

Se hace uso del estándar OAuth 2.0 para la autenticación con Trakt.tv, que es el servicio que ofrece la funcionalidad de seguimiento más completa y que actúa como punto central de la gestión del contenido del usuario. TMDb también ofrece soporte para autenticación mediante OAuth 2.0, aunque en este caso se ha optado por un sistema de autenticación basado en API Key para simplificar la integración y evitar la complejidad adicional de gestionar tokens de acceso y renovación.

Las APIs utilizadas en este proyecto siguen el estilo REST con las siguientes especificidades:

TMDb API v3 es una API REST pública con autenticación por API Key. Proporciona acceso a su base de datos de más de 800.000 películas y 150.000 series de televisión, con soporte para más de 40 idiomas. Su límite de peticiones es de 40 peticiones por 10 segundos para cuentas gratuitas.

Trakt.tv API v2 combina autenticación por API Key para *endpoints* públicos con OAuth 2.0 Bearer Token para *endpoints* autenticados. Un aspecto técnico relevante es el sistema de identificadores: Trakt mantiene sus propios slugs e IDs, pero acepta identificadores de IMDb y TMDb mediante el *endpoint* de búsqueda por ID externo.

OMDb API se utiliza como fuente complementaria para los metadatos y puntuaciones externas vinculadas al identificador de IMDb. Su integración se realiza mediante peticiones de lectura a través de la API interna del proyecto para unificar el formato de respuesta y control de errores.

JustWatch se integra para obtener enlaces de disponibilidad en plataformas de *streaming*. En este caso no se consume una API REST pública oficial, sino que se encapsula la consulta en rutas internas del proyecto que normalizan parámetros, validan resultados y aplican caché antes de exponer el enlace en la interfaz.

Plex Media Server se incorpora como fuente del catálogo de contenido multimedia local del usuario. La comunicación se realiza por HTTP/HTTPS contra el servidor Plex configurado por el usuario y se protege mediante autenticación basada en token, lo que permite combinar en una misma vista contenido personal y datos procedentes de servicios en la nube.

2.1.3 Rendimiento web

El rendimiento de *The Show Verse* se plantea de forma global, no solo para una vista concreta. La aplicación combina páginas con alta densidad de información, por lo que la estrategia prioriza mantener tiempos de respuesta estables cuando aumentan simultáneamente el número de elementos en pantalla y el volumen de llamadas a APIs.

En la capa de renderizado se adopta un enfoque híbrido de SSR/ISR según la criticidad y frecuencia de cambio de los datos. Las vistas principales se sirven con el contenido inicial desde el servidor y revalidación periódica, mientras que los bloques secundarios se cargan de forma diferida en el cliente para no penalizar el primer renderizado. En paralelo, la agregación de datos se resuelve con `Promise.all` y límites de tiempo acotados, evitando que una dependencia externa lenta bloquee la página completa.

Este criterio se aplica tanto en los dashboards y la página de detalle como en las pantallas con listados extensos del usuario. En módulos como Historial, Favoritos, Pendientes y En progreso, la interfaz mantiene la fluidez mediante la combinación de filtrado y ordenación en cliente con cálculos

memorizados, cachés de apoyo para puntuaciones e imágenes y reutilización de resultados cuando el usuario cambia entre modos de vista o vuelve a una sección ya consultada. Así se reduce trabajo repetido de renderizado y se evita recargar información ya resuelta.

Como criterio general, el proyecto guarda temporalmente información en varios niveles (servidor, API y navegador) para evitar repetir la misma consulta muchas veces en poco tiempo. Además, cuando varios componentes piden el mismo dato a la vez, la aplicación intenta resolverlo con una sola petición y reutilizar el resultado. En la práctica, esto reduce carga innecesaria sobre servicios externos con límites de uso, mejora la estabilidad cuando hay más tráfico y mantiene una navegación fluida en páginas con muchos elementos.

2.1.4 Diseño de interfaz

El diseño de interfaz de *The Show Verse* utiliza un tema oscuro general para destacar los carteles (*posters*) y fondos (*backdrops*) del contenido teniendo como referencia las plataformas de *streaming* actuales.

Vistas personalizadas. Las principales secciones de la aplicación ofrecen múltiples modos de visualización: cuadrícula (*grid*), lista (*list*) o vista compacta (*compact*). La elección del usuario se almacena en el navegador mediante *localStorage* para mantener la preferencia en futuras visitas. Además, el Historial incluye una vista específica de calendario y la sección En progreso incorpora una vista de tarjetas con barras de porcentaje para mostrar el avance de las series.

Diseño responsive. La interfaz se adapta al dispositivo del usuario. En la versión de escritorio, se utilizan efectos de animación al pasar el cursor (*hover*) por las tarjetas para desplegar menús e información. En dispositivos móviles, estos efectos se desactivan y se sustituyen por toques directos en botones adaptados a pantallas táctiles evitando problemas de navegación.

Componentes propios. Para evitar depender de librerías genéricas y mantener una identidad visual coherente, se han desarrollado componentes personalizados:

- `LiquidButton`: Botones de acción principales con diseño fluido y animado.
- `StarRating`: Sistema de valoración para el usuario.
- `DetailsSectionMenu`: Menú dinámico para organizar y navegar por las secciones dentro de la página de detalles de una película o serie.
- `ScoreboardBar`: Componente unificado que muestra las puntuaciones de las distintas plataformas externas así como los enlaces externos y una serie de estadísticas adicionales.

- `TraktWatchedModal` y `TraktEpisodesWatchedModal`: Ventanas emergentes propias para registrar directamente el visionado de películas y episodios de series en Trakt para cada usuario.

Transiciones. Se emplea la librería *Framer Motion* para animar la interfaz al aplicar filtros, cambiar de vista o desplegar información lo que evita cambios de pantalla bruscos y mejora la experiencia de usuario.

2.2 Trabajos relacionados

Letterboxd (fundado en 2011, Auckland, Nueva Zelanda) es la plataforma de referencia para el registro y la crítica cinematográfica personal. Con más de 20 millones de usuarios registrados en 2024, su principal fortaleza es la componente social y el sistema de reseñas. Su limitación más significativa para los propósitos de este trabajo es la ausencia de soporte para el seguimiento de series de televisión por episodio.

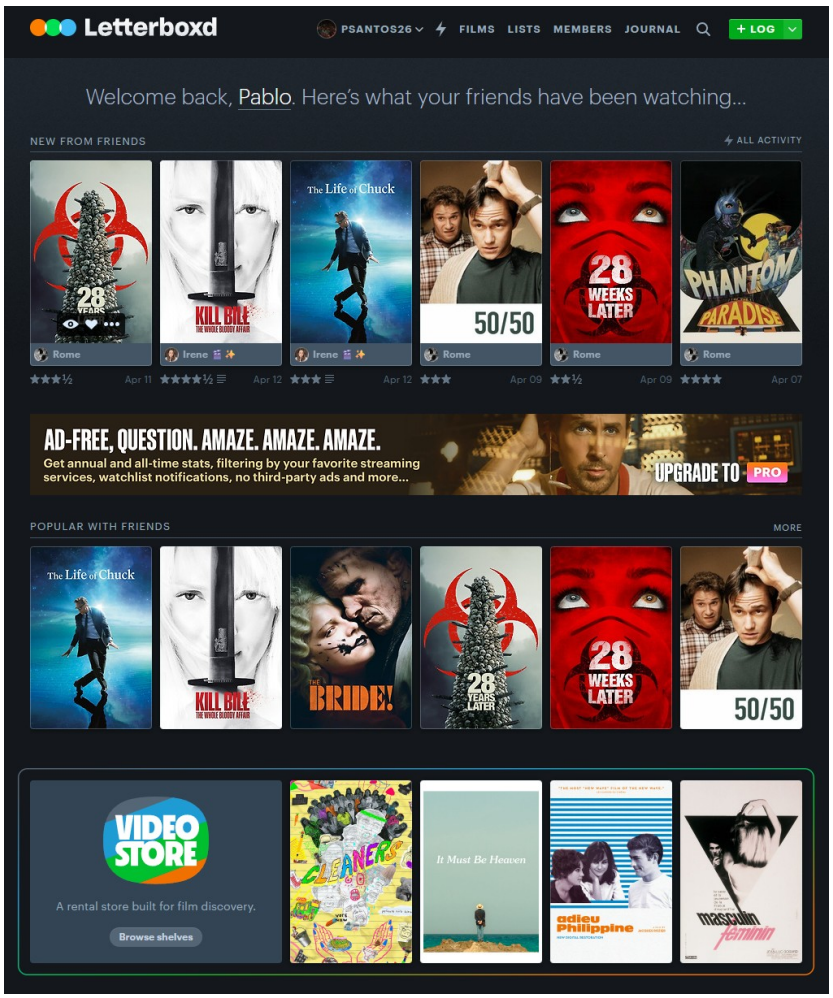


Figura 2.1: Interfaz principal de Letterboxd.

Trakt.tv (fundado en 2010) es la plataforma más completa en cuanto a funcionalidades de *tracking*, con soporte completo para películas y series a nivel de episodio. Su característica diferencial es la API pública v2, documentada y con límites de uso generosos para desarrolladores registrados. Sin embargo, su interfaz de usuario presenta un diseño anticuado que no ha evolucionado significativamente en la última década.

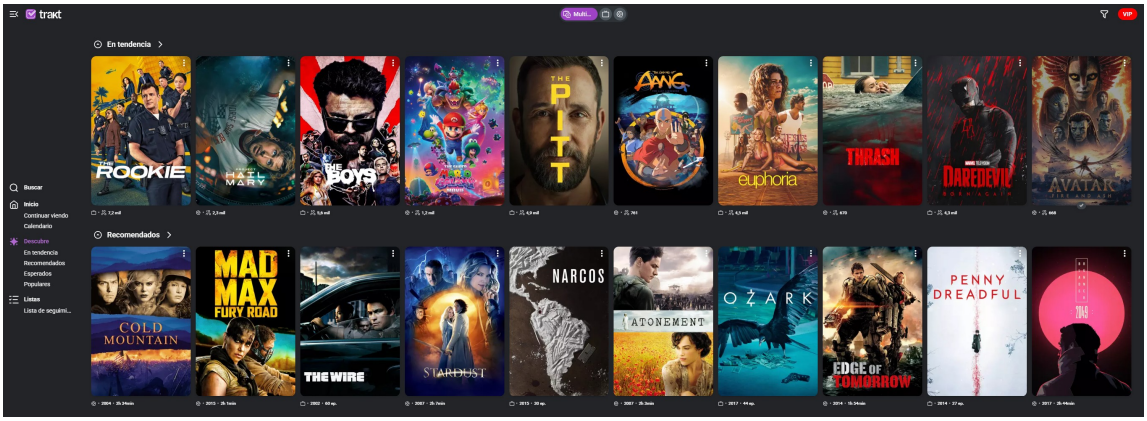


Figura 2.2: Interfaz principal de Trakt.tv.

Simkl (fundado en 2012) ofrece funcionalidades similares a Trakt con una interfaz algo menos moderna y con un ecosistema de API menos estable y una comunidad de usuarios significativamente menor.

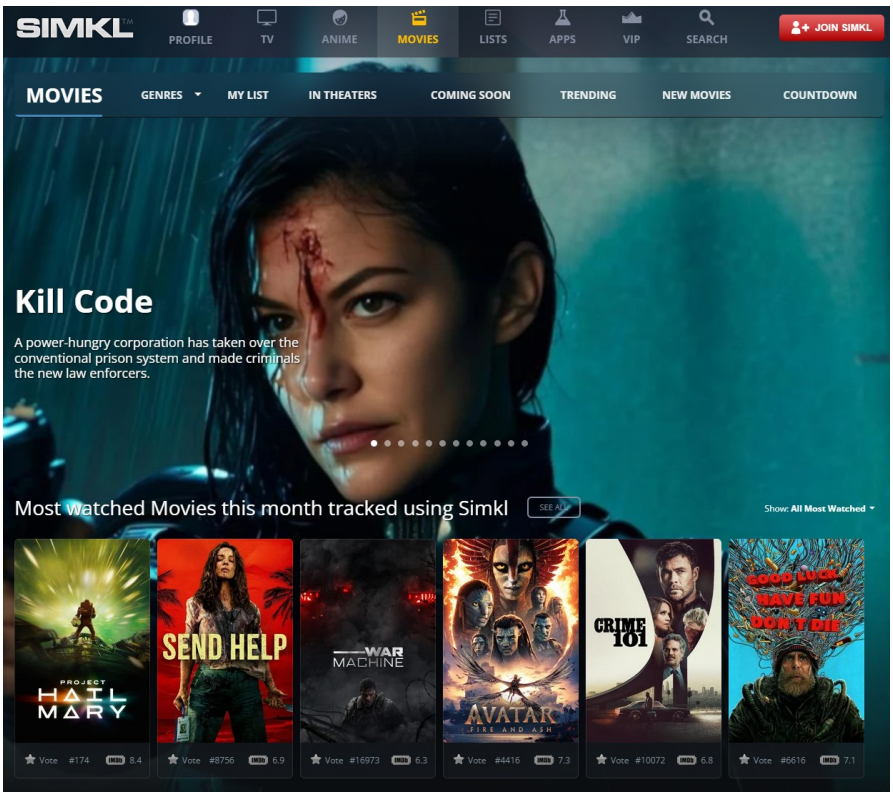


Figura 2.3: Interfaz principal de Simkl.

JustWatch (fundado en 2014) se especializa en la localización de contenido entre plataformas de *streaming*, pero carece de funcionalidades de *tracking* personal.

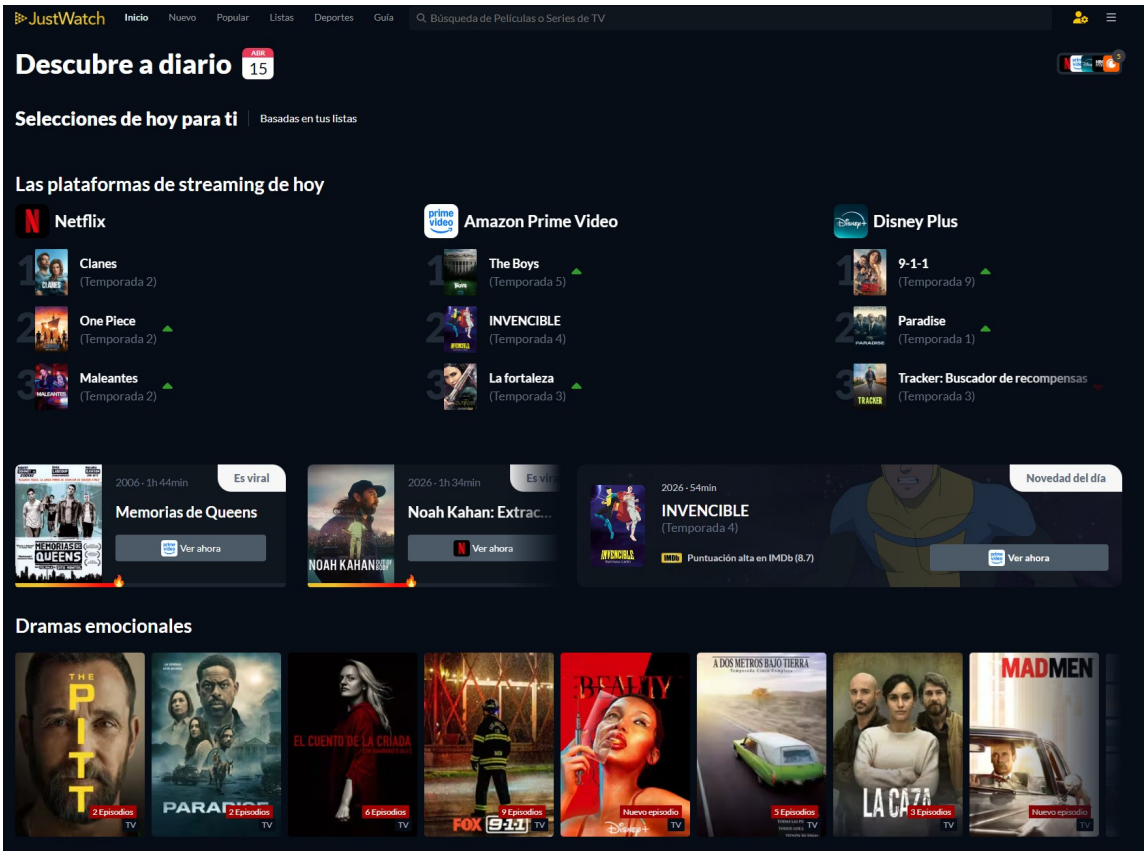


Figura 2.4: Interfaz principal de JustWatch.

Capítulo 3
Aspectos metodológicos

El desarrollo de *The Show Verse* se ha abordado mediante una metodología iterativa basada en el ciclo de vida ágil, adaptada a un proyecto individual. La elección de este enfoque frente a las metodologías en cascada se justifica por la naturaleza exploratoria del proyecto ya que la integración con APIs de terceros implica incertidumbres técnicas que no siempre se pueden anticipar en una fase de análisis y la prioridad entre funcionalidades puede variar a medida que se comprende mejor el problema.

El proceso de desarrollo se realiza en iteraciones de aproximadamente una semana de duración. Al inicio de cada iteración se seleccionan del *backlog* las funcionalidades a abordar, priorizadas según tres criterios:

- **Dependencia técnica:** las funcionalidades que sirven de base a otras (autenticación, cliente de API) se abordan primero.
- **Valor aportado al usuario:** se priorizan las funcionalidades más visibles y relevantes para la experiencia de uso.
- **Riesgo técnico:** las integraciones más inciertas se adelantan para conocer cuanto antes sus restricciones.

3.1 Metodología

3.1.1 Criterios de integración de APIs

La selección de las APIs externas ha seguido los siguientes criterios:

- TMDb fue seleccionada frente a otras alternativas por ofrecer la mayor cobertura de contenido, soporte en diferentes idiomas, galería de imágenes y datos de *streaming*, además de disponer de la documentación más completa y una comunidad de desarrolladores activa.

- Trakt.tv fue seleccionada como plataforma de sincronización por su API pública v2 bien documentada, su soporte completo de series a nivel de episodio y por ser la plataforma de *tracking* más referenciada por la comunidad de desarrolladores independientes.
- OMDb fue seleccionada únicamente para los datos de *rating* de IMDb y Rotten Tomatoes, que TMDb no proporciona directamente.

3.1.2 Criterios de decisión para la autenticación

El problema de la autenticación OAuth 2.0 admitía varias opciones de implementación en Next.js. Se descartó el uso de librerías de autenticación genéricas como NextAuth.js / Auth.js por las siguientes razones:

- Trakt.tv no es un proveedor OAuth soportado de forma nativa por estas librerías.
- La personalización necesaria (refresco de tokens específico de Trakt, intercambio de códigos con *headers* personalizados) habría requerido más configuración que implementar el flujo directamente.
- La implementación propia ofrece mayor control sobre el manejo de errores y el almacenamiento de tokens.

Se optó por implementar el flujo Authorization Code de forma manual mediante **API Routes de Next.js**, almacenando los tokens en cookies `httpOnly` con atributos `Secure` y `SameSite=Lax`. Esta decisión garantiza que los tokens no sean accesibles desde JavaScript del navegador (protección frente a XSS) y que no se envíen en peticiones *cross-site* no intencionadas (protección parcial frente a CSRF).

3.1.3 Estrategia de caché

Para cada tipo de dato gestionado en la aplicación se ha definido una estrategia de caché en función de la frecuencia de actualización del dato en origen y su importancia para la experiencia de usuario.

Esta diferenciación permite reducir el número de llamadas a APIs externas en un estimado del 70–80 % para los datos de catálogo, mientras se garantiza la actualización inmediata de los datos de usuario que requieren frescura absoluta.

Caché en múltiples niveles: el sistema implementa cuatro niveles de caché complementarios:

1) **ISR (Incremental Static Regeneration):** páginas estáticas regeneradas periódicamente en servidor, con servicio desde caché entre regeneraciones.

Tipo de dato	Almacenamiento	TTL
Catálogo general (top rated, populares)	ISR servidor	10 minutos
Detalles de película/serie	ISR servidor	1 hora
Imágenes TMDb	CDN Vercel	1 hora
Datos OMDb (proxy servidor)	Map en proceso	24 h / 60 s
Datos OMDb en cliente	sessionStorage	24 horas
Puntuaciones IMDb y Trakt en listas	localStorage	30 días
Rating personal del usuario	Map en módulo	10 min / 45 s
Estado personal del usuario	No-cache	En cada petición

Tabla 3.1: Estrategia de caché por tipo de dato

2) **Memory cache en proceso Node.js:** `Map` global que persiste durante la vida del proceso serverless (típicamente 5–15 min en Vercel), evitando peticiones redundantes a OMDb.

3) **sessionStorage en cliente:** caché de puntuaciones que persiste durante la sesión del navegador (hasta cerrar pestaña), común entre páginas.

4) **localStorage en cliente:** caché persistente de puntuaciones con TTL de 30 días, sobrevive entre sesiones del navegador.

Deduplicación en vuelo: implementada en `lib/api/omdb.js`, si múltiples componentes solicitan el mismo dato simultáneamente, solo se realiza una petición HTTP y todas las solicitudes esperan el resultado compartido. Esto es crítico en listas de 100+ ítems donde varias tarjetas pueden necesitar el mismo `imdbId`.

Pool de concurrencia: para respetar los *rate limits* de las APIs (OMDb: 1000 peticiones/día en plan gratuito), las peticiones paralelas se limitan a 4–5 simultáneas mediante un patrón de *worker pool*. Cuando hay 100 peticiones pendientes, se ejecutan en lotes de 4, esperando a que una termine antes de lanzar la siguiente.

Algoritmo LRU para localStorage: al superar 500 entradas cacheadas, se aplica *Least Recently Used* para eliminar entradas antiguas y mantener el tamaño del almacenamiento controlado (típicamente <2 MB).

3.2 Tecnologías empleadas

Next.js 16 (Framework principal). Seleccionado frente a otras alternativas (Remix, SvelteKit, Nuxt 3) por ser el framework React con mayor adopción industrial (>40 % de cuota entre frameworks React según npm trends 2024), la mejor integración con Vercel y la documentación más extensa. El App Router de Next.js 13+ es la única opción que implementa React Server Components de forma estable y con soporte oficial.

React 19. La elección de React como biblioteca de UI es consecuencia directa de la elección de Next.js. React 19 introduce mejoras en el manejo de acciones de servidor y la promesa nativa en componentes de cliente.

Tailwind CSS 4. Seleccionado frente a CSS Modules, Styled Components o Emotion por el paradigma *utility-first* que acelera el desarrollo sin sacrificar la personalización. La versión 4, basada en PostCSS con motor Rust nativo, elimina el PurgeCSS separado e incorpora mejoras sustanciales de rendimiento en el tiempo de compilación.

Framer Motion 12. La elección de Framer Motion frente a otras librerías de animación (React Spring, CSS Animations puras) se basa en su modelo declarativo basado en variantes, su soporte nativo para animaciones de *layout* y su integración con el ciclo de vida de los componentes React mediante `AnimatePresence`.

3.3 Gestión del desarrollo y control de versiones

3.3.1 Flujo de trabajo con GitHub

El proyecto está alojado en un repositorio de GitHub con la siguiente estructura de ramas:

- **main:** rama de producción. Solo contiene código testeado y listo para desplegar. Se configura con protección de rama (require pull requests, require status checks).
- **develop:** rama de integración. Recibe todos los pulls requests de desarrollo. Se despliega automáticamente a un entorno de staging.
- **feature/*:** ramas de funcionalidades. Una rama por característica importante (feature/oauth-trakt, feature/plex-integration, etc.), ramificando desde `develop`.

3.4 Proceso de despliegue en producción

El proyecto ha pasado por dos fases de despliegue a lo largo de su desarrollo. En la fase inicial se utilizó Vercel como plataforma de hosting gestionada en la nube. Posteriormente, la aplicación

se migró a una infraestructura propia basada en un servidor NAS con acceso público a través de Cloudflare Tunnel, que constituye la configuración actualmente en producción.

3.4.1 Fase inicial: plataforma Vercel

Durante las primeras fases del desarrollo la aplicación se desplegó en Vercel, una plataforma de hosting optimizada para Next.js. Las razones de esta selección inicial son:

- **Integración nativa con Next.js:** Las características de Next.js como Image Optimization, ISR y Edge Functions se soportan de forma nativa sin configuración adicional.
- **API serverless:** Las API Routes se auto-escalan sin gestionar servidores o contenedores.
- **Edge computing:** Los Edge Functions ejecutan código más cerca del usuario, reduciendo latencia para SPA middleware.
- **Deployments automáticos:** Cada pull request genera un URL único de vista previa con el código del branch.
- **Git-based deploys:** El despliegue se dispara automáticamente al hacer push o merge a una rama configurada.
- **Monitoreo integrado:** Web Analytics, Performance Monitoring y Error Tracking nativos.

3.4.2 Proceso de deployment en Vercel

El despliegue se ejecuta automáticamente cuando se hace merge a la rama `main` en GitHub. Vercel recibe la notificación, instala dependencias, ejecuta `next build` y valida que no haya errores de compilación. Si el build es exitoso, despliega la nueva versión y actualiza el dominio, en caso de fallo, realiza un rollback automático a la versión anterior.

3.4.3 Despliegue actual: servidor NAS propio

La aplicación se despliega actualmente en un servidor propio bajo el dominio `theshowverse.com`, mediante tres componentes principales. La aplicación se empaqueta en un contenedor Docker construido con un `Dockerfile` multietapa que separa la fase de compilación de la de ejecución, reduciendo el tamaño final de la imagen. Las variables de entorno sensibles se inyectan en tiempo de ejecución mediante un archivo `.env` externo al repositorio. El acceso público se gestiona a través de Cloudflare Tunnel: un segundo contenedor ejecuta *cloudflared*, que mantiene una conexión cifrada saliente hacia Cloudflare, enrutando el tráfico del dominio al puerto

`3000` del servidor sin necesidad de exponer puertos en el router ni disponer de IP pública fija. Por último, existe una página de mantenimiento para cuando el servidor no está disponible mediante un Cloudflare Worker.

3.4.4 GitHub Actions

Los despliegues en producción se automatizan mediante un *workflow* de GitHub Actions. Cuando se hace *push* a la rama principal del repositorio, el *workflow* se conecta por SSH al NAS, descarga la nueva versión del código, reconstruye la imagen Docker y reinicia el contenedor de la aplicación. De esta forma, cada cambio confirmado en el repositorio se refleja en producción sin intervención manual.

Capítulo 4
Desarrollo del trabajo

En este capítulo se describe el desarrollo completo de *The Show Verse*, desde la arquitectura general del sistema hasta la implementación de cada módulo funcional y la integración con las APIs externas. Es la parte más importante del TFG y recoge las decisiones técnicas tomadas durante el proceso de construcción de la plataforma.

4.1 Arquitectura del sistema

4.1.1 Estructura de capas

El sistema se estructura en tres capas diferenciadas, implementadas en una única base de código Next.js:

Capa de presentación (Client Components). Componentes React que se ejecutan y se hidratan en el navegador. Gestionan el estado local de la interfaz, las interacciones del usuario y las peticiones a la capa intermedia. Los componentes de mayor complejidad son los encargados de la gestión del panel de control principal, la vista dinámica de detalles de películas o series y los módulos de contenido del usuario.

Capa de servidor (Server Components + API Routes). Los Server Components de Next.js realizan la búsqueda inicial de datos directamente en el servidor, sin coste de JavaScript para el cliente. Las rutas de la API interna actúan como *proxy* hacia los servicios externos, asegurando que los tokens de autenticación permanezcan privados en el servidor y transformando los datos al formato que requiere la interfaz para su renderizado.

Capa de datos (APIs externas). TMDb, Trakt.tv, OMDb, JustWatch y Plex son las fuentes de los datos principales del catálogo y la información del perfil del usuario.

4.1.2 Estructura de carpetas

La estructura principal se organiza en cuatro directorios bajo `src/`:

- `src/app/` — Rutas, páginas y API Routes. Cada subdirectorio corresponde a una ruta de la aplicación.
- `src/components/` — Componentes React reutilizables para el diseño de la interfaz y los módulos.
- `src/lib/` — Código reutilizable para conectar con APIs externas y funciones auxiliares comunes.
- `src/context/` — Contextos globales de React para el estado compartido entre componentes.

Las rutas principales de la aplicación son:

Ruta	**Descripción**
`/`	Dashboard principal con carrusel y secciones
`/details/[type]/[id]`	Página de detalle de película o serie
`/favorites`	Lista de favoritos del usuario (TMDb)
`/watchlist`	Lista de pendientes del usuario (TMDb)
`/history`	Historial de visionado (Trakt)
`/in-progress`	Series en progreso (Trakt)
`/stats`	Estadísticas del usuario (Trakt)
`/calendar`	Calendario de estrenos y próximos episodios
`/discover`	Búsqueda y filtrado avanzado de contenido
`/biblioteca`	Librería de contenido (servidor Plex)
`/lists`	Listas de TMDb, colecciones y listas de Trakt
`/movies`	Exploración de catálogo de películas
`/series`	Exploración de catálogo de series
`/details/person/[id]`	Ficha de actor o director

Tabla 4.1: Rutas principales de la aplicación

4.2 Implementación del sistema de autenticación OAuth 2.0

La autenticación con Trakt se ha implementado con OAuth 2.0 *Authorization Code* y una arquitectura híbrida cliente-servidor, de forma que la parte sensible del proceso queda en el backend y el frontend solo gestiona el estado de conexión.

1) Inicio del flujo de autorización. Cuando el usuario pulsa «Conectar con Trakt», la aplicación llama a una ruta interna de inicio que calcula el `origin` activo (local o producción), construye el `redirect_uri` correcto y genera un `state` con *nonce* y ruta de retorno. Después redirige al usuario a la URL oficial de autorización de Trakt. Antes de redirigir, guarda en cookies `httpOnly` temporales el `state`, la ruta a la que debe volver el usuario y el `redirect_uri` exacto usado en la solicitud.

2) Retorno desde Trakt e intercambio del código por sesión. Tras autorizar en Trakt, el proveedor redirige al callback interno de la aplicación con `code` y `state`. En esta fase el servidor valida el estado recibido frente al almacenado y si es correcto intercambia el código por `access_token` y `refresh_token` mediante una llamada servidor-a-servidor a `https://api.trakt.tv/oauth/token`. Con los tokens recibidos, el backend crea cookies `httpOnly` (token de acceso, token de refresco y fecha de expiración) y limpia las cookies temporales del flujo OAuth.

3) Verificación de sesión. Para saber si el usuario está conectado, el frontend consulta un endpoint de estado. Ese endpoint lee las cookies, comprueba si el token sigue vigente y valida la sesión contra Trakt con una petición de comprobación de usuario. Si detecta token inválido o revocado, limpia la sesión local de forma controlada.

4) Renovación automática de tokens. Además del endpoint de estado, el proyecto incluye un endpoint específico de refresco y un *hook* de cliente que mantiene los tokens sincronizados. El hook calcula caducidad con margen de seguridad, solicita renovación cuando corresponde y actualiza su persistencia local para que la experiencia de uso no se interrumpa durante la navegación.

5) Desconexión. Al cerrar sesión de Trakt, la aplicación elimina las cookies de autenticación del lado servidor y limpia el estado de conexión del cliente.

Con este diseño, la gestión real del intercambio OAuth permanece en el servidor y el navegador nunca accede directamente a credenciales privadas y solo consume un estado de autenticación ya procesado por el backend.

4.3 Integración con las APIs externas

4.3.1 The Movie Database (TMDb) API v3

El cliente de TMDb (`lib/api/tmdb.js`) encapsula toda la lógica de acceso a la API v3. La función central `tmdb(path, params, options)` implementa:

- Un **constructor de URL unificado** que añade automáticamente la API Key y el idioma por defecto (`es-ES`).

- Un **mecanismo de timeout** mediante `AbortController` (8 segundos por defecto) para evitar que peticiones lentas bloqueen la interfaz.
- **Estrategia de caché**: en servidor se usa `force-cache` con revalidación ISR para reutilizar respuestas y reducir llamadas repetidas, mientras que en cliente se emplea `no-store` para evitar datos obsoletos en interacciones en tiempo real.
- **Gestión de errores**: cuando la API devuelve un 404, la función interpreta que el recurso no existe y retorna `null` sin registrar un error. En el resto de casos, el fallo sí se registra en logs para diagnóstico y la función retorna igualmente `null` para no bloquear la interfaz.

Sobre esta función base se construyen más de 40 funciones nombradas por dominio (`fetchTopRatedMovies`, `getDetails`, `getCredits`, `getWatchProviders`, `getActorDetails`, etc.) que constituyen la API pública del módulo.

Un aspecto técnico importante es la resolución de imágenes: la función `getLogos` obtiene el logo oficial de un título aplicando un criterio de selección que prioriza logos en español, luego en inglés, luego sin idioma, seleccionando el de mayor número de votos dentro de cada grupo.

La Tabla 4.2 recoge los *endpoints* principales utilizados.

Función	**Endpoint TMDb**
`fetchTopRatedMovies()`	`GET /movie/top_rated`
`fetchTrendingMovies()`	`GET /trending/movie/week`
`getDetails(type, id)`	`GET /{type}/{id}? append_to_response=external_ids`
`getLogos(type, id)`	`GET /{type}/{id}/images`
`getCredits(type, id)`	`GET /{type}/{id}/credits`
`getWatchProviders(type, id)`	`GET /{type}/{id}/watch/providers`
`getRecommendations(type, id)`	`GET /{type}/{id}/recommendations`
`markAsFavorite({...})`	`POST /account/{id}/favorite`
`markInWatchlist({...})`	`POST /account/{id}/watchlist`
`fetchFavoritesForUser(...)`	`GET /account/{id}/favorite/movies|tv`
`fetchWatchlistForUser(...)`	`GET /account/{id}/watchlist/movies|tv`

Tabla 4.2: Principales funciones del cliente TMDb y sus *endpoints*

4.3.2 OMDb API (Open Movie Database)

OMDb es la única fuente pública que permite obtener el *rating* de IMDb, el número de votos y las puntuaciones de Rotten Tomatoes y Metacritic por `imdbId`. Para proteger la API Key, todas las peticiones a OMDb pasan por la API Route interna `/api/omdb`.

El *proxy* incorpora dos medidas para reducir carga y mejorar tiempos de respuesta. La primera es la deduplicación de peticiones en curso (`inflight Map` global): si varias tarjetas piden a la vez el mismo `imdbId`, todas comparten una única consulta a OMDb en lugar de lanzar peticiones duplicadas. La segunda es una caché en memoria por tipo de resultado, las respuestas correctas se guardan durante 24 horas y se reutilizan en solicitudes posteriores, mientras que los errores temporales se almacenan solo 60 segundos para evitar insistir de forma inmediata y permitir un nuevo intento poco después.

4.3.3 Proxy de Trakt y transformación de datos

Las API Routes de Trakt actúan como intermediario que resuelve dos problemas estructurales:

Problema de identificadores. El *frontend* trabaja exclusivamente con `tmdbId`, mientras que muchos *endpoints* de Trakt requieren sus propios identificadores (`traktId`, `slug`). Las API Routes realizan internamente la traducción mediante el *endpoint* de búsqueda por ID externo de Trakt (`/search/tmdb/{id}?type=movie|show`).

Problema de datos incompletos. Las respuestas de Trakt incluyen datos de *tracking* pero no imágenes. Las API Routes que devuelven listas de contenido realizan peticiones paralelas a TMDb para obtener las URLs de *poster* y *backdrop*, completando la respuesta antes de enviarla al cliente.

La tabla 4.3 recoge los *endpoints* de Trakt utilizados por módulo funcional.

Endpoint Trakt	Descripción
`GET /sync/history`	Historial completo de películas y episodios vistos
`POST /sync/history`	Registrar película/episodio como visto
`DELETE /sync/history`	Desmarcar como visto
`GET /sync/watched/shows`	Estado de visionado de todas las series
`POST /sync/ratings`	Añadir valoración (1–10)
`GET /users/me/stats`	Estadísticas globales del usuario
`GET /calendars/my/shows`	Próximos episodios de series seguidas
`GET /search/tmdb/{id}`	Búsqueda de item por ID de TMDb

Tabla 4.3: Principales *endpoints* de Trakt utilizados

4.3.4 JustWatch (API GraphQL no oficial)

JustWatch no tiene una API oficial pública. El cliente (`lib/api/justwatch.js`) utiliza los mismos *endpoints* GraphQL que la web de JustWatch, enviando peticiones `POST` a `https://apis.justwatch.com/graphql`.

Se utilizan dos *queries* GraphQL:

1) **Búsqueda del título** (`GetSearchTitles`): recibe el título y tipo de contenido y devuelve el `nodeId` interno de JustWatch.

2) **Detalles con ofertas** (`GetTitleDetails`): recibe el `nodeId` y devuelve las ofertas de *streaming* disponibles para España, incluyendo el tipo de monetización (`FLATRATE`, `RENT`, `BUY`), la URL directa al contenido en la plataforma y los datos del proveedor.

La función principal `getStreamingProviders(title, type, year)` filtra las ofertas de tipo `FLATRATE` (suscripción), les asigna el logo de la plataforma mediante un mapeo estático a los logos de TMDb y devuelve la URL directa al contenido.

4.3.5 Plex Media Server

La integración con Plex se ha diseñado para conectar el catálogo de la biblioteca personal del usuario. El objetivo es que, si un título existe en el servidor Plex configurado, pueda abrirlo directamente desde la aplicación sin búsquedas manuales.

Funcionamiento general. Cuando el usuario pulsa la opción de abrir en Plex, el frontend llama a una ruta interna del servidor con los metadatos del título (nombre, tipo, año e identificadores externos). Esa ruta busca la equivalencia del contenido en Plex, prepara el enlace más adecuado según el dispositivo y devuelve una respuesta unificada mostrando el botón de acceso.

Gestión de acceso y seguridad. La comunicación con Plex se realiza siempre desde el backend, usando credenciales guardadas en variables de entorno. El sistema admite una configuración simple basada en token y otra más robusta con generación y renovación automática de credenciales temporales. En ambos casos, la aplicación evita exponer claves sensibles en el navegador.

Experiencia de uso. Para el usuario final, el proceso es transparente ya que desde la misma pantalla de detalle puede acceder al contenido en Plex, tanto en escritorio como en móvil.

4.4 Arquitectura de API Routes propias

Next.js permite definir API Routes como funciones *serverless*, proporcionando endpoints HTTP personalizados. El proyecto implementa API Routes organizadas en el directorio `src/app/api/`, que actúan como capa intermedia entre el cliente y las APIs externas, implementando la lógica, transformación de datos, caché y seguridad.

4.4.1 Funciones principales de las API Routes

Las API Routes del proyecto cumplen seis funciones fundamentales:

1. Proxy seguro. Protegen las claves API (API keys) de exposición al navegador. Todas las peticiones a TMDb, Trakt, OMDb, Plex y JustWatch pasan por endpoints intermedios que almacenan las credenciales en variables de entorno del servidor, garantizando que nunca se envíen al cliente.

2. Transformación y enriquecimiento de datos. Las respuestas de las APIs externas se procesan antes de enviarlas al cliente: se traducen identificadores entre sistemas (TMDb ID, Trakt ID, IMDb ID), se combinan datos de múltiples fuentes en una única respuesta y se añaden metadatos adicionales como títulos, posters y años de estreno desde TMDb.

3. Caché y optimización. Implementan caché en memoria para reducir llamadas redundantes a APIs externas con límite de peticiones.

4. Gestión de autenticación. Manejan flujos OAuth 2.0 completos para Trakt y TMDb, almacenando tokens de sesión en cookies HTTP-only inaccesibles desde JavaScript. Implementan renovación automática de tokens caducados y validación de estado de conexión.

5. Agregación de datos. Consultan múltiples APIs en paralelo y combinan los resultados. Por ejemplo, `/api/tv/[id]/ratings` obtiene puntuaciones de TMDb, Trakt e IMDb

simultáneamente y las devuelve en un objeto unificado.

6. Validación y normalización. Validan parámetros de entrada y normalizan formatos antes de procesarlos. Las fechas se convierten a formato ISO 8601 para Trakt, los TMDb IDs se validan como enteros positivos, los ratings se restringen a rangos válidos según cada plataforma y se gestionan timeouts configurables para evitar bloqueos en peticiones lentas.

4.4.2 Categorías funcionales de endpoints

4.4.2.1 Autenticación y gestión de sesiones

Flujo OAuth 2.0 con Trakt. La autenticación con Trakt implementa el flujo estándar OAuth 2.0 Authorization Code mediante cuatro endpoints que gestionan el ciclo completo de sesión. El endpoint de inicio genera una URL de autorización con protección CSRF y redirecciona al usuario a Trakt para aprobar el acceso. El callback recibe el código de autorización, lo valida e intercambia por tokens de acceso y refresco que se almacenan en cookies HTTP-only seguras. El sistema incluye verificación automática del estado de sesión con renovación transparente de tokens caducados y un endpoint de desconexión que revoca credenciales tanto en el servidor como en Trakt.

Autenticación TMDb. Se utiliza un sistema de request token que requiere aprobación manual del usuario en la web oficial. El proceso obtiene un token temporal, espera la aprobación del usuario y crea una sesión persistente almacenada en cookies.

4.4.2.2 Datos de usuario y actividad

Historial de visualización. El sistema de historial combina datos de Trakt y TMDb para proporcionar una visión completa del contenido visto por el usuario. El endpoint principal acepta filtros por tipo de contenido, rangos de fechas y paginación, ejecutando peticiones paralelas a TMDb con control de concurrencia para enriquecer los datos básicos de Trakt con metadatos visuales y descriptivos. El sistema calcula automáticamente estadísticas agregadas como total de visualizaciones por título y fechas de primer y último visionado. Los endpoints complementarios permiten añadir visualizaciones con marcas de tiempo personalizadas, modificar fechas de entradas existentes y eliminar registros del historial.

Progreso y estado de series. El seguimiento de series implementa un modelo estructurado que organiza episodios vistos por temporada, facilitando la visualización del progreso de visionado. El marcado masivo de episodios divide automáticamente las peticiones en bloques para respetar límites de la API, validando previamente contra TMDb para prevenir errores. El sistema proporciona resúmenes completos del estado de cualquier título, incluyendo número de reproducciones, última visualización e historial de fechas.

Estadísticas de usuario. El módulo de estadísticas agrega datos de múltiples fuentes para generar análisis completos del historial de visualización del usuario. Calcula rankings de contenido más visto, identifica actores y directores frecuentes mediante análisis de créditos, agrupa géneros con distribución porcentual, suma tiempos totales de visionado y genera visualizaciones de actividad temporal por mes, día de la semana y hora del día.

4.4.2.3 Valoraciones y ratings

Gestión de ratings personales. El sistema de valoraciones permite consultar, crear, actualizar y eliminar puntuaciones del usuario con escala normalizada de 1 a 10. Tanto TMDb como Trakt soportan valoraciones con decimales y el sistema incluye renovación automática de tokens de autenticación cuando detecta sesiones caducadas.

Ratings agregados de múltiples fuentes. Los endpoints de agregación combinan puntuaciones de TMDb, Trakt e IMDb consultando las tres fuentes en paralelo. El sistema extrae el rating promedio de TMDb, obtiene la valoración comunitaria de Trakt mediante búsqueda por identificador y consulta OMDb para ratings de IMDb, Rotten Tomatoes y Metacritic. La arquitectura garantiza que un error en cualquier fuente no bloquee las demás.

4.4.2.4 Proxy y caché de datos externos

Proxy OMDb con caché optimizada. El proxy de OMDb implementa un sistema multinivel de caché en memoria que almacena respuestas exitosas durante 24 horas y errores temporalmente durante 60 segundos para permitir reintentos rápidos. La deduplicación de peticiones concurrentes garantiza que múltiples solicitudes simultáneas del mismo recurso compartan una única petición HTTP, reduciendo significativamente el tráfico durante cargas masivas. El sistema soporta timeouts configurables adaptados al contexto de uso y valida respuestas para descartar errores HTML.

Carga de imágenes. Cuando la aplicación necesita obtener imágenes de portada y fondo para muchos títulos a la vez, el sistema realiza las peticiones en paralelo controlando el número máximo de consultas simultáneas para no saturar la API de TMDb. La selección automática de imágenes aplica varios criterios en función de la resolución, títulos, idiomas y votos de la comunidad.

4.4.2.5 Integración con Plex Media Server

Los endpoints de Plex permiten buscar contenido en múltiples servidores configurados simultáneamente, probando conexiones HTTPS con respaldo a HTTP para entornos locales. La búsqueda implementa estrategia de coincidencia en dos niveles: primero intenta emparejar por identificadores únicos de IMDb o TMDb, y si no están disponibles, compara títulos normalizados

con margen de tolerancia de un año. El sistema aplica timeouts individuales por servidor para evitar esperas prolongadas y retorna el primer resultado exitoso.

El listado de biblioteca implementa paginación automática para colecciones extensas y ordena resultados por calidad de vídeo descendente, eliminando duplicados cuando existen múltiples versiones del mismo título.

4.4.2.6 Enlaces externos y descubrimiento

Generación inteligente de enlaces. El sistema de enlaces externos implementa estrategias adaptativas según la disponibilidad de metadatos. Para servicios con soporte de identificadores únicos como Letterboxd, se utilizan URLs directas cuando existe IMDb ID, mientras que en su ausencia se realiza scraping de metadatos con caché de resultados. JustWatch requiere extracción de datos embebidos en páginas dinámicas con matching por año para desambiguar títulos homónimos y múltiples estrategias de fallback ante cambios estructurales.

Descubrimiento y recomendaciones. Los endpoints de recomendaciones consultan múltiples fuentes de Trakt en paralelo, intercalando resultados de películas y series para variedad. El enriquecimiento con datos de TMDb añade metadatos visuales y descriptivos localizados. El sistema de contenido relacionado traduce identificadores entre plataformas, obtiene sugerencias algorítmicas de Trakt y enriquece resultados con control de concurrencia, limitando respuestas para mantener tiempos de carga óptimos.

4.4.2.7 Listas y colecciones

Listas personales de Trakt. El sistema de listas personales obtiene todas las colecciones del usuario con sus metadatos completos incluyendo nombre, descripción, configuración de privacidad y estadísticas. Para cada lista se recuperan los primeros elementos enriquecidos con información visual de TMDb mediante peticiones paralelas. El sistema implementa manejo robusto de límites de tasa con reintentos automáticos y eliminación de duplicados.

Listas comunitarias. Los endpoints de listas públicas permiten descubrir colecciones creadas por la comunidad que incluyen un título específico. El sistema soporta múltiples criterios de ordenación incluyendo popularidad, tendencias y actividad reciente, traduciendo identificadores entre plataformas y retornando metadatos del creador junto con previsualizaciones visuales.

Colecciones de TMDb. Las colecciones proporcionan acceso a sagas y franquicias cinematográficas mediante búsqueda textual o acceso a colecciones destacadas curadas por popularidad. Los detalles completos incluyen todas las películas de la saga ordenadas cronológicamente con metadatos visuales y descriptivos.

4.4.3 Patrones técnicos transversales

Todos los endpoints implementan patrones comunes de robustez y fiabilidad:

Manejo de errores estructurado. Las respuestas de error siguen un formato JSON consistente con códigos HTTP semánticos: 400 para validación fallida, 401 para autenticación requerida, 404 para recursos inexistentes, 429 para límites de tasa excedidos, 500 para errores internos y 503 para servicios externos no disponibles. Cada error incluye descripción legible y opcionalmente código de error y detalles adicionales para debugging.

Validación y sanitización de entrada. Los parámetros se validan exhaustivamente antes de procesamiento: identificadores numéricos deben ser enteros positivos, fechas deben cumplir ISO 8601, ratings se restringen a rangos específicos por plataforma y se rechazan payloads excesivamente grandes para prevenir ataques de denegación de servicio.

Resiliencia y degradación. Las peticiones a servicios externos incluyen timeouts adaptativos según el tipo de operación. Los servicios que fallan repetidamente activan degradación controlada retornando datos parciales, cacheados o fallbacks cuando están disponibles, evitando interrupciones completas del servicio.

Observabilidad. El sistema registra errores con contexto completo incluyendo endpoint, parámetros sanitizados, estado de autenticación y trazas de ejecución. Las operaciones lentas se marcan automáticamente como advertencias para identificación de cuellos de botella y optimización futura.

4.5 Implementación de los módulos funcionales

En esta sección se documentan en profundidad los módulos principales de la aplicación, explicando su arquitectura, flujos de datos, patrones de optimización y características funcionales.

4.5.1 Dashboard principal (`/`)

Es el punto de entrada de la aplicación y muestra un conjunto de secciones filtradas y organizadas por temas.

La página de inicio (`src/app/page.jsx`) es un Server Component que ejecuta en paralelo todas las peticiones necesarias de TMDb mediante `Promise.all()` lo que permite reducir el tiempo total de carga a la latencia de la petición más lenta.

A continuación se muestran las secciones principales del dashboard, su respectivo endpoint de TMDb y el resultado obtenido:

Sección	Endpoint TMDb	Descripción
Hero	`/trending/movie/week`	Carrusel de películas de este mes
Top Películas	`/movie/top_rated`	Películas mejor valoradas de todos los tiempos
Tendencias	`/trending/movie/week`	Películas populares esta semana
Populares	`/movie/popular`	Contenido con mayor momentum
Acción	`/discover/movie?with_genres=28`	Películas de acción clásicas
Dramas	`/discover/movie?with_genres=18`	Películas dramáticas imprescindibles
Series Populares	`/tv/popular`	Series trending del momento
Series Top Rated	`/tv/top_rated`	Series clásicas mejor valoradas
Series Trending	`/trending/tv/week`	Series en tendencia esta semana
Emitiéndose Hoy	`/tv/airing_today`	Episodios con emisión real-time
En Emisión	`/tv/on_the_air`	Series con temporada activa
Favoritos (auth)	`/account/{id}/favorite/movies`	Favoritos personales del usuario

Tabla 4.4: Secciones del dashboard y sus fuentes de datos

Selección de backdrops: el carrusel principal utiliza *backdrops* de TMDb y se aplica un criterio de selección a partir de la función `pickBestBackdropByLangResVotes(...)` que prioriza primero los backdrops en inglés (`en`, `en-US`) cuando existen, después filtra por resolución y por `vote_count` y `vote_average`.

4.5.2 Página de detalles (`/details/[type]/[id]`)

La página de detalles es el componente más complejo de la aplicación. Su funcionalidad es mostrar la información completa de la película o serie seleccionada a partir de los datos de TMDb, Trakt.tv, OMDb (IMDb), Plex Media Server, JustWatch, Letterboxd y servicios adicionales. Además, permite al usuario interactuar con el contenido (marcar como favorito, añadir a pendientes, valorar, compartir, etc.) y navegar por secciones relacionadas (reparto, recomendaciones, colecciones).

4.5.2.1 Arquitectura de datos

Se hace uso de Server Component y Client Component que optimiza tanto el SEO como la interacción en la página web. El Server Component se encarga de obtener los datos críticos iniciales en tiempo de renderizado *server-side*, incluyendo los metadatos completos del título, los logos oficiales, el reparto y equipo técnico completo, las recomendaciones de contenido similar, los proveedores de *streaming* disponibles por país, los videos oficiales (trailers, teasers, clips) y las

reseñas de la comunidad. Si el usuario está autenticado, también se obtiene el estado de la cuenta (si ha marcado el título como favorito, si está en su lista de pendientes, y su valoración personal).

Una vez que el Client Component se ejecuta en el navegador, se empiezan a realizar peticiones paralelas para ir construyendo la interfaz. Primero se carga el scoreboard de Trakt mediante `/api/trakt/scoreboard` para obtener la puntuación, el número de usuarios que están viéndolo actualmente (`watchers`), el número total de reproducciones (`plays`), cuántos usuarios lo tienen en sus bibliotecas (`collectors`) y cuántas listas de Trakt lo incluyen. En paralelo, si el título tiene IMDb ID, se consulta OMDb mediante `/api/omdb` para obtener el rating de IMDb, el número de votos, las puntuaciones de Rotten Tomatoes y Metacritic, y los premios ganados. Todos estos datos se guardan en caché: OMDb en `sessionStorage` con TTL de 24 horas, y el scoreboard de Trakt en memoria con TTL de 5 minutos.

A partir de la API de Plex se detecta si el título está disponible en la biblioteca personal y se guarda durante 7 días en `sessionStorage` y se generan múltiples URLs de reproducción: una para el navegador web (`plexUrl.web`) y otra para la aplicación móvil de Plex (`plexUrl.mobile`).

Los proveedores de *streaming* se obtienen de dos fuentes distintas. Por un lado, TMDb proporciona información básica de la disponibilidad mediante `watch/providers`, indicando si el título está disponible para alquiler, compra o en plataformas de suscripción. Por otro lado, JustWatch (mediante la API no oficial en GraphQL) proporciona enlaces directos al contenido en cada plataforma. Si la película pertenece a una colección (por ejemplo, la saga de *El Señor de los Anillos*), se carga la información completa de la colección mediante `/api/tmdb/collection` y se muestra una nueva sección con un carrusel y todas las películas de la saga, ordenadas cronológicamente.

4.5.2.2 Menú de navegación por secciones

El componente `DetailsSectionMenu.jsx` implementa un menú de navegación con pestañas que se adapta dinámicamente en función del número de secciones disponibles que haya y el dispositivo cliente. Para películas, el menú incluye las secciones Media (galería de imágenes y videos), Sentimientos (análisis de pros y contras extraído de comentarios de Trakt mediante IA), Comentarios (reviews de TMDb y comentarios de Trakt), Listas (listas de Trakt que incluyen el título, con pestañas Popular y Trending), Colección (si la película pertenece a una saga), Reparto (reparto completo con fotos y personajes) y Recomendaciones (títulos similares sugeridos por el algoritmo de TMDb y Trakt). Para las series, se añaden las secciones Temporadas y Episodios ya que son exclusivas para este tipo.

4.5.2.3 Sistema de puntuaciones

El componente `ScoreboardBar.jsx` centraliza las puntuaciones de todas las plataformas en una barra horizontal. Aquí se muestran las puntuaciones de TMDb, Trakt, IMDb (OMDb), Rotten Tomatoes (OMDb) y Metacritic (OMDb) si están disponibles y cada una de ellas se muestra dentro de un badge con el logo de la plataforma, el valor numérico, el número de votos y el enlace externo a la página del título correspondiente de cada plataforma.

La puntuación de IMDb se obtiene de OMDb (`/api/omdb?imdb={imdbId}`) y se muestra junto con el número de votos. Esta petición solo se realiza si el título tiene asociado un IMDb ID en los `external_ids` de TMDb, y el resultado se cachea en `sessionStorage` con una clave específica `omdb:{imdbId}` y un TTL de 24 horas. OMDb también devuelve puntuaciones de Rotten Tomatoes y Metacritic (campos `RottenTomatoes` y `Metascore`).

Además de las puntuaciones, el scoreboard muestra estadísticas de Trakt mediante (`MiniStat`) que incluye iconos descriptivos: el número de usuarios que están viendo el título en este momento (`watchers`, icono Eye), el número total de reproducciones históricas (`plays`, icono Play), cuántos usuarios lo tienen en sus bibliotecas (`collectors`, icono LibraryBig), cuántos comentarios ha recibido (`comments`, icono MessageSquare) y en cuántas listas públicas de Trakt está incluido (`lists`, icono List). Estas estadísticas se cargan mediante `requestIdleCallback` para no bloquear el renderizado inicial del contenido mas relevante.

4.5.2.4 Botones de acción

La página de detalles incluye varios botones que permiten al usuario interactuar con su contenido.

Botón de Favoritos. Este botón permite marcar películas o series como favoritas en la cuenta de TMDb del usuario. Utiliza la función `markAsFavorite({accountId, sessionId, type, mediaId, favorite})` que realiza una petición POST al endpoint de TMDb `/account/{id}/favorite`. Cuando el usuario hace clic en el botón, el icono cambia inmediatamente de un corazón vacío a un corazón relleno.

Botón de Pendientes. Similar al botón de Favoritos, este botón añade o elimina contenido de la lista de pendientes (watchlist) del usuario en TMDb. Funciona mediante la misma API de TMDb pero utilizando el endpoint `/account/{id}/watchlist` con una petición POST. El icono alterna entre un marcador vacío y un marcador relleno según el estado actual y la lista sirve para que el usuario organice el contenido que desea ver próximamente.

Botón de Valoración personal. Este botón muestra una ventana emergente que permite al usuario poner una puntuación personal del 1 al 10 a la película o serie. La valoración se envía simultáneamente a TMDb y Trakt mediante peticiones POST, almacenándose en ambas

plataformas. Cuando el usuario ya tiene una valoración previa guardada, ésta se muestra en la interfaz y aparece un botón de papelera que permite eliminarla mediante la función `clearTmdbRating()`, esto ejecuta peticiones DELETE tanto a TMDb como a Trakt para mantener ambas cuentas sincronizadas. El sistema implementa sincronización bidireccional: si el usuario modifica su puntuación en una plataforma, el cambio se aplica automáticamente a la otra mediante `sendTraktRating(value)`, garantizando que ambos perfiles reflejen siempre la misma valoración. TMDb acepta puntuaciones con decimales mientras que Trakt requiere valores enteros, por lo que el sistema redondea hacia arriba automáticamente cuando es necesario.

Botón de Listas personalizadas. Exclusivo para películas, este botón abre el componente `AddToListModal` que permite gestionar las listas personales del usuario en TMDb. El modal carga todas las listas existentes consultando `GET /account/{id}/lists` y verifica en cuáles de ellas está incluida la película mediante peticiones a `GET /list/{listId}/item_status?movie_id={id}`. El resultado se muestra como una lista vertical con casillas de verificación que indican si la película pertenece o no a cada lista y el usuario puede añadir o quitar la película de cualquiera de ellas. También existe la opción de crear una nueva lista directamente desde este modal rellenando un formulario con nombre y descripción.

Botón de Marcar como visto (series). Exclusivo para series, este botón gestiona el seguimiento de episodios vistos haciendo uso de la API de Trakt. Cuando el usuario marca un episodio como visto, se ejecuta `toggleEpisodeWatched()` que envía una petición POST a `/api/trakt/episode/watched` incluyendo el identificador de la serie, número de temporada, número de episodio y la fecha/hora actual. Existen funcionalidades en este componente como el botón de marcar serie que envía todos los episodios no vistos en una única petición y marca la serie como completada en una acción.

Botón de Compartir. Este botón permite compartir el enlace de la página de detalles actual mediante las opciones nativas del navegador o del sistema operativo. Utiliza la API Web Share cuando está disponible (principalmente en dispositivos móviles) o copia el enlace al portapapeles como alternativa en navegadores de escritorio que no soporten esta funcionalidad.

4.5.2.5 Gestión avanzada de episodios para series

El seguimiento de episodios de una serie es una parte importante del proyecto. El estado de visionado se representa mediante un componente anidado `watchedBySeason` con la estructura `{"seasonNumber": {"episodeNumber": true/false}}`, que se construye a partir de la respuesta de Trakt `GET /sync/watched/shows` y se transforma por la API Route `/api/trakt/show/watched`. El componente `SeasonDetailsClient.jsx` renderiza cada temporada como una sección expandible que, al abrirse, carga los detalles de todos

los episodios de esa temporada mediante `getSeasonDetails(showId, seasonNumber)` y los muestra en formato grid o lista según la preferencia del usuario (guardada en `localStorage`). Cada episodio se renderiza con su número, nombre, fecha de emisión, puntuación de TMDb, puntuación de Trakt (si está disponible), y un checkbox interactivo que permite marcar/desmarcar el episodio como visto de forma individual. Cuando el usuario marca un episodio como visto, se ejecuta la función `toggleEpisodeWatched(seasonNumber, episodeNumber)` que realiza una actualización del checkbox y envía una petición POST a `/api/trakt/episode/watched` con el payload `{showId, season, episode, watchedAt: new Date().toISOString()}` y si tiene éxito, se recarga el estado completo de la serie mediante `GET /sync/watched/shows/{id}` para sincronizar con cualquier cambio que haya podido ocurrir en Trakt desde otros dispositivos o aplicaciones.

El componente `EpisodeRatingsGrid.jsx` ubicado en una sección de la página de detalles ofrece otro tipo de vista de todos los episodios de la serie. Cada celda del grid representa un episodio y tiene un color en función de la puntuación, es completamente interactivo y se puede hacer clic en cualquier celca para acceder a la página de detalles del episodio en cuestión (`/details/tv/{showId}/season/{season}/episode/{number}`).

Para optimizar el rendimiento en series con cientos de episodios, las puntuaciones se cargan en paralelo mediante `Promise.all(seasons.map(s =>getSeasonDetails(showId, s.season_number)))` en lugar de secuencialmente. Esto reduce la latencia total de aproximadamente N segundos (donde N es el número de temporadas).

4.5.2.6 Enlaces externos y plataformas de streaming

En la página de detalles se integran dos tipos de enlaces que se muestran de forma distinta según el modo de visualización y el dispositivo. Las plataformas de streaming (Netflix, Amazon Prime Video, HBO Max, Disney+, etc.) se muestran directamente debajo de la imagen principal ya sea backdrop o poster. Cuando el contenido está disponible en el servidor Plex personal del usuario, aparece el logo de Plex con un indicador verde que señala la disponibilidad local. Los enlaces externos a servicios de bases de datos y redes sociales (web oficial, TMDb, Trakt, IMDb, FilmAffinity, JustWatch, Letterboxd, SeriesGraph) tienen una ubicación adaptada según varios factores.

En la vista por defecto con poster vertical, los enlaces externos se integran en la barra de puntuaciones que aparece debajo de los botones de acción. En pantallas de escritorio, los iconos de cada servicio se muestran en línea con sus logos, alineados a la derecha de las puntuaciones de TMDb, Trakt e IMDb. En dispositivos móviles, debido al espacio horizontal limitado, se ocultan los iconos individuales y en su lugar se muestra un botón que abre el componente `ExternalLinksModal`. En el modo de vista previa con backdrop horizontal, los enlaces externos

cambian completamente de ubicación y se muestran junto a las plataformas de streaming debajo de la imagen principal, formando una única fila con un separador vertical.

El sistema de resolución de enlaces implementa tres niveles de complejidad según la plataforma. Los enlaces directos a TMDb, Trakt e IMDb se construyen mediante el ID correspondiente sin necesidad de peticiones adicionales: TMDb utiliza el ID numérico junto con un slug del título (por ejemplo, `/movie/550-fight-club`), Trakt usa el ID obtenido durante la carga del scoreboard (campo `trakt.traktUrl`), e IMDb emplea el IMDb ID en formato `tt1234567`. El enlace a la web oficial del título se extrae directamente del campo `homepage` de TMDb o, si no está disponible, mediante una petición a la API interna `/api/trakt/official-site`. FilmAffinity, que carece de API pública, se resuelve mediante una URL de búsqueda construida dinámicamente con el título normalizado.

Los enlaces a JustWatch y Letterboxd requieren un proceso de validación. Para JustWatch, la API interna `/api/links/justwatch` recibe el título y el año, genera un slug normalizado eliminando caracteres especiales y convirtiendo espacios en guiones, construye la URL final `https://www.justwatch.com/es/pelicula/{slug}`, ejecuta una petición HEAD para verificar que el recurso existe (código HTTP 200), y cachea el resultado en `localStorage` de forma indefinida para evitar validaciones repetidas pero si la validación falla, devuelve una URL de búsqueda genérica como fallback. Para Letterboxd (exclusivo de películas), si el título tiene IMDb ID se construye directamente la URL `https://letterboxd.com/imdb/{imdbId}` y en caso contrario, se genera un slug basado en el título y año, se valida su existencia mediante petición HEAD, y se cachea el resultado. Por último, el enlace de SeriesGraph (exclusivo de series) utiliza directamente el TMDb ID sin validación, construyendo la URL `https://seriesgraph.com/show/{tmdbId}`.

4.5.2.7 Imágenes y posters

El sistema de gestión de imágenes implementa tres modos de visualización distintos y permite a los administradores seleccionar manualmente qué versión de poster o backdrop se muestra. Para series de televisión, el sistema carga automáticamente las imágenes de todas las temporadas mediante `fetchTVImages({showId, apiKey})` que itera por cada temporada, obtiene sus imágenes mediante `GET /tv/{id}/season/{seasonNumber}/images`, y fusiona todos los resultados en un único array eliminando duplicados por `file_path`.

El sistema distingue tres tipos de imágenes según su función visual. Los posters son imágenes verticales con aspect ratio 2:3 que se muestran en el hero de la página de detalle y se optimizan en dos calidades: `w342` para la carga inicial rápida y `w780` para la carga final de alta calidad. Los backdrops de preview son imágenes horizontales con aspect ratio 16:9, optimizadas en tamaños

`w780` y `w1280`.

La selección automática de la mejor imagen se realiza mediante funciones. Para posters, `pickBestPosterTV(posters, languages)` ordena por resolución (preferencia por imágenes 4K y 2K) y número de votos, priorizando posters en español o inglés si están disponibles. Para backdrops, `pickBestBackdropTVNeutralFirst(backdrops, languages)` prioriza específicamente las imágenes con `iso_639_1 === null` (sin texto superpuesto) para el background de móvil, y luego ordena por resolución y votos para el resto.

Los administradores (específicamente el usuario `psantos26` hardcoded en el componente) tienen acceso a una galería administrable con tres pestañas: Posters, Backdrops (para modo preview) y Backgrounds (para fondo móvil). Cada pestaña incluye filtros de resolución (Todos, 720p, 1080p, 2K, 4K calculados a partir del `width` de cada imagen) y checkboxes de idioma (ES, EN). Las imágenes se muestran en un carrusel horizontal y al hacer clic en cualquier imagen se establece como activa, guardando la selección en `localStorage` con claves específicas por tipo e ID (por ejemplo, `showverse:movie:550:poster`, `showverse:tv:1399:backdrop`).

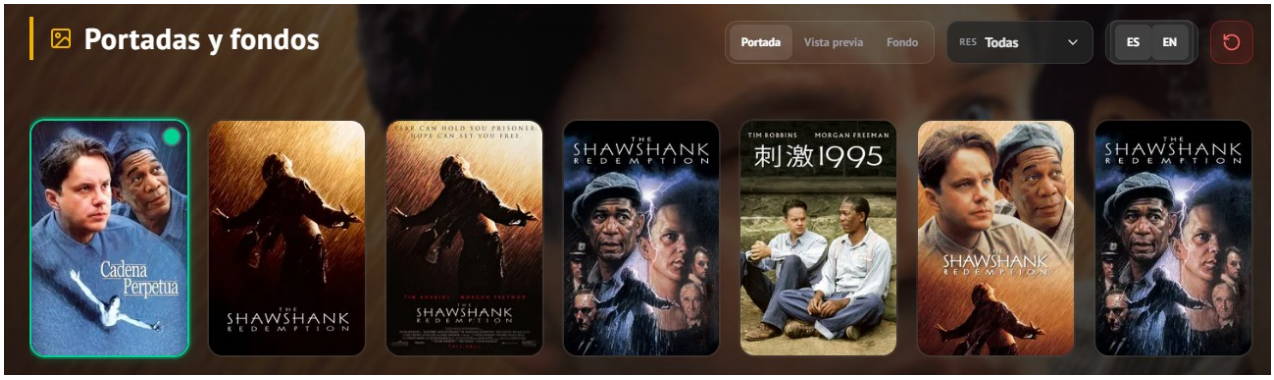


Figura 4.1: Interfaz de gestión de portadas y fondos.

4.5.2.8 Sección de reparto y ficha de personas

La sección de reparto unifica datos de múltiples fuentes. Los directores y creadores se extraen del array `credits.crew` filtrando por `job === "Director"` o `department === "Writing"` y se agregan al inicio del array de cast para darles prioridad. El array completo se ordena por el campo `order` que TMDb proporciona para indicar el orden de importancia de cada actor en la producción, de forma que los protagonistas aparecen primero. Cada tarjeta de actor muestra la foto del perfil en tamaño `w185`, el nombre real del actor, el nombre del personaje que interpreta, y un efecto de hover. Hacer clic en cualquier actor navega a la página de ficha de persona (`/details/person/{id}`) que carga mediante el componente `ActorDetails.jsx`.

La ficha de persona muestra la biografía completa del actor (campo `biography` de TMDb), su foto de perfil, fecha y lugar de nacimiento, y una filmografía completa dividida en dos pestañas:

películas y series de televisión. La filmografía se ordena cronológicamente por `release_date` o `first_air_date` en orden descendente (trabajos más recientes primero), y cada entrada muestra el poster del título, el año, el nombre del personaje o rol, y el rating de TMDb. La página también incluye enlaces externos a IMDb (si el actor tiene `imdb_id`), Instagram, Twitter, Facebook y sitio web oficial, construidos a partir del objeto `external_ids` de TMDb.

4.5.2.9 Comentarios, análisis de sentimientos y listas de Trakt

La sección de comentarios fusiona reviews de TMDb con comentarios de Trakt en la misma sección. Los comentarios de Trakt se organizan en tres categorías: «Top 30 días» que muestra los comentarios con más likes recibidos en los últimos 30 días, «Top histórico» que muestra los comentarios con más likes de todos los tiempos ordenados descendentemente por `likes` y «Más recientes» que muestra todos los comentarios ordenados por fecha de creación en orden descendente. Cada comentario incluye el avatar del usuario, su nombre de usuario, la puntuación que dio al título si es que votó, una advertencia de spoiler si el comentario está marcado como tal, el número de likes que ha recibido, el número de respuestas, la fecha de publicación relativa y el texto completo del comentario.

La funcionalidad del análisis de sentimiento de Trakt se construye mediante la función `buildSentimentFromComments(items)` que procesa todos los comentarios cargados, extrae frases que contienen palabras clave positivas (como «excelente», «increíble», «obra maestra», «recomendable») o negativas (como «decepcionante», «aburrido», «predecible», «desperdicio»), y genera dos listas: una de pros (aspectos positivos mencionados frecuentemente) y otra de contras (aspectos negativos recurrentes).

La sección de listas muestra las listas públicas de Trakt que incluyen este título, organizadas en dos pestañas: Popular (ordenadas por número de likes) y Trending (ordenadas por actividad reciente). Cada lista se renderiza como una tarjeta que muestra el nombre de la lista, una descripción, el nombre del usuario que la creó, el número de items que contiene, el número de likes que ha recibido y el número de comentarios. Hacer clic en cualquier lista navega a la página completa de esa lista en Trakt.

4.5.2.10 Recomendaciones y contenido similar

La sección de recomendaciones utiliza el algoritmo de Trakt para sugerir títulos relacionados mediante el endpoint `/{type}/{slug}/related?extended=full&limit=15`, que analiza patrones de visionado de la comunidad, géneros compartidos, directores, actores y palabras clave para determinar similitud. El proceso de obtención tiene tres fases secuenciales que transforman IDs. Primero, la API Route `/api/trakt/related` recibe el TMDb ID del título actual y lo traduce al slug de Trakt mediante el endpoint de búsqueda `GET`

`/search/tmdb/{id}?type=movie|show`, que devuelve el objeto completo del título incluyendo todos sus identificadores (`ids.slug`, `ids.trakt`, `ids.imdb`, `ids.tmdb`). Con el slug de Trakt obtenido, se ejecuta la petición de contenido relacionado que devuelve un array de hasta 15 títulos similares con metadatos básicos pero sin imágenes. Finalmente, cada resultado se completa individualmente con datos de TMDb mediante peticiones paralelas (`mapWithConcurrency`) obteniendo `poster_path`, `backdrop_path`, `vote_average`, `overview` y fecha de estreno.

4.5.3 Favoritos (`/favorites`)

La página de favoritos muestra la colección de películas y series del usuario a partir de la API TMDb. El componente `FavoritesClient.jsx` obtiene los datos mediante dos peticiones paralelas a los *endpoints* `GET /account/{id}/favorite/movies` y `GET /account/{id}/favorite/tv`, cada uno en bloques de 20 elementos. La función `fetchFavoritesForUser()` implementa un patrón de carga paralela: primero ejecuta una petición inicial para obtener el número total de páginas disponibles (`total_pages`), y posteriormente lanza todas las páginas restantes mediante `Promise.all()`, procesándolas en lotes de 5 peticiones concurrentes para respetar los límites de la API de TMDb. Los resultados de películas y series se combinan en un único array marcando cada elemento con `media_type: "movie"` o `media_type: "tv"`.

El componente implementa además un sistema de caché para evitar peticiones redundantes. Las puntuaciones de OMDb (ratings de IMDb, Rotten Tomatoes y Metacritic) se almacenan en `sessionStorage` con clave `omdb:{imdbId}` y un TTL de 24 horas, permitiendo que los datos persistan durante la sesión de navegación actual sin necesidad de nuevas peticiones. Las puntuaciones de Trakt se guardan en `localStorage` con TTL de 24 horas y marcas de tiempo en formato epoch, garantizando persistencia entre sesiones. La carga de puntuaciones externas no se realiza hasta que el usuario activa el hover sobre una tarjeta, evitando decenas de peticiones durante el montaje inicial del componente.

La interfaz proporciona tres modos de visualización como botones en la barra de herramientas. El modo *Grid* organiza los elementos en una cuadrícula responsive (6 columnas en escritorio) mostrando posters verticales en relación de aspecto 2:3, título y valoración de TMDb. El modo *List* presenta filas horizontales con imagen backdrop en relación de aspecto 16:9 de resolución 1920×1080 píxeles, título, año de estreno y puntuación TMDb y el modo *Compact* muestra más elementos por fila (8 columnas en escritorio) y cada tarjeta con menor tamaño que en Grid.

El sistema permite alternar entre dos tipos de imagen de portada: *Poster* (vertical, relación 2:3) o *Backdrop* (horizontal, relación 16:9), controlado mediante el estado `imageMode`. La lógica

implementada en `effectiveImageMode = viewMode === "list" ? "backdrop" : imageMode` fuerza el uso de backdrop cuando el modo de visualización es *List*, independientemente de la preferencia del usuario, ya que este modo requiere imágenes horizontales para su diseño de filas. En los modos *Grid* y *Compact*, el usuario puede alternar libremente entre poster y backdrop mediante un botón específico en la barra de herramientas y la selección de imágenes prioriza contenido en inglés con mayor número de votos mediante `pickBestPosterEN()` para posters y `pickBestBackdropByLangResVotes()` para backdrops, con *fallback* al `poster_path` o `backdrop_path` original de TMDb si la selección automática falla.

El menú de filtros y controles incluye cuatro categorías funcionales. La sección **Tipo** filtra el contenido mostrado mediante tres opciones: «Todo» (películas y series), «Películas» (`typeFilter = "movies"`), o «Series» (`typeFilter = "shows"`). La sección **Agrupar** organiza los elementos en grupos con cabeceras separadoras: «Sin agrupar» (predeterminado), «Por año» (usando `release_date` o `first_air_date`), «Por década» (agrupando años en bloques de 10), «Género» (utilizando el primer género del array `genre_ids`), y agrupación por valoración («TMDb», «IMDb», «Trakt», «Mis puntuaciones»). La sección **Orden** controla la ordenación de los elementos mediante ocho opciones: «Título A–Z» (`sortBy = "title-asc"`), «Título Z–A» (`sortBy = "title-desc"`), «Valoración ↓» y «Valoración ↑» (ordenación por `vote_average` de TMDb), «Añadido reciente» y «Añadido antiguo» (ordenación por timestamp de inserción en favoritos de TMDb), y «Vista reciente» y «Vista antigua» (ordenación por fecha de último visionado obtenida de Trakt mediante `GET /sync/history`).

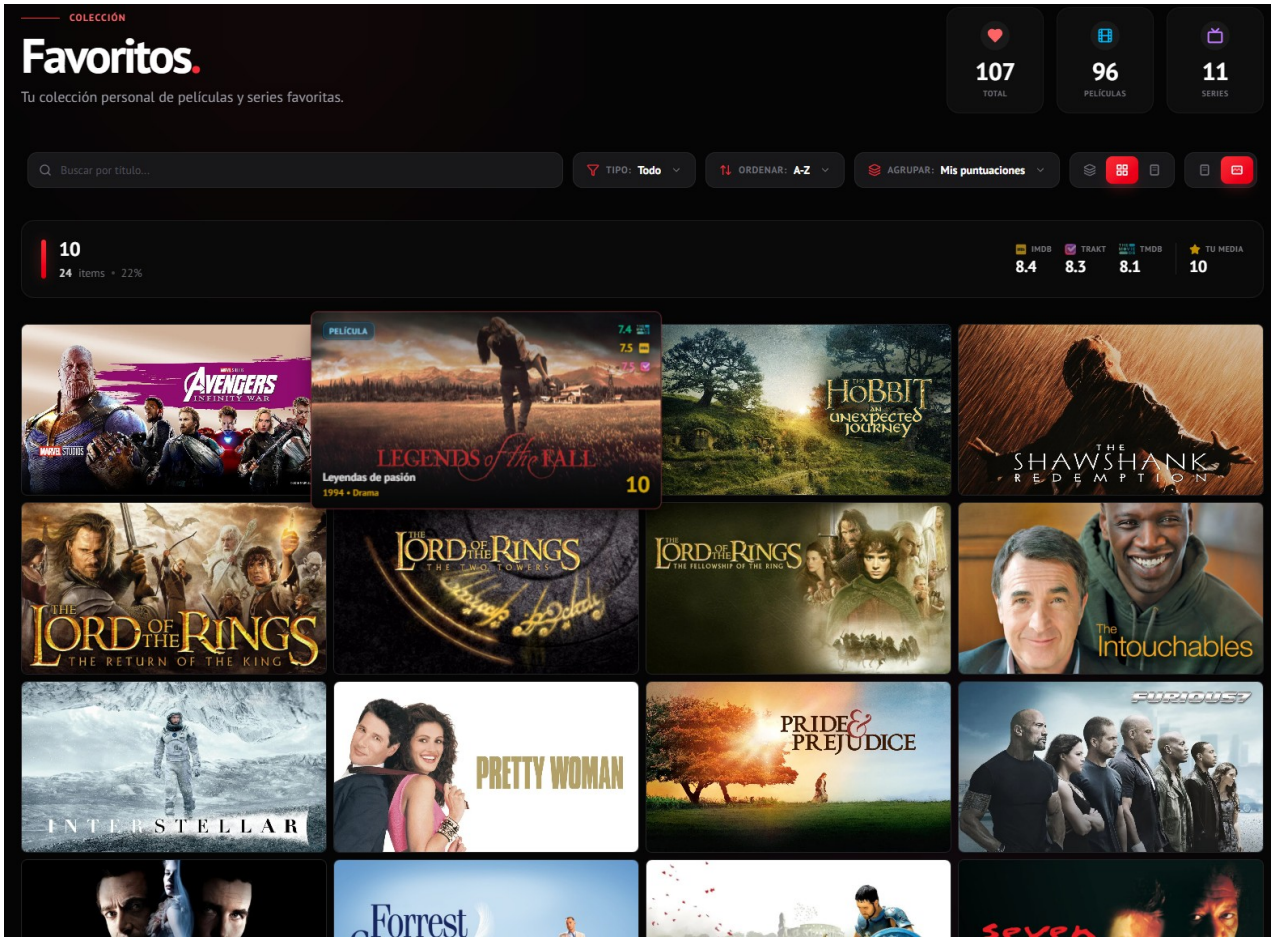


Figura 4.2: Interfaz de la sección de favoritos.

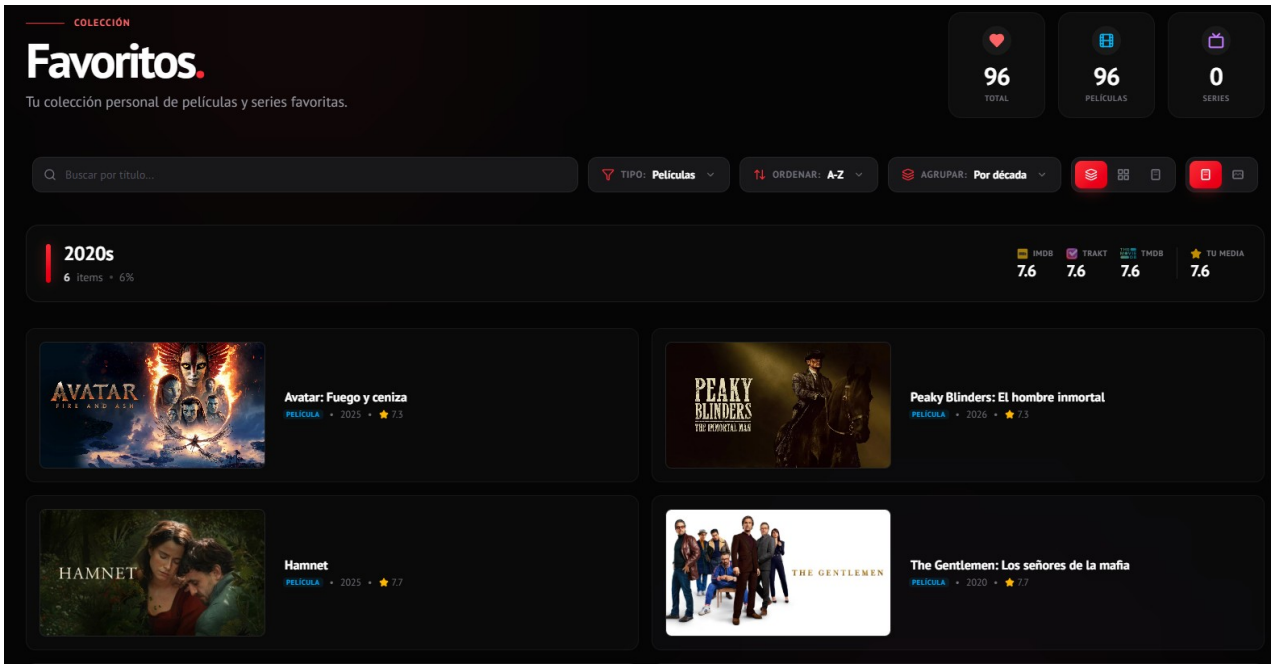


Figura 4.3: Vista tipo lista de la sección de favoritos.

4.5.4 Pendientes (`/watchlist`)

La página de watchlist muestra el contenido que el usuario tiene pendiente a partir de la API TMDb. El componente `WatchlistClient.jsx` utiliza la misma arquitectura que `FavoritesClient.jsx` y por ello cuenta con el mismo estilo de diseño y las mismas funcionalidades de filtrado, agrupación y ordenación. La única diferencia es que en lugar de consultar los endpoints de favoritos de TMDb, se consultan los endpoints de watchlist (`GET /account/{id}/watchlist/movies` y `GET /account/{id}/watchlist/tv`).

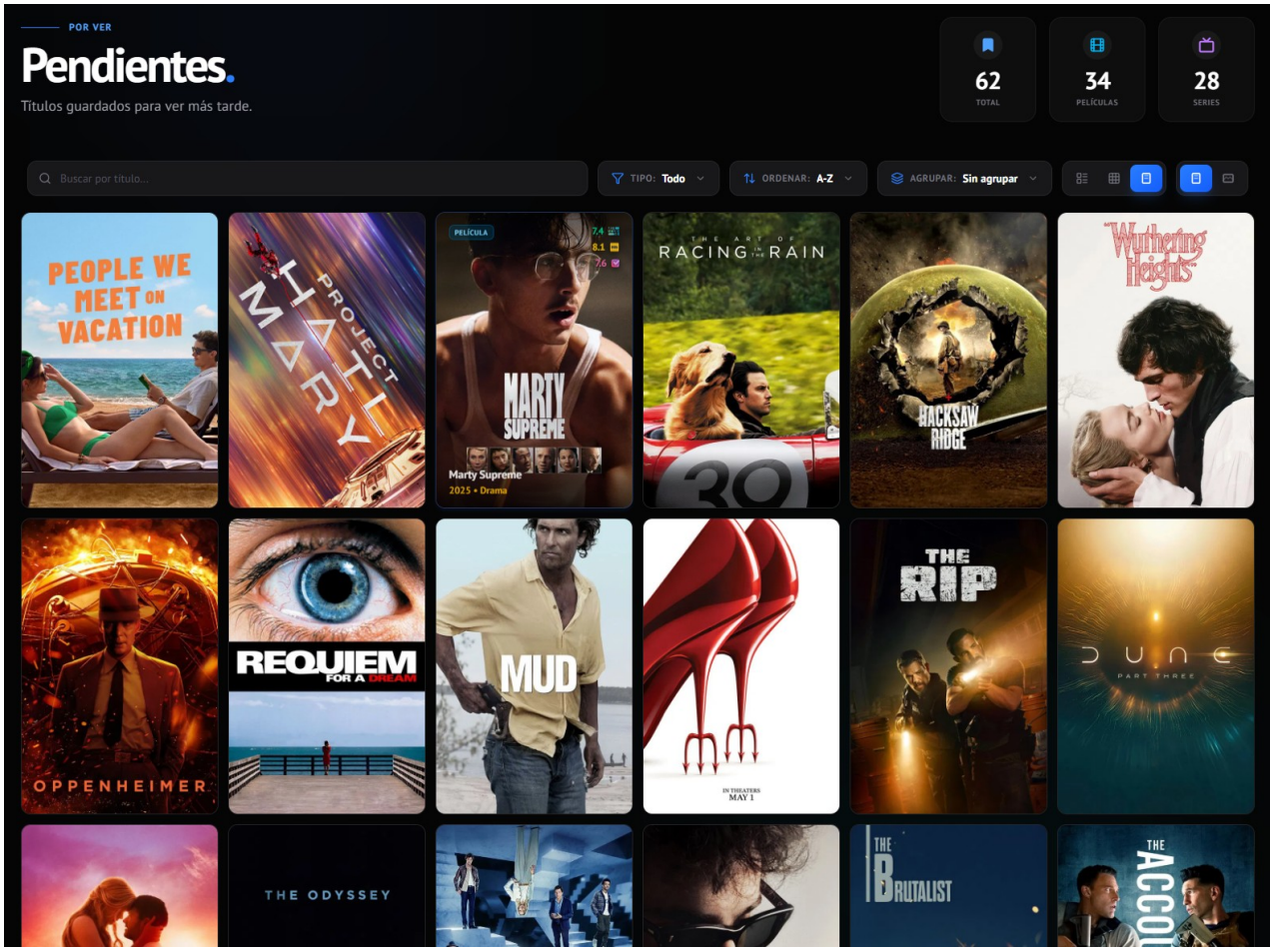


Figura 4.4: Interfaz de la sección de pendientes.

4.5.5 Historial de visionado (`/history`)

La página de historial muestra el registro completo de contenido visto por el usuario, obtenido desde Trakt mediante `GET /sync/history`. Trakt registra el timestamp exacto de cada visionado en formato ISO 8601, incluyendo la fecha, hora y episodio específico para series. Mediante la API Route interna `/api/trakt/history` se ajustan los datos de Trakt antes de enviarlos al cliente: traduce el identificador interno de Trakt a TMDb ID mediante

`/search/tmdb/{traktId}?type=movie|show`, y ejecuta peticiones paralelas a TMDb para obtener `poster_path`, `backdrop_path` y sinopsis de cada título.

El componente `HistoryClient.jsx` organiza el historial mediante agrupación por día, mes y año. Cada grupo incluye cabeceras separadoras con la fecha correspondiente. Para series, la función `collapseConsecutive()` detecta episodios consecutivos de la misma serie vistos en la misma sesión y los agrupa en una entrada única. Cada tarjeta cuenta con un diseño hover como en la mayoría de secciones que muestra el número de episodio , fecha de visionado y título.

A parte del menú de herramientas que mantiene una estética unificada y permite aplicar filtros, ordenación y agrupación, existe un calendario interactivo que permite seleccionar cualquier fecha y también acceder a la vista completa por meses.

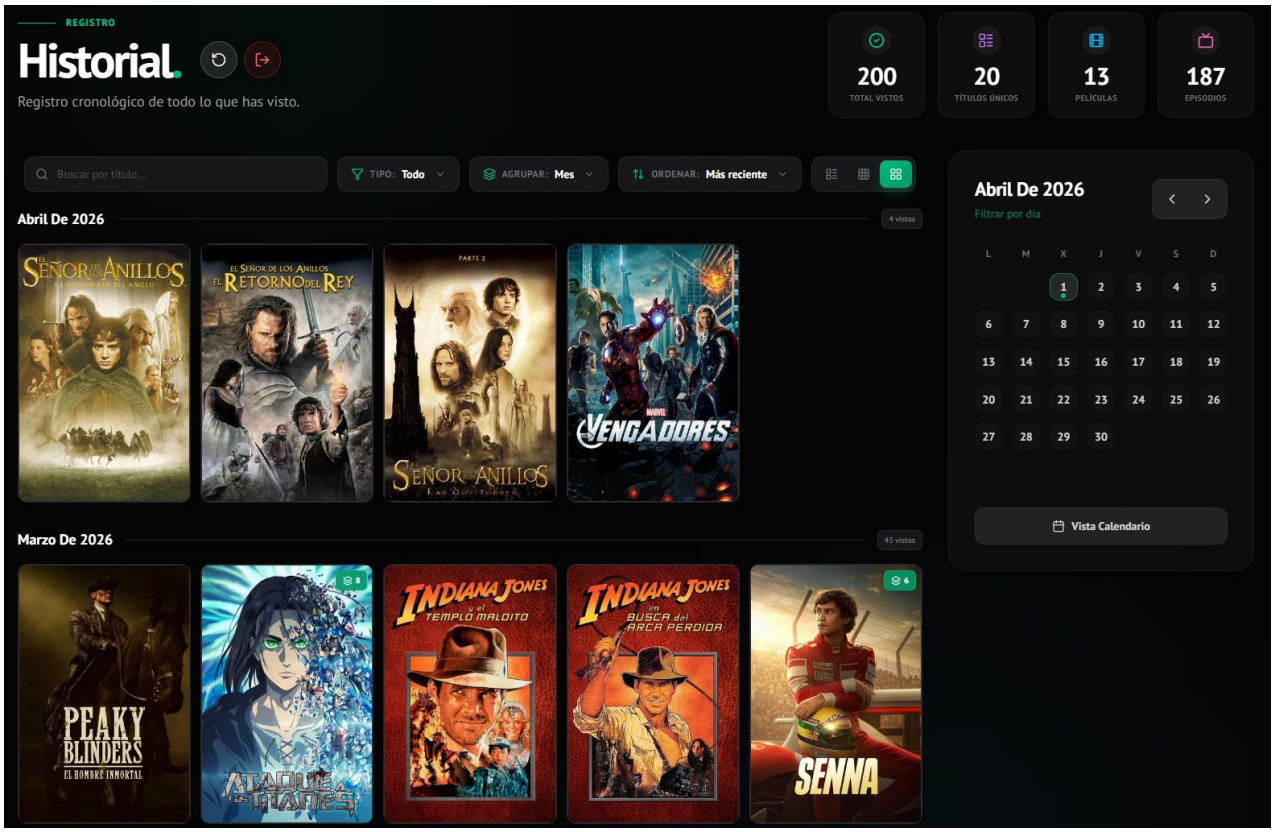


Figura 4.5: Interfaz de la sección de historial de visionado.

4.5.6 En Progreso (`/in-progress`)

La página de series en progreso determina qué series se encuentran activas cuando al menos un episodio ha sido marcado como visto pero no todos los episodios disponibles han sido completados. `InProgressClient.jsx` combina la respuesta de `GET /sync/watched/shows` de Trakt, que contiene el número de episodios vistos por temporada, con los datos de TMDb que informan del número total de episodios emitidos hasta la fecha. Además, el componente cuenta con otra sección

para las series completadas, que agrupa los títulos en los que todos los episodios disponibles han sido marcados como vistos mediante Trakt.

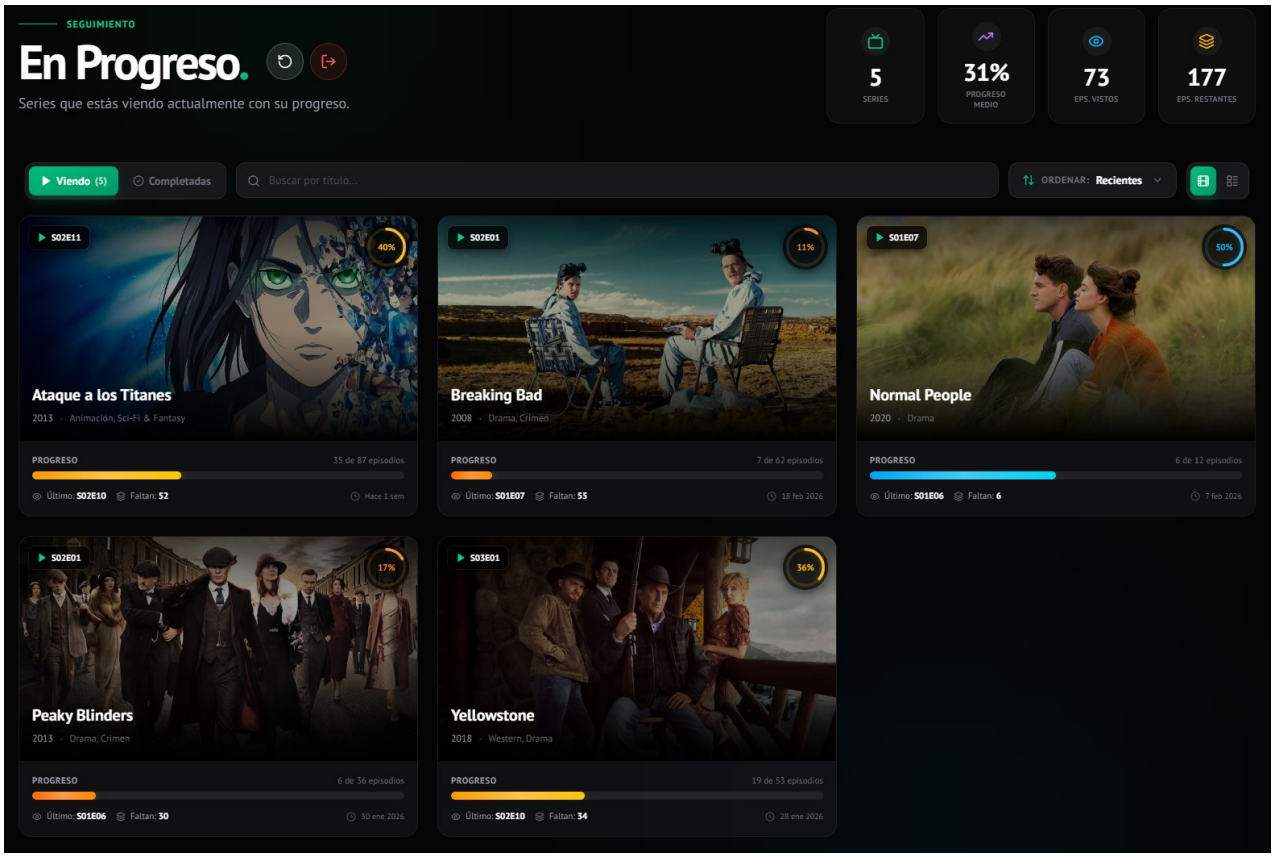


Figura 4.6: Interfaz de la sección de series en progreso.

4.5.7 Estadísticas del usuario (`/stats`)

El módulo de estadísticas unifica los datos de visionado del usuario mediante una petición a la API Route interna `/api/trakt/user-stats`, que agrega `/users/me/stats` (resumen global del usuario), el historial completo de `/sync/history` y las listas de `watchedMovies` y `watchedShows` (para identificar títulos más vistos). La respuesta incluye el total de películas, series, episodios, temporadas, tiempo en red y distribución de géneros, además de listas de actores y directores más frecuentes en el historial.

`StatsClient.jsx` transforma estos datos en gráficas y secciones visuales e interactivas haciendo uso de Recharts. La sección principal presenta tarjetas KPI con el total de horas de visionado, películas vistas, series seguidas y episodios completados. El gráfico de actividad mensual es un `AreaChart` que muestra los últimos 12 meses. La distribución por día de la semana y por hora del día se representa mediante `BarChart`. La distribución de géneros se visualiza como un `RadarChart` que mapea los seis géneros más consumidos. El histograma de ratings (distribución de puntuaciones del usuario) adopta la forma de un `BarChart` horizontal..

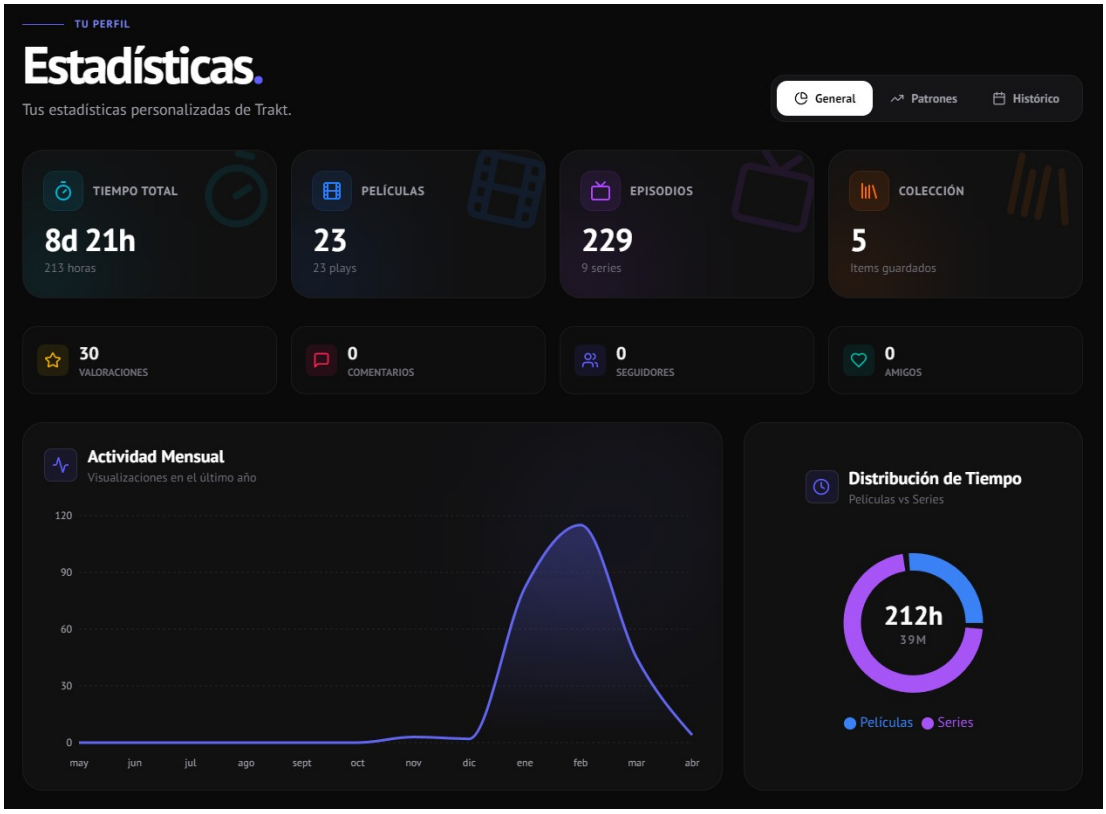




Figura 4.7: Interfaz de la sección de estadísticas del usuario (vista general y vista de patrones).

4.5.8 Calendario de estrenos (`/calendar`)

El módulo de calendario obtiene los estrenos de películas desde TMDb mediante el endpoint de descubrimiento `/discover/movie`, parametrizado con rangos de fechas según el modo de visualización activo y la región España (`region=ES`). La construcción de la URL parametrizada incluye `release_date.gte` y `release_date.lte` para acotar el rango (día específico, semana o mes completo), junto con `sort_by=release_date.asc` para ordenar los resultados cronológicamente y `language=es-ES` para localización. En caso de rangos amplios (vista de mes), se implementa limitación de paginación (`page <= 5`) para evitar peticiones excesivas y mantener tiempos de respuesta razonables.

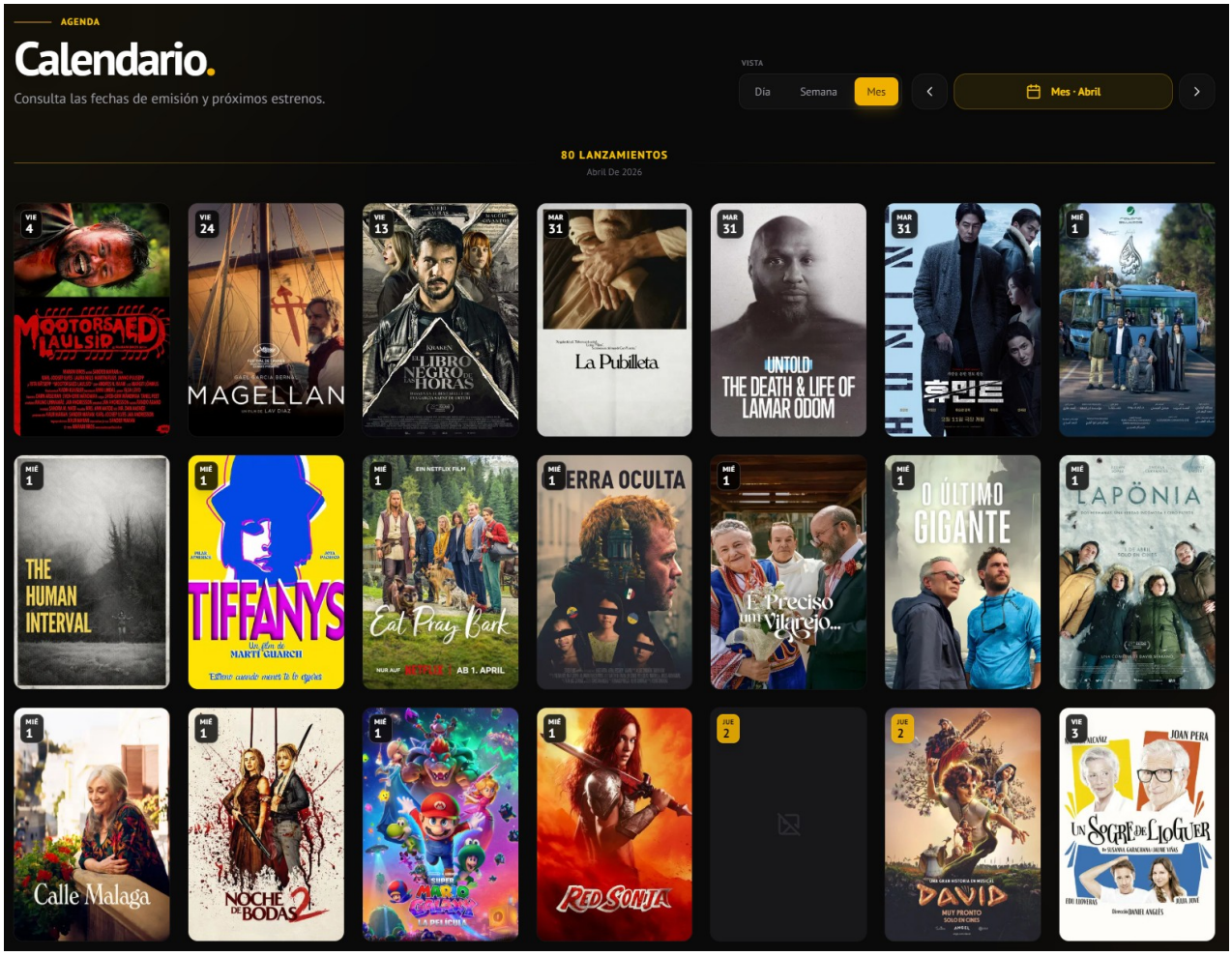


Figura 4.8: Interfaz del calendario de estrenos de películas.

4.5.9 Biblioteca Personal (`/biblioteca`)

La biblioteca personal se basa en la integración con Plex Media Server. El componente `BibliotecaClient.jsx` consulta el endpoint interno `/api/plex` para obtener el inventario completo de la biblioteca del servidor local, incluyendo metadatos técnicos que solo Plex

proporciona: resolución de cada archivo (4K/1080p/720p/576p/SD), códec de vídeo (H.264, HEVC/H.265, AV1), contenedor (`mkv`, `mp4`, `avi`), tamaño en disco en GB, y número de pistas de audio y subtítulos disponibles.

El componente implementa renderizado incremental para evitar bloquear el hilo principal cuando las bibliotecas Plex contienen un gran número de títulos, de esta forma renderiza los primeros 180 ítems de forma inmediata y añade bloques adicionales de 120 ítems cada vez que se alcanza el final de la lista, con prefetch de imágenes se muestran 80 ítems por delante de lo actualmente visible. Los datos de TMDb asociados a cada título (poster y backdrop en alta calidad) se cachean en un `Map`, y las preferencias de imagen se leen de `localStorage` con clave `showverse:movie:{id}:poster` antes de recurrir a la selección automática. La función `openPlexLink()` detecta la plataforma del usuario y genera la URL apropiada: `intent://` para Chrome en Android, el esquema `plex://` para la app nativa en iOS, o la URL web para escritorio.

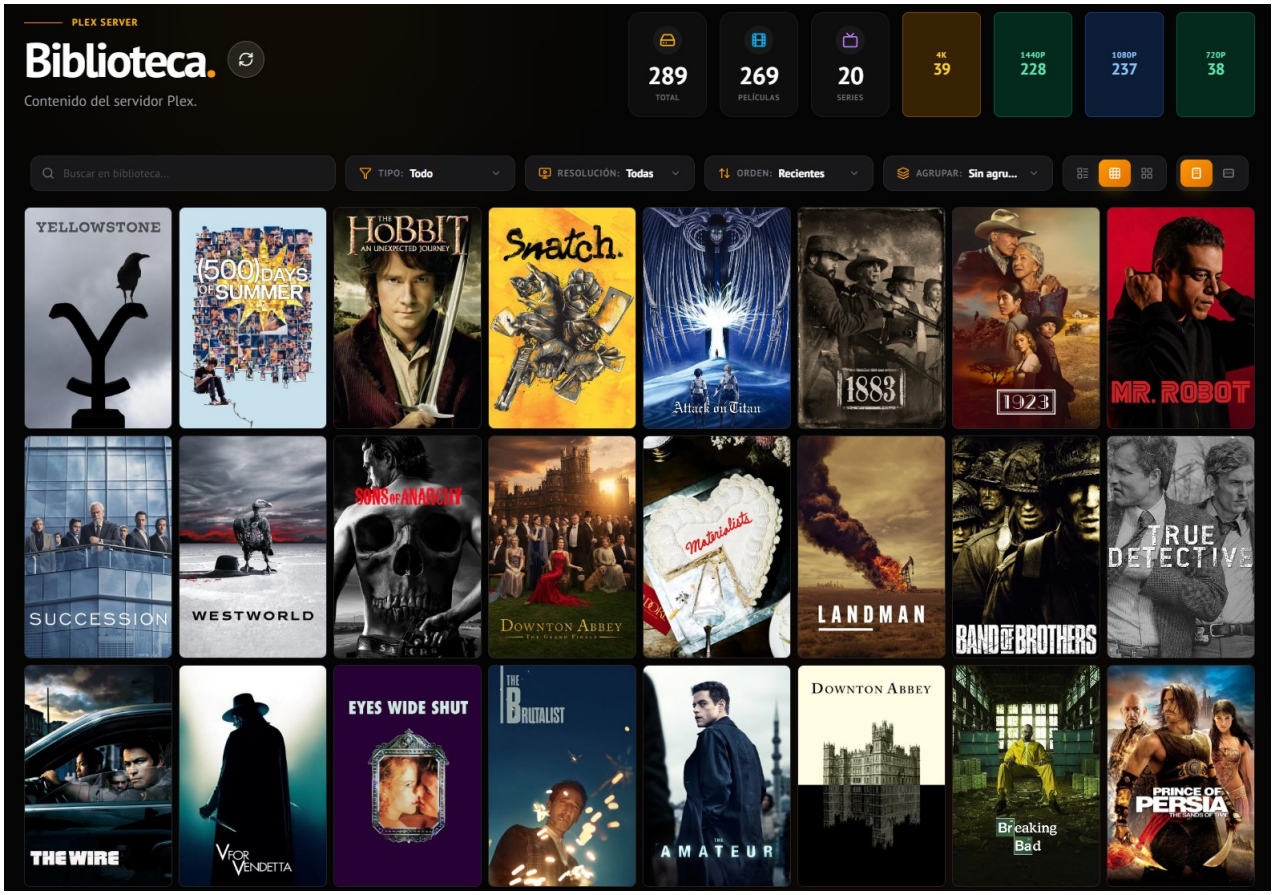


Figura 4.9: Interfaz de la biblioteca personal integrada con Plex Media Server.

4.5.10 Descubrir contenido (`/discover`)

La sección de descubrimiento emplea `/discover/movie` y `/discover/tv` de TMDb, que aceptan parámetros de filtrado completos, a diferencia de la búsqueda libre por texto que utiliza `/search/multi`. `DiscoverClient.jsx` gestiona el estado completo de los filtros activos: género, plataforma de *streaming* disponible, certificación de edad, rango de valoración media y mínimo de votos, rango de fechas de estreno y palabras clave con autocompletado contra `/search/keyword` de TMDb. Todos estos filtros se combinan en una única petición estructurada a TMDb cuyos parámetros se calculan dinámicamente mediante `buildDiscoverParams()` a partir del estado activo.

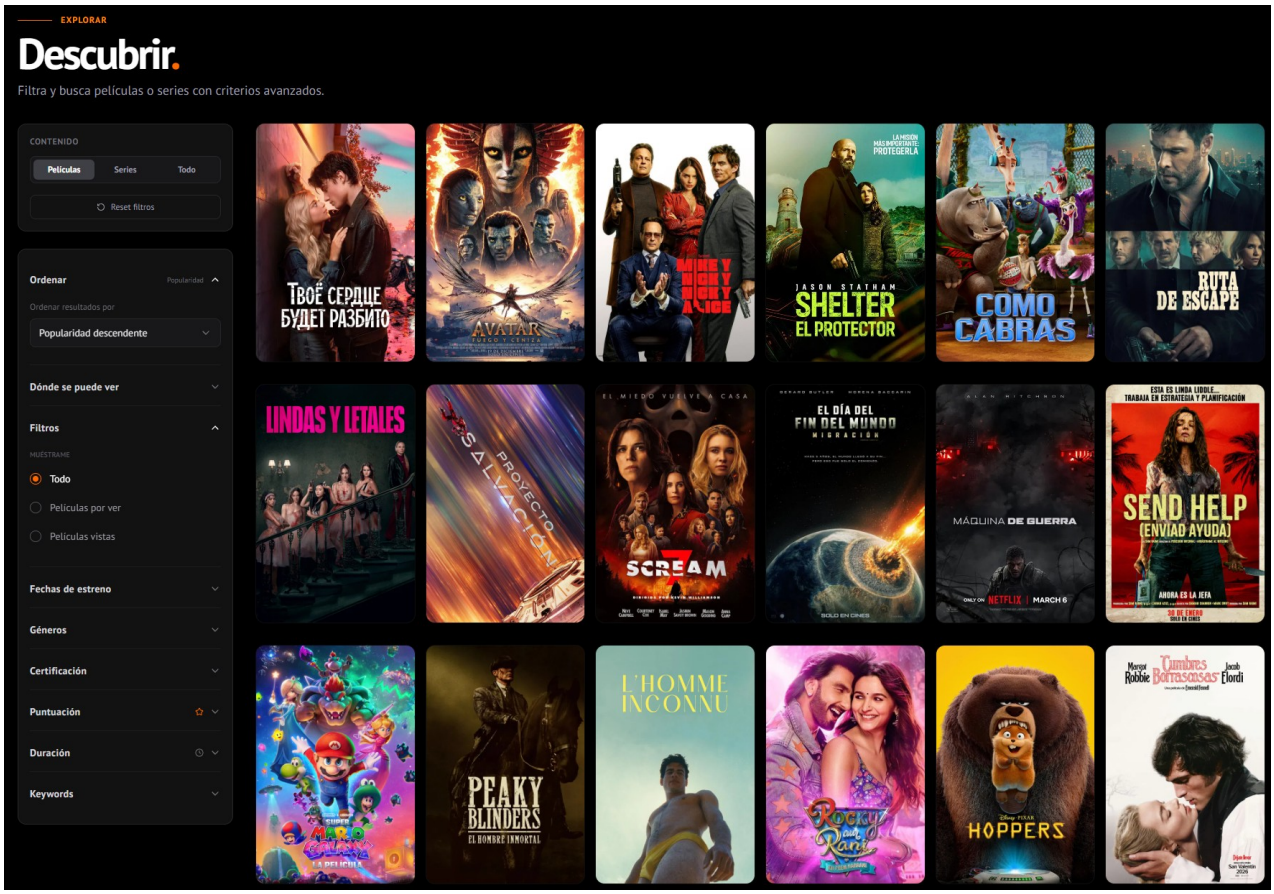


Figura 4.10: Interfaz de la sección Descubrir contenido.

4.5.11 Gestión de listas (`/lists`)

El módulo de listas unifica cuatro tipologías de contenido curatorado. Las listas propias del usuario en TMDb se obtienen mediante `GET /account/{id}/lists` y permiten operaciones completas de CRUD: creación de nuevas listas mediante formulario modal con nombre y descripción (petición `POST /list`), adición o eliminación de títulos desde la página de detalle, y edición de metadatos. Las colecciones de saga de TMDb

(franquicias como el MCU, la saga de Bourne o el universo de Star Wars) se tratan como un tipo especial de lista que `CollectionDetailsClient.jsx` renderiza con un hero propio y un grid ordenado cronológicamente. Las listas públicas de Trakt se gestionan mediante `TraktListDetailsClient.jsx`, que implementa paginación con carga bajo demanda en páginas de 48 ítems mediante un botón «Cargar más» que consulta `/api/trakt/lists/{username}/{listId}?page={n}&limit=48`.

La página principal (`/lists`) precarga automáticamente los detalles de las primeras 3 listas del usuario (`PRELOAD_FIRST_N_LISTS = 3`) para navegación instantánea al hacer clic. Las listas se muestran como tarjetas con un collage compuesto de hasta cuatro backdrops de los primeros títulos organizado en variantes de cuadrícula (1×1 para listas con un título, 2×1 para dos, 2×2 para tres o más), generado por `ListCoverBackdropCollage()`. El diseño visual de cada lista incluye metadatos como el número de ítems, el nombre del creador y la descripción truncada, y permite ordenar las listas del usuario por fecha de creación, número de ítems o valoración media del contenido. `UnifiedListDetailsLayout.jsx` actúa como envoltorio de maquetación reutilizable para todas las páginas de detalle de lista, proporcionando el hero con título y metadatos, el sistema de pestañas con estado activo, y los controles superiores de filtrado y ordenación de forma consistente independientemente del tipo de lista (TMDb, colección o Trakt).

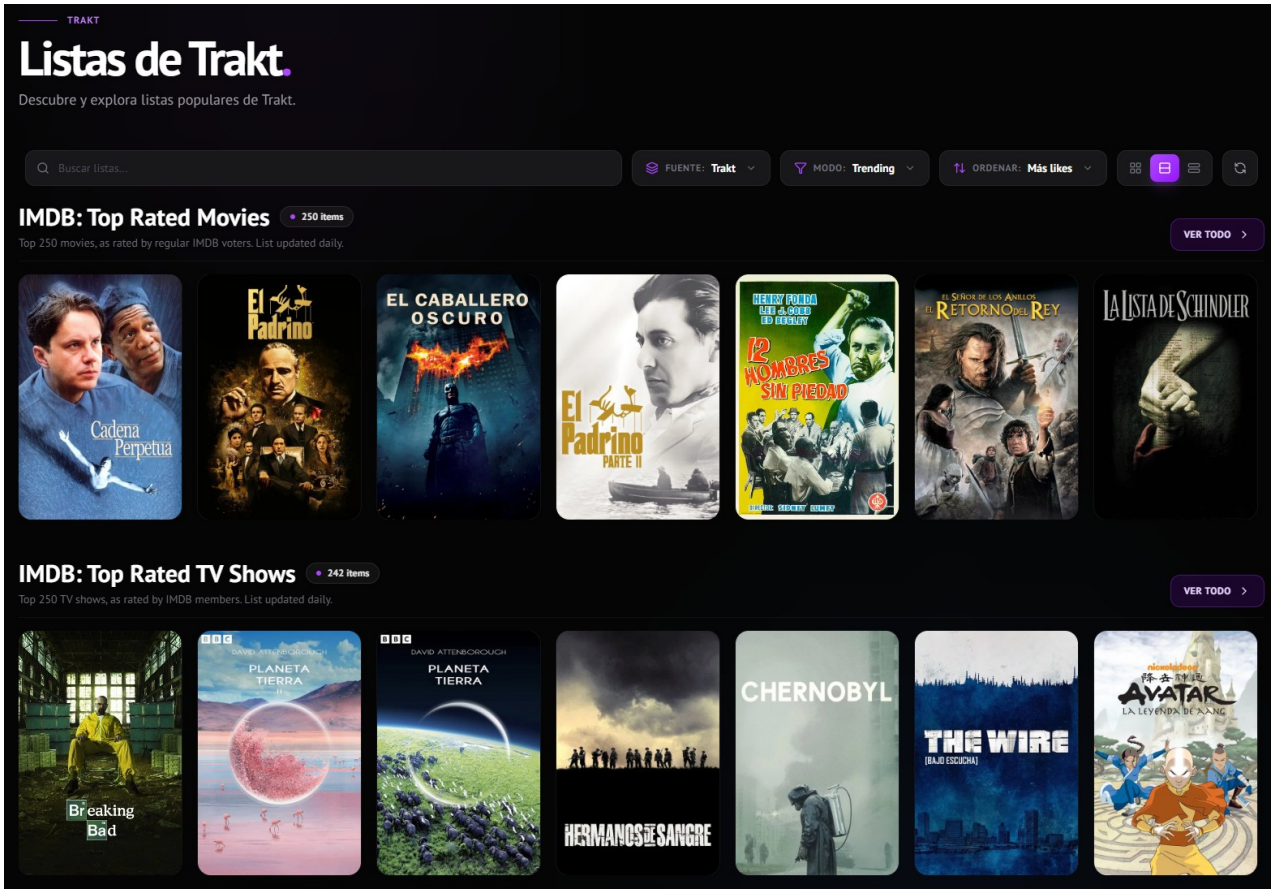


Figura 4.11: Interfaz de las listas públicas de Trakt.

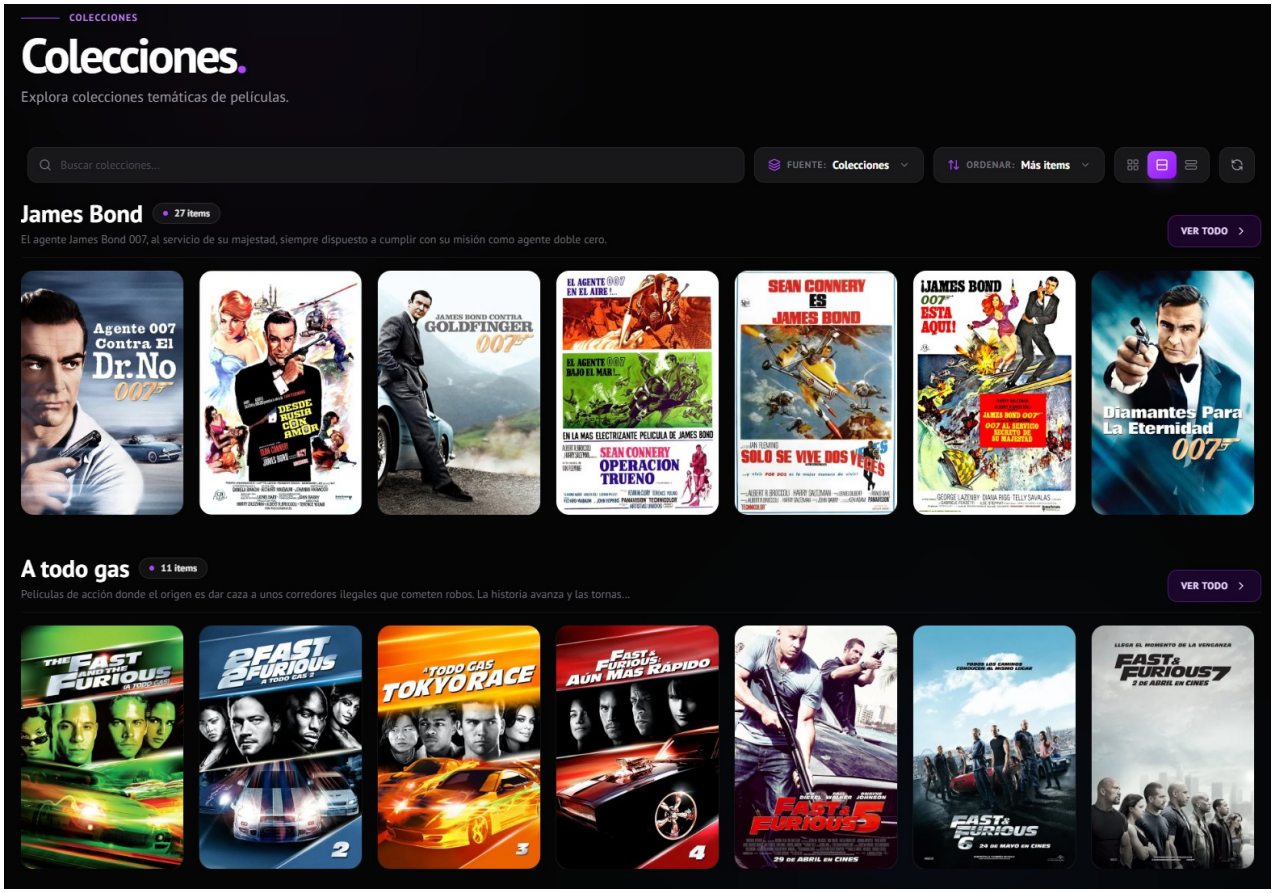


Figura 4.12: Interfaz de la sección de colecciones.

4.5.12 Películas y Series (`/movies`, `/series`)

Las páginas de películas y series especializan el dashboard principal en un único tipo de contenido. `MoviesPageClient.jsx` y `SeriesPageClient.jsx` comparten la arquitectura técnica, incluyendo sistemas de caché a nivel de módulo para extras (`movieExtrasCache`), backdrops (`movieBackdropCache`), imágenes (`movieImagesCache`) y trailers (`movieTrailerCache` con mapa de peticiones en vuelo para deduplicación). `pickBestBackdropByLangResVotes()` prioriza backdrops de resolución 1920×1080 sobre 2560×1440 y 3840×2160, priorizando la velocidad de carga sobre la calidad máxima de imagen. La selección de poster aplica `pickBestPosterByLangThenResolution()`, que calcula el área de cada imagen disponible (ancho × alto) y selecciona el de mayor resolución entre los disponibles en inglés o español, con *fallback* al `poster_path` original de TMDb.

La sección de trailers reproduce el tráiler oficial de cualquier título directamente en la página sin navegar al detalle. Cuando se hace hover sobre una tarjeta, `prefetchTrailer(id)` comprueba el caché de trailers en memoria y, si no hay datos cacheados, solicita los vídeos del título a TMDb mediante `GET /{type}/{id}/videos` y almacena el resultado. Al hacer clic en el botón de

reproducción, el componente abre `VideoModal` con el `videoKey` de YouTube del tráiler. Ambos módulos incluyen integración directa con los botones de favorito y watchlist de TMDb desde las tarjetas del grid, usando el patrón de actualización optimista descrito en el módulo de detalles, sin necesidad de navegar a la página de detalle para gestionar las listas personales.

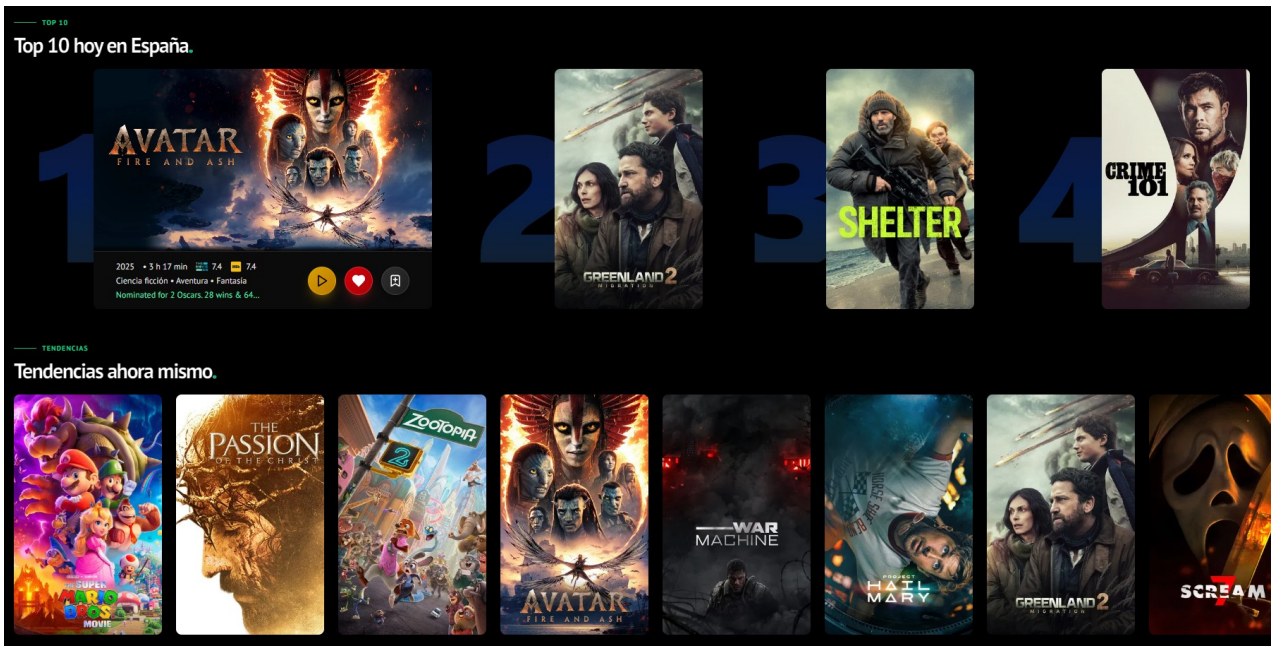


Figura 4.13: Interfaz de la sección de películas.



Figura 4.14: Interfaz de la sección de series.

4.5.13 Ficha de personas (`/details/person/[id]`)

La ficha de persona implementada en `ActorDetails.jsx` carga en paralelo mediante `Promise.allSettled()` cinco recursos independientes: identificadores externos (`/person/{id}/external_ids`), créditos combinados de películas y series (`/person/{id}/combined_credits?language=es-ES`), galería de fotos de perfil (`/person/{id}/images`), imágenes promocionales etiquetadas (`/person/{id}/tagged_images`) y traducciones disponibles (`/person/{id}/translations`), escogiendo la versión en español de la biografía si está disponible. El uso de `Promise.allSettled()` en lugar de `Promise.all()` garantiza que un fallo en uno de los recursos (por ejemplo, ausencia de imágenes etiquetadas) no cancele la carga de los demás.

La filmografía completa se computa como un valor derivado mediante `useMemo()` a partir del array `combined_credits`, normalizando el formato heterogéneo de los campos según si el ítem es una película (`release_date`) o una serie (`first_air_date`), y si el crédito es como actor (`character`) o como miembro del equipo técnico (`job`, `department`). Los filtros disponibles son: tipo de medio (todas, películas, series), tipo de crédito (actuación, equipo técnico), departamento (para filtrar directores, guionistas, productores dentro del equipo técnico), rango de años desde/hasta, y búsqueda libre por título o nombre de personaje. Todos los filtros se aplican de forma combinada en un único paso de procesamiento sobre el array completo, sin peticiones adicionales al servidor. La ordenación por popularidad, fecha, valoración o alfabético se aplica como etapa posterior al filtrado. El sistema calcula estadísticas sobre la filmografía filtrada (total de créditos, películas, series, actuaciones, roles técnicos) que se muestran como tarjetas informativas en la cabecera, actualizándose en tiempo real al cambiar los filtros. La función `calcAge()` calcula la edad actual o la edad en el momento del fallecimiento a partir de las fechas de nacimiento y muerte, y `formatHumanDate()` formatea las fechas en español con localización `es-ES`.

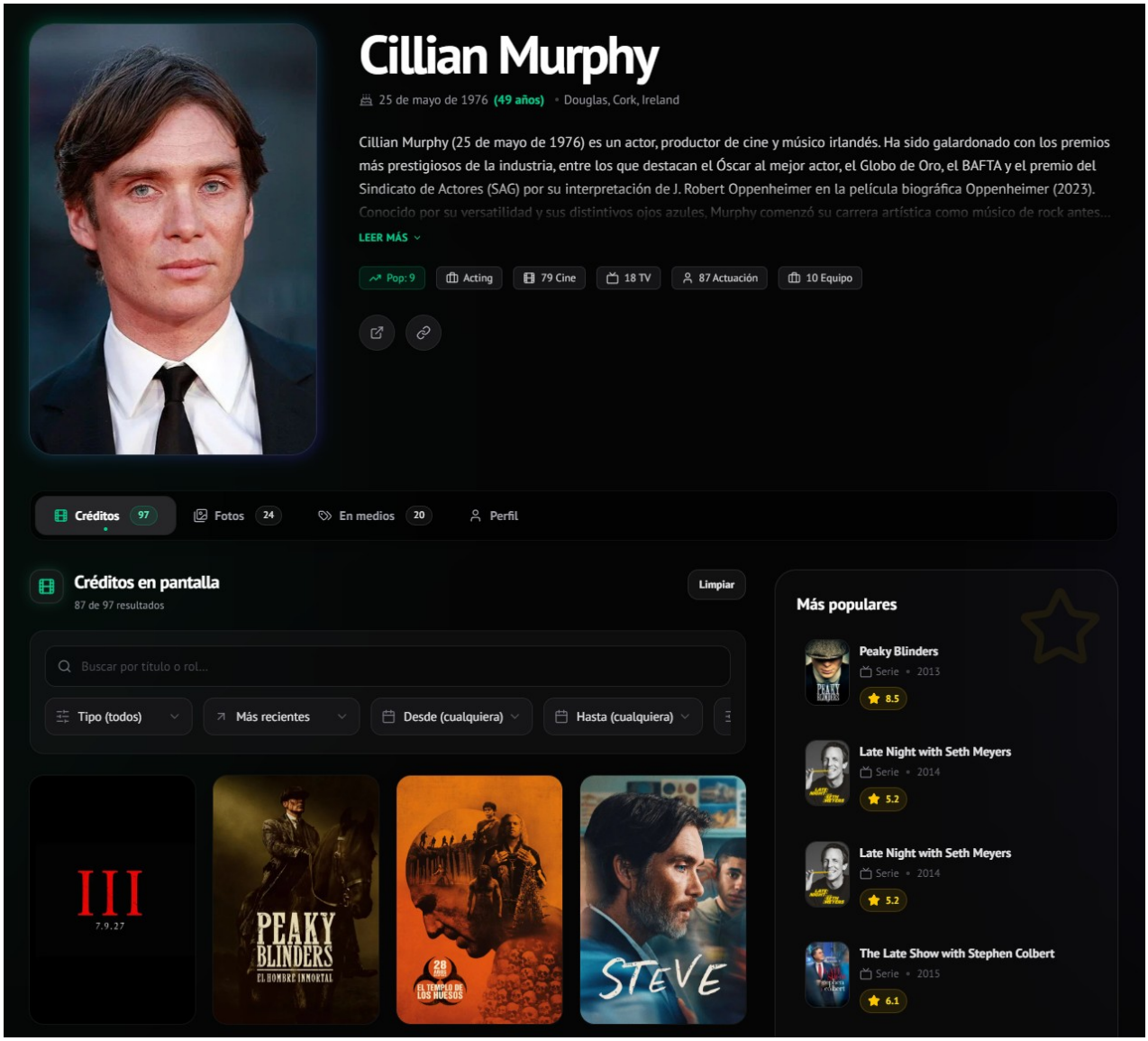


Figura 4.15: Interfaz de la ficha de persona.

4.5.14 Barra de navegación (Navbar)

La barra de navegación implementada en `Navbar.jsx` se encuentra presente en todas las páginas de la aplicación, ya que se instancia directamente en el `layout.jsx` raíz. En escritorio adopta una disposición horizontal en tres zonas: el logotipo y nombre de la aplicación a la izquierda, una barra de búsqueda centralizada, y a la derecha un conjunto de iconos de navegación con códigos de color por categoría temática (estadísticas en índigo, listas en morado, calendario en ámbar, series en progreso en verde, historial en rojo, favoritos en rojo, watchlist en azul). En móvil, la barra superior simplifica el contenido a logo, buscador y avatar, mientras que la navegación principal se traslada a una barra fija de iconos en la parte inferior de la pantalla accesible con el pulgar, complementada por un menú deslizable lateral (*drawer*) que contiene los enlaces secundarios.

La barra de búsqueda `SearchBar` se reutiliza tanto en la versión de escritorio como en la modal de búsqueda de móvil. Implementa un debounce de 300 ms antes de lanzar cada consulta a `/search/multi?query=...&language=es-ES`, y cancela las peticiones anteriores con `AbortController` para evitar que resultados de búsquedas superadas sobrescriban los de la búsqueda actual. El *dropdown* de resultados muestra hasta diez sugerencias con el tipo de contenido codificado por color (badge azul para películas, badge verde para series, badge naranja para personas), la foto de perfil o poster en miniatura, el año de estreno y el rating de TMDb. Una variante del componente Navbar incluye el botón `TraktHistoryNavButton` que muestra en tiempo real cuántos ítems hay en la lista de pendientes, obtenido del contexto de autenticación `AuthContext` sin petición adicional. El efecto de transición de transparente a *glassmorphism* al hacer *scroll* se implementa con un listener `scroll` que aplica o elimina la clase `bg-black/70 backdrop-blur-12px` al superar un umbral de 100 px.

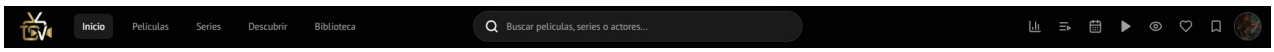


Figura 4.16: Barra de navegación en escritorio.

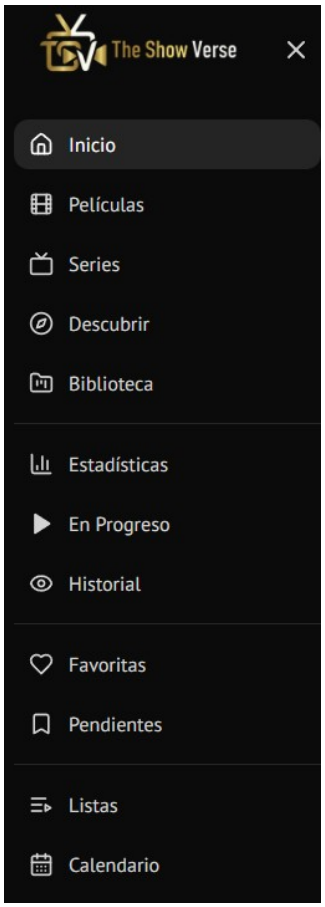


Figura 4.17: Barra de navegación en móvil.

4.5.15 Patrones de optimización transversales

La aplicación implementa un conjunto de patrones de optimización aplicados de forma sistemática que explican los tiempos de respuesta percibidos como inmediatos a pesar de integrar datos de seis APIs externas distintas.

Actualización optimista. Todas las acciones de escritura del usuario (marcar como favorito, añadir a pendientes, valorar) actualizan la interfaz instantáneamente sin esperar la confirmación del servidor. La petición real se envía en paralelo en segundo plano, y solo si falla se revierte el cambio visual mostrando un mensaje de error. Este patrón elimina la latencia perceptible de 300 a 600 milisegundos que tendría una actualización sincrónica, creando la sensación de respuesta inmediata característica de las aplicaciones nativas.

Precarga de imágenes. Las imágenes de posters y backdrops se precargan con un margen adelantado respecto al viewport, de forma que cuando el usuario hace scroll las imágenes ya están disponibles en la caché del navegador, evitando tiempos de carga visibles.

Control de concurrencia. Para respetar los límites de las APIs externas sin renunciar al paralelismo, se mantiene un máximo de 4 o 5 peticiones concurrentes. Esto reduce el tiempo de carga de colecciones paginadas de N peticiones secuenciales a aproximadamente N/4 rondas paralelas, mejorando el rendimiento entre 4 y 5 veces respecto a la ejecución secuencial.

Cancelación de peticiones obsoletas. Cuando el usuario navega a otra página mientras hay peticiones en curso, estas se cancelan automáticamente para prevenir condiciones de carrera y actualizaciones de estado en componentes ya desmontados.

Regeneración estática incremental (ISR). Los datos semi-estables como catálogos de tendencias y listas de mejor valoradas se sirven con regeneración cada 10 a 30 minutos, permitiendo que la mayoría de visitas utilicen páginas preconstruidas desde la CDN sin ejecutar peticiones a APIs externas.

4.6 Diseño de la interfaz

4.6.1 Sistema de diseño global

El sistema de diseño parte de un principio rector: el contenido visual proveniente de TMDb (backdrops y posters de cada título) es el protagonista de la interfaz. La paleta de colores, los efectos de superficie y las estrategias de iluminación se han diseñado para potenciar, no competir con, estas imágenes.

Paleta de colores. El color de fondo global es negro puro, sobre el cual se despliegan los backdrops

a pantalla completa con gradientes perimetrales que los integran suavemente. Este enfoque traslada la estética de las plataformas de streaming (modo teatro) a la interfaz de seguimiento.

Superficies de cristal (glassmorphism). Las tarjetas, paneles laterales, la barra de navegación y los menús desplegables utilizan un efecto de cristal semitransparente: fondo blanco con 5 % de opacidad, desenfoque de 10 píxeles del contenido subyacente y borde sutil de 1 píxel. La opacidad baja mantiene la legibilidad del texto mientras proyecta el color del backdrop en la superficie del elemento. El desenfoque se procesa mediante GPU y no genera sobrecarga en el renderizado.

Tipografía. El proyecto utiliza la familia Geist Sans y Geist Mono, tipografías de Vercel distribuidas como fuentes variables. Se cargan como fuentes del sistema sin peticiones externas, evitando el problema de *Flash of Unstyled Text* (FOUT) durante la carga inicial.

Accesibilidad y movimiento reducido. El sistema respeta las preferencias de accesibilidad del sistema operativo, desactivando automáticamente todas las transiciones y animaciones CSS para usuarios que han configurado la opción de movimiento reducido en su dispositivo.

4.6.2 Componentes de Next.js utilizados

`<Image>` de Next.js. Todos los pósters, *backdrops* y logos de la aplicación se renderizan mediante el componente `<Image>` de Next.js en lugar del elemento `<img>` nativo. Este componente aplica automáticamente:

- Conversión del formato original JPEG a WebP para los navegadores que lo soportan, reduciendo el peso de las imágenes entre un 25 y un 35 % sin pérdida visual apreciable.
- *Lazy loading* nativo: las imágenes fuera del *viewport* no se solicitan hasta que el usuario se aproxima a ellas.
- Reserva de espacio (*placeholder*) mediante `width` y `height` explícitos, evitando el salto de layout (CLS) que se produciría si la imagen tardase en cargar.
- Optimización automática a través del CDN de Vercel, que sirve cada imagen en el tamaño exacto requerido por el dispositivo del usuario.

`<Link>` de Next.js. La navegación entre páginas utiliza el componente `<Link>` del App Router de Next.js, que implementa *prefetching* automático de las rutas visibles en el *viewport*. En producción (Vercel), cuando el usuario se aproxima a un enlace, Next.js precarga en segundo plano los datos del Server Component de la ruta destino, logrando transiciones de página percibidas como instantáneas.

`usePathname` y estado activo en la Navbar. El componente `Navbar.jsx` utiliza el *hook* `usePathname()` de `next/navigation` para detectar la ruta activa y aplicar el estilo

visual correspondiente a cada enlace de navegación sin necesidad de estado global. La función `isActive(href)` comprueba si la ruta actual comienza con el *href* del enlace, marcando toda la sección como activa aunque el usuario esté en una subruta.

4.6.3 Componentes propios: Navbar

El componente `Navbar.jsx` (30 KB) implementa una navegación adaptativa con dos modos de presentación.

Modo escritorio (lg+). Una barra horizontal fija en la parte superior con tres zonas:

- **Izquierda**: logotipo de la aplicación y navegación principal (Inicio, Películas, Series, Descubrir, Biblioteca).
- **Centro** (posición absoluta): campo de búsqueda en tiempo real con *dropdown* de resultados.
- **Derecha**: botones de acceso rápido a Estadísticas, Listas, Calendario, En Progreso, Historial, Favoritas y Pendientes; avatar del usuario autenticado.

Cada botón de icono utiliza un sistema de estilos por «tono de color» (`iconLinkClass(href, tone)`): al hacer *hover*, el botón adquiere el color semántico de su función (rojo para Favoritas, azul para Pendientes, ámbar para Calendario, verde para En Progreso, etc.) junto con un halo de sombra coloreado:

```
hover:shadow-[0_0_18px_rgba(239,68,68,0.16)]
```

Cuando el estado está activo (ruta actual), el color se intensifica con un fondo semitransparente y una sombra más pronunciada.

Modo móvil. La barra superior se simplifica al logotipo centrado, un botón de menú (hamburgesa) y un botón de búsqueda. En la parte inferior de la pantalla aparece una barra de navegación fija (*tab bar*) con las cuatro secciones de mayor uso. Esta arquitectura de doble barra es el patrón estándar de las aplicaciones de contenido multimedia en iOS y Android.

El **drawer lateral** del menú móvil se implementa con Framer Motion:

```
<motion.aside
  initial={{ x: -320 }}
  animate={{ x: 0 }}
  exit={{ x: -320 }}
  transition={{ type: "tween", duration: 0.22 }}
>
```

El fondo oscuro detrás del drawer (`bg-black/70 backdrop-blur-sm`) también se anima con `AnimatePresence` para que tanto la entrada como la salida del menú sean fluidas.

La **búsqueda en móvil** abre un *overlay* a pantalla completa (`bg-black/95 backdrop-blur-2xl`) que entra con una animación de escala y opacidad (`scale: 0.95→1, opacity: 0→1`), dando la sensación de que emerge desde el botón de búsqueda.

4.6.4 Componente propio: LiquidButton

`LiquidButton.jsx` (18 KB) es el componente de botón de acción rápida utilizado en la página de detalle para las operaciones de favoritos, pendientes, historial y valoración personal. Es un Client Component completamente autónomo que implementa un sistema de efectos visuales interactivos mediante la API `Canvas 2D` del navegador.

La arquitectura del componente se basa en tres capas superpuestas dentro de un `<button>`:

1) **Canvas de efectos**: un elemento `<canvas>` que ocupa toda la superficie del botón y gestiona, mediante `requestAnimationFrame`, las partículas flotantes, las ondulaciones de *ripple* y los efectos de explosión al hacer clic.

2) **Superposición de brillo**: un `<div>` con un gradiente lineal animado mediante CSS (`@keyframes liquidShine`) que recorre la superficie del botón de forma continua cuando está activo o en *hover*.

3) **Borde pulsante**: un segundo `<div>` circular con borde coloreado animado mediante `@keyframes liquidPulse` (escala $1 \rightarrow 1.15 \rightarrow 1$, con periodo de 2 s).

El sistema de efectos de *canvas* implementa:

- **Partículas flotantes**: 12 partículas con movimiento senoidal suave que se generan al entrar en *hover*. Cada partícula tiene posición, velocidad, tamaño, opacidad y fase trigonométrica independientes.
- **Ripple (ondulación)**: al hacer *hover* o clic, se crean ondas circulares que se expanden desde el punto de contacto y se desvanecen al alcanzar su radio máximo.
- **Trail effect**: al mover el ratón sobre el botón, se generan puntos de rastro que se desvanecen en 0,4 s, creando un efecto de «estela líquida».
- **Explosión**: al hacer clic, se generan 12 partículas que se dispersan en ángulos uniformes desde el punto de impacto con gravedad simulada (`vy += 0.12`) y decaimiento de opacidad.

El componente soporta un sistema de **propagación entre botones del mismo grupo** (`groupId`): al interactuar con un botón, este emite un `CustomEvent` al documento (`liquid-spread`, `liquid-click`, `liquid-proximity`) que es capturado por todos los botones del mismo grupo. Los botones próximos a la fuente del evento reaccionan con ondulaciones de intensidad proporcional a la distancia, creando la sensación de que el líquido se propaga físicamente entre botones adyacentes.

El color activo del botón es configurable mediante la prop `activeColor` (blue, red, yellow, purple, green, teal, orange), lo que permite usar el mismo componente para acciones semánticamente distintas manteniendo coherencia visual.



Figura 4.18: Botones de acción implementados con el componente LiquidButton.

4.6.5 Componente propio: StarRating

`StarRating.jsx` gestiona la valoración personal del usuario en Trakt (escala 1–10). Utiliza `LiquidButton` como *trigger* con `activeColor="yellow"`: si el usuario ya tiene una valoración, el botón muestra el valor numérico con el efecto de partículas activo; si no, muestra el icono de estrella.

Al hacer clic, el componente abre un modal renderizado mediante `createPortal()` directamente en `document.body`, evitando problemas de apilamiento CSS (`z-index`) con los elementos de la página de detalle. El modal contiene:

- Un número grande que muestra el valor actual centrado sobre una estrella decorativa semitransparente.
- Un *slider* HTML nativo (`<input type="range">`) con paso de 0,5 y recorrido de 1 a 10, cuyo *thumb* nativo se oculta con `opacity: 0` y se sustituye por un *thumb* personalizado posicionado con CSS (`left: calc(...)`).
- Botones de ajuste fino $-0{,}5$ / $+0{,}5$ a ambos lados del *slider*.
- Botón de confirmación y, si ya existe una valoración, botón de borrado con icono de papelera.



Figura 4.19: Interfaz del componente de valoración personal StarRating.

4.6.6 Componente propio: DetailsSectionMenu

`DetailsSectionMenu.jsx` gestiona la navegación interna de la página de detalle mediante pestañas contextuales. Su función principal es organizar el contenido por bloques funcionales (Media, Comentarios, Listas, Reparto, Recomendaciones, Temporadas y Episodios) y adaptar automáticamente qué pestañas se muestran según el tipo de contenido (película o serie) y según la disponibilidad real de datos.

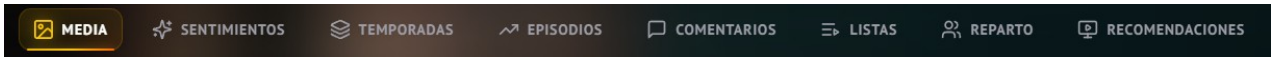


Figura 4.20: Interfaz del componente DetailsSectionMenu.

4.6.7 Componente propio: EpisodeRatingsGrid

`EpisodeRatingsGrid.jsx` presenta una vista matricial de episodios para series, donde cada celda representa un episodio y su color refleja la puntuación. Este componente permite detectar de forma rápida patrones de calidad por temporada y ofrece navegación directa al detalle de episodio cuando el usuario hace clic en una celda.

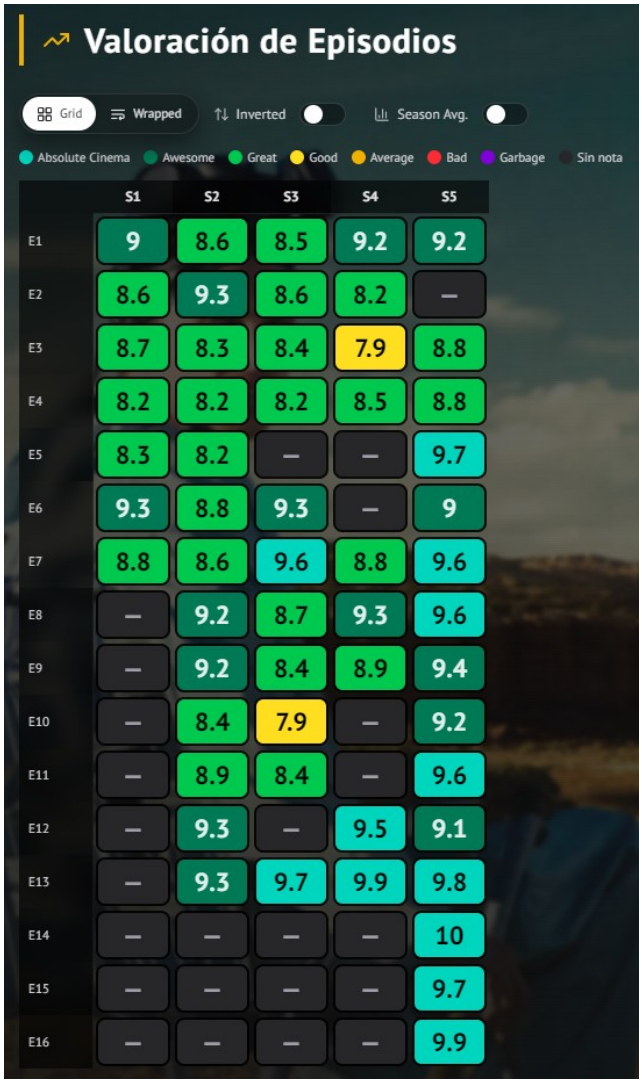


Figura 4.21: Interfaz del componente EpisodeRatingsGrid.

4.6.8 Componente propio: ScoreboardBar

`ScoreboardBar.jsx` centraliza en una sola franja la información de valoración y contexto del título, para evitar que el usuario tenga que consultar varias secciones distintas de la ficha. En este componente se agrupan:

- Puntuaciones de TMDb, Trakt e IMDb.
- Número de votos asociado a las fuentes disponibles.
- Estadísticas de Trakt vinculadas al seguimiento del título.
- Botón de compartir para difundir directamente el contenido.

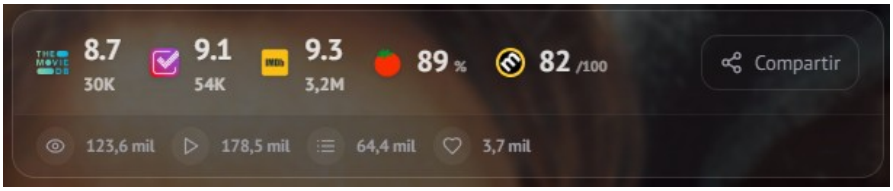


Figura 4.22: Interfaz del componente ScoreboardBar.

4.6.9 Componentes propios: control de visionado en Trakt

El bloque de control de visionado se apoya en tres componentes coordinados:

- `TraktEpisodesWatchedModal`: modal para gestionar el marcado de episodios en series de forma más detallada.
- `TraktWatchedModal`: modal de historial de visionados del título en Trakt.

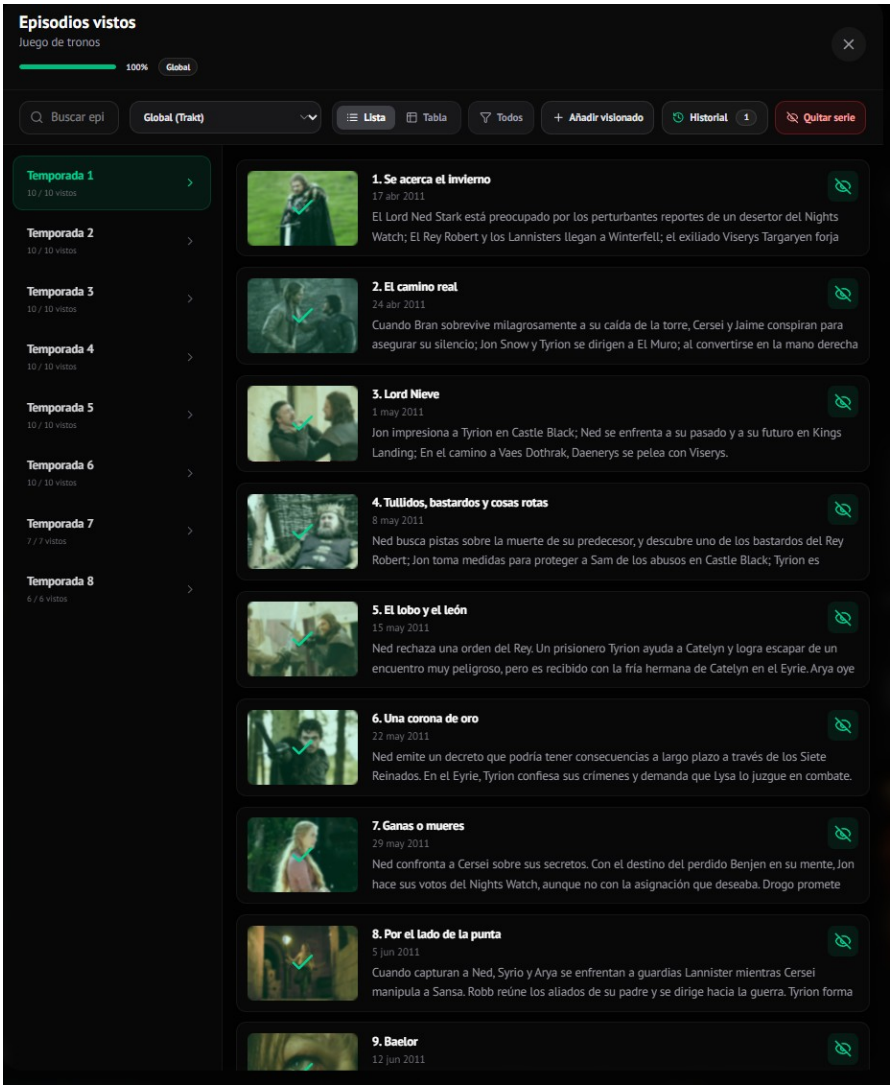


Figura 4.23: Interfaz del modal TraktEpisodesWatchedModal para gestionar episodios vistos.

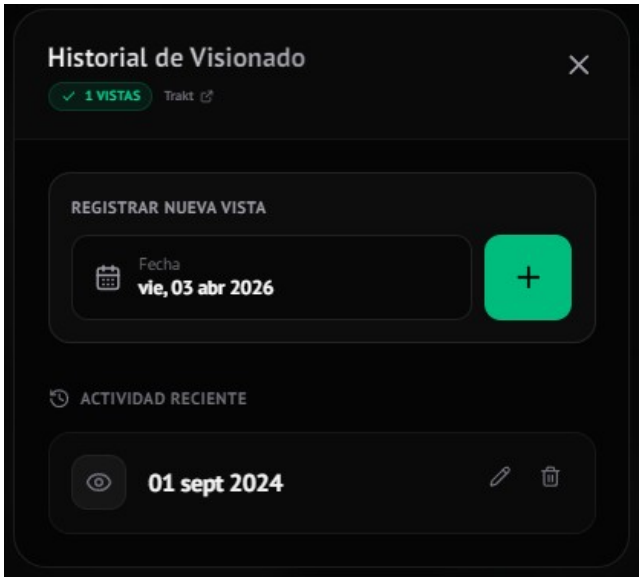


Figura 4.24: Interfaz del modal TraktWatchedModal con el historial de visionado.

En conjunto, estos componentes centralizan las acciones de seguimiento y sincronización de visionado sin sacar al usuario de la ficha.

4.6.10 Componentes propios: modales de apoyo en DetailsClient

`DetailsClient.jsx` utiliza varios modales especializados para acciones concretas:

- `AddToListModal`: añadir o quitar el título de listas personalizadas de TMDb.
- `VideoModal`: reproducir trailers y vídeos oficiales dentro de la propia página.
- `ExternalLinksModal`: agrupar enlaces externos en móvil cuando no hay espacio para mostrar todos los iconos.

Como apoyo adicional en la misma pantalla también es relevante `SeasonDetailsClient` (detalle expandible de temporadas y episodios).

4.6.11 Sistema de animaciones con Framer Motion

Framer Motion gestiona las animaciones de mayor complejidad del proyecto a través de cuatro patrones principales:

1. Animaciones de entrada escalonada (`staggerChildren`). En el dashboard y en las páginas de listas, las tarjetas no aparecen todas simultáneamente. La variante del contenedor define un `staggerChildren` de 0,05 s (reducido a 0,02 s para listas largas de más de 30 ítems), de forma que cada tarjeta entra con un ligero retraso respecto a la anterior:

```
const containerVariants = {
  hidden: { opacity: 0 },
  visible: {
    opacity: 1,
    transition: { staggerChildren: 0.05 }
  }
};
const itemVariants = {
  hidden: { opacity: 0, y: 20 },
  visible: { opacity: 1, y: 0 }
};
```

2. `AnimatePresence` para salidas animadas. El sistema de tres vistas (Grid, List, Compact) y las pestañas de la página de detalle utilizan `AnimatePresence` para animar tanto la entrada como la salida de los elementos del DOM. Cuando el usuario cambia de vista, el contenido saliente realiza una animación de salida (`exit`) antes de ser eliminado del DOM, evitando cortes bruscos:

```
<AnimatePresence mode="wait">
  <motion.div
    key={currentView}
    initial={{ opacity: 0, x: 20 }}
    animate={{ opacity: 1, x: 0 }}
    exit={{ opacity: 0, x: -20 }}
    transition={{ duration: 0.2 }}
  />
</AnimatePresence>
```

El modo `"wait"` garantiza que la animación de salida se complete antes de que empiece la de entrada.

3. Drawer y overlays móviles. El menú lateral, la búsqueda en *overlay* móvil y los paneles modales utilizan `motion.div` con valores iniciales fuera del *viewport*: `x: -320` para el drawer (deslizamiento desde la izquierda), `scale: 0.95, opacity: 0` para los modales (apertura con escala). Las transiciones de tipo `tween` con `ease: "easeOut"` producen una deceleración natural al final del movimiento.

4. Carrusel del hero. El carrusel automático de la página principal anima las transiciones entre slides con una combinación de `opacity` (entrada: 0→1, salida: 1→0) y un ligero desplazamiento

horizontal (`x: +20→0`). La pausa automática al hacer *hover* se gestiona mediante un `useEffect` que detecta el evento `mouseenter` del contenedor del hero.

Performance de las animaciones. Todas las animaciones de Framer Motion utilizan exclusivamente las propiedades CSS `transform` (para `x`, `y`, `scale`, `rotate`) y `opacity`, que el navegador puede procesar enteramente en la GPU sin necesidad de recalcular el *layout* ni repintar capas del DOM. Esto garantiza que las animaciones se ejecuten a 60 fps incluso en dispositivos de gama media.

4.6.12 Sistema de estilos global (`globals.css`)

El fichero `globals.css` define un conjunto de utilidades y animaciones CSS que complementan a Tailwind CSS:

- **`.no-scrollbar`**: oculta la barra de desplazamiento en los carruseles horizontales y los paneles de pestañas, sin deshabilitar el *scroll* táctil.
- **`.sv-scroll`**: barra de desplazamiento personalizada para los paneles con mucho contenido (lista de episodios, resultados de búsqueda): fina, semitransparente, con bordes redondeados y espacio interior.
- **`.animate-shimmer`**: efecto de carga tipo *skeleton* con un gradiente que avanza horizontalmente (`background-position: -200% → 200%`) sobre los *placeholders* de las tarjetas mientras los datos se cargan.
- **`.hover-lift`**: eleva el elemento 4 px y añade una sombra al hacer *hover*, indicando que es interactivo.
- **`.glass / .glass-dark`**: superficies de *glassmorphism* en variante clara y oscura reutilizables en cualquier componente.

4.6.13 Sistema de tres vistas (Grid, List, Compact)

El sistema de tres vistas es un componente transversal utilizado en Favoritos, Pendientes, Historial y En Progreso. La selección de vista se persiste en `localStorage` bajo una clave específica por módulo (`showverse:view:favorites`, `showverse:view:history`, etc.), de forma que el usuario puede tener configuraciones distintas en cada sección.

La transición entre vistas utiliza `AnimatePresence` de Framer Motion con `mode="wait"` y un efecto de *cross-fade* suave: el contenido saliente desaparece con `opacity: 1→0` antes de

que el nuevo contenido entre con `opacity: 0→1`. Esto evita los saltos bruscos de *layout* que se producirían si los dos estados coexistieran en el DOM simultáneamente.

4.7 Rendimiento en producción

Las métricas obtenidas en producción (Vercel) mediante Lighthouse validan las decisiones de arquitectura tomadas:

Métrica	**Valor obtenido**	**Objetivo**
Performance (Lighthouse)	92/100	>90
SEO (Lighthouse)	100/100	100
Best Practices	95/100	>90
LCP	1,8 s	<2,5 s
CLS	0,05	<0,1
FID	45 ms	<100 ms

Tabla 4.5: Métricas Lighthouse en producción

El LCP de 1,8 s se consigue principalmente gracias al SSR: el HTML con el *hero* ya renderizado llega al navegador antes de que se descargue todo el JavaScript. El CLS de 0,05 se logra mediante el uso sistemático de `width` y `height` explícitos en el componente `<Image>` de Next.js (Google Developers, 2024).

4.8 Sistema de caché multinivel

El rendimiento de *The Show Verse* depende críticamente de la estrategia de caché implementada, que opera en cuatro niveles diferenciados según el tipo de dato y su tasa de cambio.

4.8.1 Nivel 1: Incremental Static Regeneration (ISR)

Las páginas que consumen datos de TMDb en el servidor utilizan ISR mediante la opción `next: {revalidate: TTL}` del sistema de caché de Next.js. Por defecto, el módulo `lib/api/tmdb.js` aplica una revalidación de 10 minutos para todas las peticiones realizadas en Server Components, equilibrando frescura de datos con carga del servidor.

Este valor puede sobrescribirse pasando opciones personalizadas en casos específicos. Para datos personales que requieren autenticación (favoritos, historial, watchlist), las peticiones se configuran con `cache: no-store` para evitar que se sirvan datos obsoletos de otros usuarios.

4.8.2 Nivel 2: Caché en memoria del proceso Node.js

La API Route `/api/omdb` implementa un sistema de caché persistente durante toda la vida del proceso serverless de Node.js mediante estructuras `Map` almacenadas en `globalThis`. Esta caché distingue entre respuestas exitosas y fallidas con estrategias de TTL diferenciadas.

Las respuestas exitosas se almacenan durante 24 horas, permitiendo que múltiples peticiones HTTP reutilicen el mismo dato sin consultar la API externa de OMDb. Las respuestas fallidas (errores 404, timeouts de red) se cachean solamente durante 60 segundos, lo que permite reintentos rápidos sin sobrecargar la API externa si el problema es temporal.

Al ejecutarse en Vercel, esta caché persiste mientras la instancia serverless se mantenga activa. Las instancias frías pierden la caché y deben reconstruirla, pero el sistema garantiza que esto no afecte a la experiencia del usuario gracias al nivel siguiente de deduplicación.

4.8.3 Nivel 3: Deduplicación de peticiones concurrentes

El sistema implementa un mecanismo de deduplicación mediante un segundo `Map` global que registra las promesas de peticiones HTTP en curso. Cuando múltiples componentes solicitan simultáneamente el mismo `imdbId` (por ejemplo, 10 tarjetas cargándose en paralelo), solo se ejecuta una petición real a OMDb.

El resultado de esta petición única se comparte con todos los solicitantes concurrentes. Una vez completada la petición, la promesa se elimina del mapa de deduplicación y el resultado se almacena en la caché persistente del Nivel 2. Esta estrategia reduce el tráfico hasta 10 veces durante la carga inicial de listas extensas.

4.8.4 Nivel 4: Caché en el navegador (sessionStorage)

Los componentes del cliente que muestran listas de contenido implementan caché de puntuaciones en el almacenamiento de sesión del navegador. Los datos de OMDb (ratings de IMDb, Rotten Tomatoes y Metacritic) se almacenan bajo la clave `showverse:omdb:{imdbId}` con un tiempo de vida de 24 horas.

Esta caché persiste durante toda la navegación del usuario, incluyendo recargas de página, pero se elimina automáticamente al cerrar la pestaña del navegador. El sistema valida la frescura de los datos

comparando la marca de tiempo almacenada con el tiempo actual antes de utilizarlos, descartando automáticamente los datos expirados.

4.9 Middleware y optimización SEO

El sistema implementa una estrategia de optimización para asegurar que los bots de redes sociales y motores de búsqueda puedan indexar correctamente el contenido de la aplicación. Esto es fundamental porque, al ser una aplicación React basada en componentes del lado del cliente, los bots no ejecutarían el JavaScript y recibirían un HTML vacío sin metadatos útiles.

4.9.1 Detección de bots y reescritura de rutas

El middleware intercepta las peticiones entrantes e identifica si provienen de bots (WhatsApp, Facebook, Twitter, Telegram, Discord, LinkedIn, Googlebot, Bingbot, etc.) mediante el análisis del encabezado `User-Agent`. Cuando se detecta un bot accediendo a una página de detalle (como `/details/movie/550`), el sistema reescribe transparentemente la URL a una ruta alternativa (`/s/movie/550`) que sirve contenido HTML estático con todos los metadatos necesarios.

Esta reescritura es completamente transparente: la URL visible no cambia, pero internamente el bot recibe una respuesta HTML completa con toda la información de Open Graph y Twitter Cards. El patrón limita su ejecución exclusivamente a las rutas de detalle, evitando cualquier sobrecarga en el resto de la aplicación.

4.9.2 Generación de HTML optimizado para compartir

Las rutas alternativas (`/s/*`) construyen páginas HTML estáticas que incluyen:

- **Metadatos Open Graph**: título, descripción, tipo de contenido (película, serie o persona), URL canónica e imagen representativa.
- **Twitter Cards**: información adaptada para previsualizaciones en Twitter/X.
- **Selección inteligente de imágenes**: se prioriza el uso de backdrops horizontales (1280×720 píxeles) sobre posters verticales, ya que las redes sociales muestran previsualizaciones en formato apaisado que aprovechan mejor este tipo de imágenes.
- **Redirección automática para usuarios reales**: si un humano accede directamente a estas rutas, se le redirige inmediatamente a la página interactiva completa mediante una metaetiqueta `refresh`.

El sistema aplica una estrategia de caché de dos niveles: la CDN almacena estas respuestas durante 24 horas y puede servir versiones ligeramente obsoletas hasta 7 días más mientras regenera el contenido en segundo plano. Esto permite que el 99 % de las peticiones de bots se resuelvan desde la CDN sin ejecutar código en el servidor, reduciendo drásticamente la carga y mejorando los tiempos de respuesta.

4.9.3 Metadatos globales y por página

El archivo `layout.jsx` raíz define los metadatos por defecto de la aplicación mediante el objeto estático `metadata`:

```
export const metadata = {
  title: 'The Show Verse',
  description: 'Tu plataforma de películas y series',
  manifest: '/site.webmanifest',
  icons: {
    icon: [
      { url: '/icon.png', sizes: '256x256', type: 'image/png' },
      { url: '/icon-512.png', sizes: '512x512' },
    ]
  }
}
```

4.9.4 Resumen de la arquitectura dual de rutas

El flujo completo de una petición a `/details/movie/550` funciona de la siguiente manera:

1) La petición llega al *middleware* Edge de Next.js.

2) El middleware evalúa el `User-Agent` contra la expresión regular `BOT_UA`.

3) **Si es un bot:** reescritura transparente a `/s/movie/550`. El Route Handler devuelve HTML con Open Graph completo, cacheado 24h en CDN.

4) **Si es un usuario:** la petición pasa sin modificar a `/details/movie/550`. Se ejecuta el Server Component que hidrata el Client Component interactivo.

Esta arquitectura garantiza:

- Los bots indexan las rutas `/s/*` con metadatos completos (OGP, Twitter Cards, canonical).
- Los usuarios navegan las rutas `/details/*` con el Client Component completo e interactivo.
- No hay contenido duplicado: un usuario que acceda directamente a `/s/movie/550` es redirigido por el `<meta refresh>` a la ruta interactiva.
- El 99 % del tráfico de bots se resuelve desde la CDN sin invocar funciones serverless.

4.10 Sistema de artwork personalizado

El sistema de *artwork* personalizado permite sustituir manualmente el *poster*, *backdrop* o imagen de fondo asociados a una película o serie. Para ello se utiliza un archivo de configuración en la raíz del proyecto, desde el que se definen sobrescrituras vinculadas al identificador de TMDb de cada obra.

4.10.1 Motivación

TMDb selecciona automáticamente el *poster* con mayor número de votos para cada región lingüística. Sin embargo, en algunos casos el *poster* automático no es el más representativo:

- El póster en español puede ser visualmente inferior al en inglés.
- Para series antiguas, el póster preferido puede ser el original de su estreno y no el rediseñado recientemente.
- Algunos títulos tienen pósters promocionales con mejor reconocimiento visual que el póster oficial.

Las sobrescrituras pueden apuntar tanto a imágenes alojadas en la carpeta pública del proyecto como a recursos externos o a rutas de la propia galería de TMDb, lo que ofrece flexibilidad suficiente para adaptar la identidad visual de cada ficha.

4.10.2 Funcionamiento general

Cuando la aplicación necesita mostrar una imagen principal, comprueba primero si existe una sobrescritura definida para ese título. Si la encuentra, utiliza esa referencia; en caso contrario, mantiene el comportamiento por defecto y recurre al material obtenido automáticamente desde TMDb. Este mecanismo introduce una capa de personalización no intrusiva, ya que permite corregir casos puntuales sin alterar el resto del flujo de obtención de datos.

El sistema también dispone de una API interna para consultar y actualizar estas preferencias de forma dinámica. De este modo, la gestión del *artwork* puede realizarse sin modificar manualmente el código fuente, lo que simplifica el mantenimiento y facilita futuras tareas de administración del catálogo.

Los tipos de recurso admitidos incluyen:

- Ruta relativa al *public folder* (`/custom/...`).
- Ruta de imagen de TMDb, resuelta automáticamente contra su CDN.
- URL externa completa (`https://...`).

4.11 Integración con comunidad Trakt

4.11.1 Sistema de comentarios

La página de detalle incluye una pestaña «Comunidad» que muestra los comentarios públicos de Trakt para el título actual. API Route implementada: `/api/trakt/community/comments`.

Ordenación disponible:

- `likes`: comentarios con más *likes* primero (default).
- `newest`: más recientesprimero.
- `oldest`: históricos primero.
- `replies`: con más respuestas primero.

Paginación: 20 comentarios por página, con carga bajo demanda al hacer *scroll* hasta el final (`IntersectionObserver`).

Renderizado: cada comentario muestra:

- Avatar y username del autor (link a perfil Trakt).
- Texto del comentario (Markdown renderizado con `react-markdown`).
- *Spoiler warning*: los comentarios marcados como *spoiler* en Trakt están ocultos por defecto, requieren clic para expandir.
- Contador de *likes* y respuestas.
- Fecha de publicación relativa («hace 3 días»).

4.11.2 Listas de la comunidad

La ruta `/api/trakt/community/lists` obtiene listas públicas populares de Trakt para un título:

```
GET /api/trakt/community/lists?type=movie&tmdbId=550
    &tab=popular&page=1&limit=10
```

Pestañas disponibles:

- `popular`: listas con más *likes*.
- `trending`: listas con actividad reciente.
- `personal`: listas del usuario autenticado que contienen este título.

Cada lista muestra: nombre, descripción, autor, número de ítems, número de *likes*, y botón de acceso rápido para ver la lista completa en `/lists/trakt/[listId]`.

4.11.3 Estadísticas de temporada

Para series, la pestaña «Comunidad» incluye ratings promedio de Trakt por temporada y episodio, obtenidos de `/api/trakt/community/seasons`.

4.12 Integración con Letterboxd

Letterboxd es la plataforma de referencia para cinéfilos y críticos de cine, con más de 10 millones de usuarios. Aunque no tiene API pública, *The Show Verse* implementa integración mediante scraping controlado.

4.12.1 Búsqueda de películas en Letterboxd

La API Route `/api/links/letterboxd` construye URLs de búsqueda en Letterboxd:

```
GET /api/links/letterboxd?title=Inception&year=2010

Response:
{
  "searchUrl": "https://letterboxd.com/search/Inception/",
```

```
  "filmSlug": "inception-2010",
  "directUrl": "https://letterboxd.com/film/inception-2010/"
}
```

Algoritmo de construcción del slug:

1) Normalizar título: minúsculas, eliminar caracteres especiales, espacios → guiones.

2) Añadir año si el título es ambiguo (ej. «Dune» → `dune-2021` vs. `dune-1984`).

3) Validar existencia mediante `HEAD` a la URL construida (código 200 confirma).

Si la URL directa falla, se devuelve la URL de búsqueda como fallback.

4.12.2 Botón de acceso rápido

La página de detalle incluye un botón «Ver en Letterboxd» en la sección de enlaces externos que abre la película directamente en Letterboxd en nueva pestaña. Útil para usuarios que desean:

- Leer reseñas detalladas de la comunidad cinéfila.
- Añadir la película a sus listas personales de Letterboxd.
- Ver las listas de la comunidad que incluyen esta película.

4.13 Manejo de errores y estados de carga

4.13.1 Estrategia de *graceful degradation*

Todas las integraciones con APIs externas implementan un patrón de degradación elegante mediante gestión de excepciones que permite continuar con funcionalidad reducida en lugar de bloquear la interfaz completa. Cuando una API externa no responde o devuelve un error, la aplicación captura la excepción, registra el problema en consola para diagnóstico, y asigna valores nulos o por defecto a los datos que no pudieron obtenerse.

Por ejemplo, si la API de OMDb no está disponible al cargar los detalles de una película, el rating de IMDb simplemente se muestra como «N/A» en lugar de impedir que se renderice toda la tarjeta. De esta forma, el usuario puede seguir accediendo al resto de información proveniente de TMDb y Trakt sin interrupciones. Este patrón se aplica consistentemente en todas las peticiones a servicios externos, garantizando que ningún fallo aislado comprometa la experiencia general.

4.13.2 Skeleton loaders

Durante la carga de datos, los componentes renderizan *skeletons* animados que simulan la estructura visual del contenido final. Estos componentes de carga utilizan animaciones de pulso implementadas con Tailwind CSS que crean un efecto de brillo sutil sobre fondos grises, proporcionando feedback visual inmediato al usuario mientras se completan las peticiones asíncronas.

El sistema incluye diferentes tipos de *skeletons* especializados según el contenido que representan. El *PosterSkeleton* reproduce la proporción 2:3 de los pósters de películas y series. El *CardSkeleton* simula tarjetas completas con bloques rectangulares que representan título, descripción y metadatos. El *GridSkeleton* genera una cuadrícula de *skeletons* para listas de resultados. El *ScoreboardSkeleton* muestra una barra con círculos animados que anticipan la posición de los indicadores de puntuación.

Todos estos componentes se definen de forma centralizada en un módulo dedicado y se reutilizan a lo largo de la aplicación, garantizando consistencia visual en los estados de carga.

4.13.3 Mensajes de error contextuales

Los errores críticos que requieren atención del usuario, como fallos de autenticación o timeouts de red, se comunican mediante notificaciones no intrusivas que aparecen temporalmente en la esquina superior derecha de la pantalla siguiendo el patrón *toast*. Estas notificaciones se generan dinámicamente como elementos del DOM con estilos diferenciados según el tipo de mensaje (error, advertencia, éxito o información).

El sistema crea el elemento visual, lo inserta en el documento, y programa su eliminación automática tras cinco segundos. Este comportamiento temporal evita que el usuario tenga que cerrar manualmente cada notificación y previene la acumulación de mensajes en pantalla, manteniendo la interfaz limpia mientras se proporciona feedback claro sobre operaciones críticas.

4.13.4 Estados vacíos

Las listas vacías se gestionan con componentes específicos que muestran ilustraciones y mensajes motivacionales en lugar de dejar el espacio en blanco. Cuando el usuario no tiene favoritos, historial de visionado o elementos en su lista de pendientes, se renderiza un estado vacío ilustrado que explica la situación e invita a la acción.

Estos estados incluyen un icono representativo de gran tamaño, un título que describe el estado vacío, un mensaje secundario que explica cómo poblarlo, y un botón de llamada a la acción que redirige al usuario hacia la funcionalidad relevante. Por ejemplo, el estado vacío de favoritos guía al usuario hacia la página de descubrimiento de contenido, facilitando que comience a explorar y

agregar títulos a su colección personal. Este enfoque mejora la primera experiencia del usuario y reduce la fricción en la adopción de funcionalidades.

4.14 Diseño responsive y adaptación móvil

4.14.1 Breakpoints y estrategia mobile-first

El diseño utiliza los breakpoints estándar de Tailwind CSS:

Breakpoint	Ancho mínimo	Dispositivos objetivo
`sm`	640 px	Móviles grandes (landscape)
`md`	768 px	Tablets
`lg`	1024 px	Laptops
`xl`	1280 px	Desktops
`2xl`	1536 px	Pantallas grandes

Tabla 4.6: Breakpoints responsive

La estrategia *mobile-first* implica que todos los estilos base se definen para móvil (320 px), y se sobrescriben progresivamente para pantallas más grandes:

```
<div className="grid grid-cols-2 md:grid-cols-4 lg:grid-cols-6 gap-4">
```

4.14.2 Barra de navegación adaptativa

El componente Navbar implementa dos arquitecturas completamente diferentes según el dispositivo:

Desktop (lg+):

- Barra horizontal superior con tres zonas (logo, búsqueda, links).
- Búsqueda siempre visible en centro.
- Todos los links principales visibles simultáneamente.
- Dropdown de avatar en esquina superior derecha.

Móvil (<lg):

- Barra superior simplificada: logo centrado + botón menú + botón búsqueda.
- *Tab bar* inferior fijo con 4 secciones principales (Inicio, Descubrir, Favoritos, Perfil).
- Drawer lateral deslizante (`translate-x`) para menú completo.
- Búsqueda en *overlay* a pantalla completa.

La transición entre modos ocurre automáticamente en `1024px` sin recarga de página.

4.14.3 Imágenes responsive

El componente `<Image>` de Next.js genera automáticamente `srcset` con múltiples resoluciones:

```
<Image
  src="/poster.jpg"
  width={300}
  height={450}
  sizes="(max-width: 768px) 50vw,
         (max-width: 1024px) 33vw,
         25vw"
  alt="Poster"
/>
```

El navegador selecciona la resolución óptima según el tamaño de renderizado real. Resultado: ahorro de ancho de banda en móviles (imagen 300 px en lugar de 1000 px).

4.15 Plan de calidad: estrategia de testing

El proyecto no incluyó una suite de tests automatizados durante su desarrollo. La validación se realizó mediante pruebas manuales sistemáticas en navegador, una decisión pragmática dada la velocidad de iteración durante el prototipado. Esta limitación técnica debe abordarse antes de escalar el proyecto a producción.

La estrategia propuesta para futuras iteraciones incluye **Vitest** para pruebas unitarias e integración, **React Testing Library** para validar componentes desde el comportamiento del usuario, **MSW** para simular respuestas de APIs externas, y **Playwright** para pruebas *end-to-end* de flujos completos.

Los módulos que requieren mayor cobertura de tests son: algoritmo de caché y deduplicación en vuelo, middleware de detección de bots, flujo OAuth y gestión de tokens, transformaciones de datos

entre Trakt y TMDb, y algoritmos de selección de imágenes. Los componentes de presentación pura tienen menor necesidad de cobertura al aportar menos valor en la validación automatizada.

Capítulo 5
Conclusiones

El trabajo ha demostrado que los objetivos planteados son alcanzables con las tecnologías actuales de desarrollo web y sin necesidad de infraestructura de servidor propia. Los resultados obtenidos permiten extraer las siguientes conclusiones.

En primer lugar, la arquitectura Next.js App Router con React Server Components ha demostrado ser particularmente adecuada para aplicaciones de este tipo, que combinan una gran cantidad de datos externos con la necesidad de interactividad en el cliente. La separación entre la obtención de datos (servidor) y la presentación interactiva (cliente) simplifica el flujo de datos y reduce las fuentes de error, de esta forma es posible obtener los datos en paralelo de múltiples APIs en el servidor sin coste para el cliente, lo que permite reducir de forma significativa el tiempo de carga inicial frente a enfoques donde casi todo el procesamiento se realiza en el navegador.

En segundo lugar, la integración de OAuth 2.0 mediante API Routes de Next.js ha demostrado ser una estrategia robusta y segura sin necesidad de librerías de autenticación genéricas. La decisión de almacenar los tokens en cookies `httpOnly` y resolver la autenticación en servidor sigue las buenas prácticas descritas en OAuth 2.0 (RFC 6749) (Hardt, 2012) y en las recomendaciones actuales de OWASP para este tipo de flujos (OWASP, 2024). Además, la implementación manual permitió gestionar con mayor precisión tanto la renovación de tokens como los errores específicos de la API de Trakt.tv.

En tercer lugar, la estrategia de caché diferenciada por tipo de dato ha demostrado ser eficaz en todas sus capas: ISR en servidor para el catálogo, `no-store` para los datos de usuario protegidos por autenticación y una caché en cliente de cuatro niveles (memoria del proceso, sessionStorage, localStorage y deduplicación en vuelo) para las puntuaciones externas de IMDb, Trakt y OMDb en las vistas de listas, reduciendo drásticamente el número de llamadas a APIs externas.

En cuarto lugar, la integración con Plex Media Server confirma que es viable unificar en una misma aplicación el contenido local del usuario y la información procedente de servicios en la nube. Para ello, se implementó un flujo de autenticación basado en JWT con renovación automática de tokens y un mecanismo de fallback entre HTTP y HTTPS, lo que reduce fallos de conexión en entornos

domésticos con distintas configuraciones de red.

Finalmente, en el plano de diseño, la combinación de animaciones con Framer Motion, superficies semitransparentes con desenfoque de fondo (*glassmorphism*) y una navegación adaptativa según el dispositivo (barra horizontal en escritorio y *drawer* con *tab bar* en móvil) permitió construir una interfaz coherente, reconocible y más cercana a patrones de uso actuales. Estas decisiones no solo aportan diferenciación visual frente a otras plataformas de *tracking*, sino que también mejoran la legibilidad, la jerarquía de la información y la fluidez de interacción entre vistas.

5.1 Logros alcanzados

El proyecto ha cumplido todos los objetivos específicos planteados inicialmente y ha añadido funcionalidades adicionales significativas:

Objetivos iniciales cumplidos:

1) ✓ Integración completa con TMDb para metadatos de películas y series.

2) ✓ Sistema de autenticación OAuth 2.0 con Trakt.tv con refresh automático.

3) ✓ Módulo de gestión personal con sincronización bidireccional.

4) ✓ Sistema de seguimiento de episodios completo para series.

5) ✓ Listas personalizadas por usuario.

6) ✓ Historial de visionado y estadísticas de usuario.

7) ✓ Interfaz responsiva con múltiples modos de visualización.

8) ✓ Sistema de caché optimizado.

9) ✓ Despliegue en producción (Vercel).

Funcionalidades adicionales implementadas:

- Sistema de comentarios y listas de comunidad Trakt.
- Integración con Plex Media Server para biblioteca local.
- Sistema de artwork personalizado con sobrescrituras manuales.
- Integración con Letterboxd para enlaces directos.

- Componente LiquidButton con efectos Canvas avanzados.
- Sistema de tres vistas (Grid, List, Compact) con persistencia de preferencias.
- Calendario de estrenos con filtrado por fecha y región.
- Estadísticas de usuario con gráficos interactivos (Recharts).
- Manejo de errores con *graceful degradation* y *skeleton loaders*.

5.2 Discusión

El trabajo presenta algunas limitaciones que deben reconocerse.

Ausencia de tests automatizados. El proyecto no incluyó una suite de tests unitarios ni de integración durante el desarrollo. La validación se realizó exclusivamente mediante pruebas manuales. Los componentes con lógica de negocio compleja (cálculo de progreso de episodios, normalización de fechas para Trakt, estrategia de selección de *backdrops*, algoritmo de deduplicación en vuelo) serían candidatos prioritarios para la cobertura de tests unitarios y la integración de Jest y React Testing Library sería el siguiente paso natural del proyecto.

Dependencia de APIs de terceros. La plataforma depende de la continuidad y estabilidad de cinco APIs externas (TMDb, Trakt.tv, OMDb, JustWatch y Plex), ninguna de las cuales está bajo el control del proyecto. Un cambio en los términos de servicio, límites de uso o estructura de datos de cualquiera de estas APIs podría afectar a la funcionalidad de la aplicación, por lo que es una limitación estructural de la arquitectura elegida, aceptada conscientemente por las ventajas que ofrece frente a la alternativa de construir una base de datos propia. El patrón de *graceful degradation* implementado mitiga parcialmente este riesgo.

Escalabilidad del sistema de caché. El uso de `localStorage` para puntuaciones tiene un límite práctico de 5–10 MB según el navegador y este límite podría alcanzarse para listas con un gran número de elementos. La implementación del algoritmo LRU mitiga el problema pero no lo elimina completamente. Por esta razón, una migración a IndexedDB (límite 50 GB) sería recomendable para una mejor escalabilidad a largo plazo.

5.3 Análisis de riesgos y estrategias de mitigación

El estado actual de *The Show Verse* refleja un producto funcional y desplegado, pero con una exposición natural a riesgos técnicos, legales y de adopción. El enfoque seguido ha sido integrar la mitigación en la propia arquitectura y dejar identificadas las acciones pendientes para la siguiente fase.

5.3.1 Riesgos técnicos

El principal punto de exposición técnica es la dependencia de servicios externos (TMDb, Trakt.tv, OMDb, JustWatch y Plex). Para reducir el impacto de cambios o caídas parciales, la aplicación encapsula las integraciones en módulos y rutas API propias, de modo que una variación en un proveedor no obliga a modificar toda la interfaz. Además, la estrategia de caché multinivel y el patrón de *graceful degradation* ya implementados permiten mantener una experiencia usable cuando alguna fuente responde de forma lenta o incompleta.

También existe riesgo operativo en los flujos autenticados de Trakt, especialmente cuando un token expira o el usuario revoca permisos. Este escenario se mitiga con renovación de token desde el servidor y manejo explícito de errores de autorización, evitando estados incorrectos en el cliente y permitiendo la reconexión controlada.

En cuanto al rendimiento bajo carga, el proyecto parte con ventaja por el uso de Server Components, fetching paralelo e ISR en catálogo, que reducen llamadas repetidas. Como línea de refuerzo para crecimiento de usuarios, queda planificado endurecer políticas de reintento y observabilidad para anticipar límites de consumo antes de que afecten al servicio.

5.3.2 Riesgos legales y de privacidad

La aplicación trabaja con datos y recursos sujetos a condiciones de uso de terceros, por lo que la continuidad del proyecto depende de mantener conformidad con sus términos. En la práctica, esto implica conservar atribuciones visibles, evitar usos no permitidos de metadatos e imágenes, y revisar periódicamente cambios en políticas de API para adaptar el comportamiento cuando sea necesario.

Desde el punto de vista de protección de datos, el proyecto ya aplica una medida clave: los tokens se gestionan en cookies `httpOnly` y la autenticación se resuelve en servidor, minimizando exposición en cliente. No obstante, para una explotación pública más amplia sigue siendo necesario completar el marco documental de privacidad (información al usuario, gestión de consentimiento y procedimientos de eliminación/exportación de datos) para asegurar cumplimiento sostenido en contexto RGPD.

5.3.3 Riesgos de producto y adopción

En el plano de mercado, el riesgo no está en la viabilidad técnica inmediata, sino en la capacidad de diferenciarse de alternativas ya consolidadas. La mitigación realista consiste en reforzar aquello que ya aporta valor diferencial en el estado actual: integración transversal de fuentes, control de visionado por episodios, biblioteca unificada con Plex y una presentación visual más cuidada que la de herramientas tradicionales.

Existe además un riesgo de abandono temprano si el usuario no percibe rápido esa propuesta de valor. Por ello, las acciones más efectivas a corto plazo son mejorar onboarding y comunicación de funcionalidades clave, además de mantener una evolución incremental centrada en utilidades concretas antes que en expansiones de gran alcance.

5.3.4 Síntesis operativa

Con la arquitectura actual, el riesgo residual es asumible para una fase MVP avanzada: la base técnica es sólida, las mitigaciones críticas ya están parcialmente incorporadas en diseño y queda como prioridad cerrar el bloque de cumplimiento legal y observabilidad operativa para escalar con mayor garantías.

A partir del trabajo realizado, se identifican las siguientes líneas de desarrollo futuro:

Implementación de Progressive Web App (PWA). La conversión de la aplicación en una PWA, mediante la adición de un *Service Worker* y un *manifest* web, permitiría la instalación en dispositivos móviles y habilitaría un modo de funcionamiento *offline* básico. El *Service Worker* podría cachear las imágenes de pósters y los datos de catálogo para consulta sin conexión. Next.js 13+ soporta PWA mediante el plugin `next-pwa`.

Sistema de recomendaciones personalizado. El historial de visionado acumulado por el usuario constituye un conjunto de datos con potencial para la implementación de algoritmos de recomendación colaborativa o basada en contenido. TensorFlow.js en el cliente o una función *serverless* con Python + scikit-learn permitirían generar recomendaciones personalizadas complementarias a las de Trakt. La técnica de *matrix factorization* (SVD) o redes neuronales de *collaborative filtering* serían candidatas naturales.

Expansión de las funcionalidades sociales. La comparación del historial entre usuarios, las recomendaciones de amigos y la visualización del perfil público de otros usuarios de Trakt son funcionalidades que la API de Trakt soporta pero no han sido implementadas en la versión actual. Un módulo de «Amigos» con feed de actividad reciente aumentaría significativamente el componente social de la plataforma.

Backend propio y base de datos local. A largo plazo, la eliminación de la dependencia total de Trakt.tv como fuente de verdad requeriría la implementación de un *backend* propio con base de datos (PostgreSQL o MySQL) que almacenara el historial y las preferencias localmente. Esto permitiría además implementar funcionalidades propias como sesiones de visionado grupal (*watch parties*), sistemas de *achievements* y estadísticas avanzadas no disponibles en Trakt.

Integración con más plataformas. La arquitectura modular del sistema de integración API permitiría añadir fácilmente nuevas fuentes: MyAnimeList para anime, Goodreads para literatura

adaptada, Metacritic para puntuaciones adicionales, o incluso Spotify/Apple Music para bandas sonoras. Cada integración seguiría el mismo patrón: API Route de *proxy*, caché multinivel, *graceful degradation*.

Sistema de notificaciones push. Notificaciones web (*Web Push API*) para alertar al usuario de:

- Nuevos episodios de series seguidas.
- Respuestas a comentarios en Trakt.
- Nuevas películas añadidas al catálogo que coinciden con sus gustos.
- Amigos que han visto contenido que el usuario tiene en pendientes.

Análisis avanzado con IA. Integración con modelos de lenguaje (GPT-4, Claude) para:

- Generación de resúmenes personalizados de temporadas.
- Respuestas a preguntas sobre la trama sin spoilers.
- Recomendaciones explicadas («Te recomiendo X porque...»).
- Detección de contenido similar por temas y no solo por género.

Bibliografía

CNMC. (2024). *Informe sobre el consumo de vídeo en España*. https://www.cnmc.es/

Google Developers. (2024). *Web Vitals — Essential metrics for a healthy site*. https://web.dev/vitals/

Hardt, D. (2012). *RFC 6749: The OAuth 2.0 Authorization Framework*. IETF. https://www.rfc-editor.org/rfc/rfc6749

OWASP. (2024). *OWASP Top 10: Broken Authentication*. https://owasp.org/www-project-top-ten/

Schwartz, B. (2004). *The Paradox of Choice: Why More Is Less*. Ecco/HarperCollins.

Statista. (2024). *Number of Netflix subscribers worldwide from 1st quarter 2013 to 4th quarter 2023*. https://www.statista.com/statistics/250934/quarterly-number-of-netflix-streaming-subscribers-worldwide/

Glosario

API	*Application Programming Interface*. Interfaz de programación de aplicaciones.
CDN	*Content Delivery Network*. Red de distribución de contenidos.
CLS	*Cumulative Layout Shift*. Métrica de estabilidad visual de Core Web Vitals.
CSRF	*Cross-Site Request Forgery*. Tipo de ataque web que fuerza acciones no deseadas.
FID	*First Input Delay*. Tiempo de respuesta al primer input del usuario.
GraphQL	Lenguaje de consulta para APIs desarrollado por Meta.
httpOnly	Atributo de cookie que impide su acceso desde JavaScript del navegador.
ISR	*Incremental Static Regeneration*. Estrategia de caché de Next.js.
JWT	*JSON Web Token*. Estándar para la transmisión segura de información entre partes.
LCP	*Largest Contentful Paint*. Métrica de velocidad de carga de Core Web Vitals.
OAuth	*Open Authorization*. Protocolo estándar de delegación de acceso.
OMDb	*Open Movie Database*. Base de datos de películas con datos de IMDb.
REST	*Representational State Transfer*. Estilo arquitectónico para servicios web.
RSC	*React Server Components*. Componentes React que se ejecutan exclusivamente en servidor.
SPA	*Single Page Application*. Aplicación web de una sola página.
SPR	*Serverless Page Rendering*. Renderizado de páginas sin servidor propio.
SSR	*Server-Side Rendering*. Renderizado de páginas en el servidor.
TMDb	*The Movie Database*. Base de datos de películas y series con API pública.
TTL	*Time To Live*. Tiempo de vida de un elemento en caché.
XSS	*Cross-Site Scripting*. Tipo de ataque de inyección de código en el navegador.

Anexo

Variables de entorno del proyecto

La siguiente tabla recoge todas las variables de entorno necesarias para el despliegue de *The Show Verse*.

Variable	**Descripción**	**Requerida**
`NEXT_PUBLIC_TMDB_API_KEY`	API Key de TMDb v3	Sí
`TMDB_SESSION_ID`	Session ID para operaciones de cuenta TMDb	Cuentas
`NEXT_PUBLIC_TRAKT_CLIENT_ID`	Client ID de la aplicación Trakt	Sí
`TRAKT_CLIENT_SECRET`	Client Secret de Trakt (solo servidor)	Sí
`TRAKT_REDIRECT_URI`	URI de callback OAuth registrado en Trakt	Sí
`OMDB_API_KEY`	API Key de OMDb	Para IMDb/RT
`PLEX_URL`	URL del servidor Plex	Para Plex
`PLEX_URLS`	URLs adicionales Plex (separadas por coma)	Opcional
`PLEX_TOKEN`	Token de autenticación Plex (modo simple)	Modo simple
`PLEX_JWT_PRIVATE_KEY`	Clave privada RSA/Ed25519 (modo JWT)	Modo JWT
`PLEX_JWT_KID`	Key ID del JWK (modo JWT)	Modo JWT
`PLEX_CLIENT_IDENTIFIER`	Identificador único de la app en Plex	Modo JWT
`PLEX_MACHINE_IDENTIFIER`	Machine ID del servidor Plex (fallback)	Opcional

Tabla 5.2: Variables de entorno del proyecto

Resumen del sistema de caché multicapa

Nivel	**Almacenamiento**	**TTL**
1 – ISR servidor	Data Cache de Next.js	10 min / 1 h
2 – CDN Vercel	Edge Network	1 – 24 h
3 – Memoria proceso	`Map` en `globalThis`	24 h (éxito) / 60 s (fallo)
4 – `sessionStorage`	Navegador (sesión)	24 h
5 – `localStorage`	Navegador (persistente)	30 días
6 – `Map` en módulo	Memoria del cliente (tab)	10 min / 45 s

Tabla 5.3: Sistema de caché multicapa